

C1

A 小水獭和主定理

小水獭正在学习分治和递归，在计算复杂度时用到了主定理，希望你可以帮助它写一个程序来计算满足特定形式递归式的算法复杂度。

具体来说，给定正整数 a, b, k ，所求递归式为：

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^k)$$

根据主定理，有：

$$T(n) = \begin{cases} O(n^{\log_b a}), & \log_b a > k \\ O(n^k \log n), & \log_b a = k \\ O(n^k), & \log_b a < k \end{cases}$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 2 \times 10^5$)，表示数据组数。

对于每组数据，一行三个正整数 a, b, k ($1 \leq a, k \leq 10^9, 2 \leq b \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行：

- 若 $T(n) = O(n^{\log_b a})$ ，输出 $n^{\{\log_ba\}}$ 。
- 若 $T(n) = O(n^k \log n)$ ，输出 $n^k \log n$ 。
- 若 $T(n) = O(n^k)$ ，输出 n^k 。

```
#include <bits/stdc++.h>
using namespace std;
int t, a, b, k;
int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%d", &a, &b, &k);
        long long x = 1;
        if(b > 1){
            for(int i = 1; i <= k && x <= a; i++) x *= b;
        }
        if(a == x) puts("n^k\\log n");
        else if(a > x) puts("n^{\log\_ba}");
        else puts("n^k");
    }
}
```

B 一元多项式计算器

Zhoues 对《算法导论》这本书简直是爱不释手，当他翻到这本书的第 23 页准备暗中观察时，立马被 Horner 规则给吸引到了，于是他打算做一个基于 Horner 规则的一元多项式计算器，帮助他进行一类特殊的二元多项式。

具体来说，给定如下两个一元多项式：

$$\sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n$$
$$\sum_{i=0}^m b_i y^i = b_0 + b_1 y + b_2 y^2 + \dots + b_m y^m$$

定义如下的二元多项式：

$$f(x,y) = \left(\sum_{i=0}^n a_i x^i \right) \left(\sum_{i=0}^m b_i y^i \right)$$

你需要处理 q 次计算，第 i 次计算给定两个变量值 X_i 和 Y_i ，你需要求解 $f(X_i, Y_i) \bmod 10007$ 。

输入格式

第一行一个正整数 n ($1 \leq n \leq 3 \times 10^4$)，表示第一个一元多项式的次数。

第二行 $n + 1$ 个非负整数 $a_0, a_1, \dots, a_n$ ($0 \leq a_i \leq 10^3$)，表示第一个一元多项式的系数。

第三行一个正整数 m ($1 \leq m \leq 3 \times 10^4$)，表示第二个一元多项式的次数。

第四行 $m + 1$ 个非负整数 $b_0, b_1, \dots, b_m$ ($0 \leq b_i \leq 10^3$)，表示第二个一元多项式的系数。

第五行一个正整数 q ($1 \leq q \leq 2.5 \times 10^3$)，表示计算的次数。

接下来 q 行，第 i 行两个非负整数 X_i, Y_i ($0 \leq X_i, Y_i \leq 10^4$)，表示第 i 次计算给定的两个变量值。

输出格式

输出 q 行，第 i 行一个非负整数，表示 $f(X_i, Y_i) \bmod 10007$ 。

```
#include<stdio.h>
#define M 10007
long long a[1000001],b[1000001];
long long n,m,x,y,i,t,c;
long long Horner(long long *a,long long n,long long x) {
    if(n<1)
        return a[0] % M;
    else
        return (a[0] % M + (Horner(a+1,n-1,x) * x) % M) % M;
}

int main() {
    scanf("%lld",&n);
    for(i=0; i<=n; i++) {
        scanf("%lld",&a[i]);
    }
    scanf("%lld",&m);
    for(i=0; i<=m; i++) {
        scanf("%lld",&b[i]);
    }
    scanf("%lld",&c);
    while(c--)
    {
        scanf("%lld %lld",&x,&y);
        printf("%lld\n",( Horner(a,n,x) % M) * (Horner(b,m,y) % M) ) % M );
    }
    return 0;
}
```

C 小水獭和模运算

小水獭正在学习模运算，它通过 Hint 发现 $ab \bmod c = (a \bmod c)(b \bmod c) \bmod c$ 。

现在小水獭有 n 个非负整数 $a_1, a_2, \dots, a_n$ ，它定义 $f(i)$ 为除了 a_i 以外的 $n - 1$ 个非负整数的乘积对 $10^9 + 6$ 取模后的结果，形式化地说：

$$f(i) = \left(\prod_{j=1, j \neq i}^n a_j \right) \bmod (10^9 + 6)$$

小水獭希望你可以帮助它写一个程序计算 $f(1), f(2), \dots, f(n)$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($2 \leq n \leq 10^5$)，含义同题目描述。

第二个 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 10^9 + 6$)，含义同题目描述。

输出格式

对于每组数据，输出一行 n 个非负整数，表示 $f(1), f(2), \dots, f(n)$ 。

```
#include <bits/stdc++.h>
#define maxn 100086
using namespace std;
const int p = 1e9 + 6;
int t, n;
int a[maxn], f[maxn], g[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        f[0] = 1; for(int i = 1; i <= n; i++) f[i] = 1ll * f[i - 1] * a[i] % p;
        g[n + 1] = 1; for(int i = n; i >= 1; i--) g[i] = 1ll * g[i + 1] * a[i] % p;
        for(int i = 1; i <= n; i++) printf("%d ", 1ll * f[i - 1] * g[i + 1] % p); puts("");
    }
}
```

D 日期统计

本题中所使用的纪年体系为公元，仅考虑年份位于 [2000, 9999] 的日期。

所有日期均可以表示为一个唯一的 8 位十进制数字，前四位表示年，中间两位表示月，最后两位表示日，例如 2022 年 9 月 6 日可以表示为 20220906。

一个日期是好的当且仅当这个日期按上述规则表示为 8 位十进制数字 $a_1a_2a_3a_4a_5a_6a_7a_8$ 时满足下面两个条件中至少一条：

- $\exists i \in \{1, 2, \dots, 8\}$ 使得 $\forall 1 \leq j < i, a_j \leq a_{j+1}$ 并且 $\forall i < j \leq n, a_j \leq a_{j-1}$ 。
- $\exists i \in \{1, 2, \dots, 8\}$ 使得 $\forall 1 \leq j < i, a_j \geq a_{j+1}$ 并且 $\forall i < j \leq n, a_j \geq a_{j-1}$ 。

比如 20000123 就是一个好的日期。小水獭想知道对于给定两个日期，这两个日期之间（包含这两个日期）共有多少个好日期。

输入格式

第一行一个 8 位十进制数字，表示第一个日期。

第二行一个 8 位十进制数字，表示第二个日期。

保证第一个日期在时间上位于第二个日期之前，并且两个日期均是年份位于 [2000, 9999] 的合法日期。

输出格式

一行一个非负整数，表示两个日期间好日期的个数。

```

#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 300005
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

char s[10], t[10];
int y[2], m[2], d[2];
int mx[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

bool check(int a, int b, int c){
    int s[10], t = 0;
    for(int i = 1; i <= 2; i++) s[++t] = c % 10, c /= 10;
    for(int i = 1; i <= 2; i++) s[++t] = b % 10, b /= 10;
    for(int i = 1; i <= 4; i++) s[++t] = a % 10, a /= 10;
    for(int i = 1; i <= 8; i++){
        bool f = 1;
        for(int j = i + 1; j <= 8; j++) if(s[j] < s[j - 1]) f = 0;
        for(int j = i - 1; j >= 1; j--) if(s[j] < s[j + 1]) f = 0;
        if(f) return 1;
        f = 1;
        for(int j = i + 1; j <= 8; j++) if(s[j] > s[j - 1]) f = 0;
        for(int j = i - 1; j >= 1; j--) if(s[j] > s[j + 1]) f = 0;
        if(f) return 1;
    }
    return 0;
}

bool isleap(int x){
    return (x % 4 == 0 && x % 100 != 0) || (x % 400 == 0);
}

void solve(){
    scanf("%s", s + 1), scanf("%s", t + 1);
    for(int i = 1; i <= 4; i++) y[0] = y[0] * 10 + s[i] - '0';
    for(int i = 5; i <= 6; i++) m[0] = m[0] * 10 + s[i] - '0';
    for(int i = 7; i <= 8; i++) d[0] = d[0] * 10 + s[i] - '0';
    for(int i = 1; i <= 4; i++) y[1] = y[1] * 10 + t[i] - '0';
    for(int i = 5; i <= 6; i++) m[1] = m[1] * 10 + t[i] - '0';
    for(int i = 7; i <= 8; i++) d[1] = d[1] * 10 + t[i] - '0';
    int ans = 0;
    while(y[0] != y[1] || m[0] != m[1] || d[0] != d[1]){
        ans += check(y[0], m[0], d[0]);
        d[0]++;
        if(d[0] > mx[m[0]] + (m[0] == 2 && isleap(y[0]))) m[0]++, d[0] = 1;
        if(m[0] > 12) m[0] = 1, y[0]++;
    }
    ans += check(y[1], m[1], d[1]);
}

```

```

        printf("%d\n", ans);
    }

    int main(){
        int t = 1, cnt = 1;
        while(t--){
            solve();
        }

        return 0;
    }

```

E 单词游戏-青春版

四六级考试将至，the_ignorant、Zhoues、2sozx 为提高英语水平，便想了一个单词游戏。这个游戏一共有 t 轮，且每轮游戏相互独立。

每轮游戏中，三人每人说出 n 个单词。最终，对于每个被说出的英文单词：

- 若其恰好被一人说出，则此人加 3 分。
- 若其恰好被两人说出，则两人各加 1 分。
- 若其被三人均说出，则三人均不加分。

the_ignorant 想知道他们每轮最后得了多少分。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示游戏轮数。

对于每轮游戏，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示单词数量。

接下来三行，每行 n 个互不相同的非空小写字母单词，且长度不超过 4，表示每个人此轮说出的单词。

输出格式

对于每轮游戏，输出一行三个非负整数，代表三个人的最后得分。

```

#include<bits/stdc++.h>
using namespace std;
int a[600000][3];
char s[10];
int ans[3];
int tr(char s[])
{
    int res=0;
    int len=strlen(s);
    for(int i=0;i<len;i++)
    {
        res=res*27+s[i]-'a'+1;
    }
    return res;
}
int main()
{
    int t,n;
    scanf("%d",&t);
    while(t--)
    {
        memset(a,0,sizeof(a));
        memset(ans,0,sizeof(ans));
        scanf("%d",&n);
    }
}

```

```

for(int j=0;j<3;j++)
{
    for(int i=0;i<n;i++)
    {
        scanf("%s",s);
        a[tr(s)][j]++;
    }
}
for(int i=0;i<27*27*27*27;i++)
{
    if(a[i][0]&&a[i][1]&&a[i][2])
    {
        continue;
    }
    else if(a[i][0]&&a[i][1])
    {
        ans[0]++;
        ans[1]++;
    }
    else if(a[i][0]&&a[i][2])
    {
        ans[0]++;
        ans[2]++;
    }
    else if(a[i][1]&&a[i][2])
    {
        ans[1]++;
        ans[2]++;
    }
    else if(a[i][0])
    {
        ans[0]+=3;
    }
    else if(a[i][1])
    {
        ans[1]+=3;
    }
    else if(a[i][2])
    {
        ans[2]+=3;
    }
}
for(int i=0;i<3;i++)
{
    printf("%d ",ans[i]);
}
puts("");
}
}

```

F 逆序 k 倍对

Zhoues 很喜欢研读《算法导论》，有一天在暗中观察该书第 24 页的时候，想到了一个问题并需要你来帮他解决，问题如下：

- 给定一个序列 $a_1, a_2, \dots, a_n$ 和一个正整数 k ，如果 $1 \leq i < j \leq n$ 且 $a_i > k \cdot a_j$ 我们就将 (i, j) 称作一个**逆序 k 倍对**。请你计算序列中逆序 k 倍对的个数。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^5, 1 \leq k \leq 10$)，含义同题目描述。

第二个 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i < 2^{31}$)，含义同题目描述。

对于得分占比 10% 的测试点，保证 $1 \leq n \leq 100$ 。

输出格式

一行一个非负整数，表示序列中逆序 k 倍对的个数。

```
#include<stdio.h>
int a[1000002], n, k, num;

long long reversePairsRecursive(int* nums, int left, int right) {
    if (left == right) {
        return 0;
    } else {
        int mid = (left + right) / 2;
        int n1 = reversePairsRecursive(nums, left, mid);
        int n2 = reversePairsRecursive(nums, mid + 1, right);
        long long ret = n1 + n2;

        int i = left;
        int j = mid + 1;
        while (i <= mid) {
            while (j <= right && (long long)nums[i] > k * (long long)nums[j])
                j++;
            ret += (j - mid - 1);
            i++;
        }

        int sorted[right - left + 1];
        int p1 = left, p2 = mid + 1;
        int p = 0;
        while (p1 <= mid || p2 <= right) {
            if (p1 > mid) {
                sorted[p++] = nums[p1++];
            } else if (p2 > right) {
                sorted[p++] = nums[p2++];
            } else {
                if (nums[p1] < nums[p2]) {
                    sorted[p++] = nums[p1++];
                } else {
                    sorted[p++] = nums[p2++];
                }
            }
        }
        for (int i = 0; i < right - left + 1; i++) {
            nums[left + i] = sorted[i];
        }
        return ret;
    }
}

int reversePairs(int* nums, int numsSize) {
```

```

        if (numssize == 0) {
            return 0;
        }
        return reversePairsRecursive(nums, 0, numssize - 1);
    }

    int main() {
        scanf("%d %d",&n,&k);
        for(int i = 0; i < n ; i++) {
            scanf("%d",&a[i]);
        }
        printf("%lld\n",reversePairs(a,n));
        return 0;
    }
}

```

G 暗中观察

Zhoues 这个同学估计大家都不怎么认识，但是这不重要，重要的是你只需要知道他有一项特殊的能力——**暗中观察**。他经常通过这个能力轻松看透一些事物背后的算法原理。

虽然学校因为疫情封校，但是却校园内却盛行一款游戏：

- 该游戏中有小 A 和小 B 两个角色，初始给定两个正整数 n 和 k ，小 A 和小 B 轮流对 n 减去 k 的任意非负整数次幂（即 $k^i, i \in \mathbb{N}$ ），但需要保证 n 始终非负，最终在自己回合将 n 变为 0 的一方获胜。

每次游戏开始都是小 A 先手，且小 A 小 B 两个人都会采取最优策略，那么每轮谁会赢呢？

Zhoues 暗中观察了几场这两款游戏的对局，已经看透了背后的算法奥秘，现在你只要告诉他游戏的初始情况，他就可以直接告诉你最后的赢家是谁！

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，一行两个正整数 n, k ($1 \leq n, k \leq 10^3$)，含义同题目描述。

输出格式

对于每组数据，输出一行：

- 若小 A 获胜，输出 `~A~`。
- 若小 B 获胜，输出 `~B~`。

```

#include <bits/stdc++.h>
#define maxn 1005
using namespace std;

int ans[maxn], vis[maxn];
int n, k;

int play(int x) {
    if(!x) return 0;
    if(vis[x]) return ans[x];
    vis[x] = 1;
    int res = 0;
    for(int i = 1; i <= x; i *= k) {
        res |= !play(x - i);
    }
    ans[x] = res;
    return res;
}

int main() {
    int t;
    scanf("%d",&t);
}

```

```

while(t--){
    scanf("%d%d",&n,&k);
    if(k == 1 || n < k){
        puts(n%2==0 ? "~B~" : "~A~");
    } else {
        for(int i = 0; i <= n; i++){
            vis[i] = 0, ans[i] = 0;
        }
        puts(play(n) ? "~A~" : "~B~");
    }
}
return 0;
}

```

H XIAO7 想要变成猫猫

XIAO7 每天都在感叹：「我什么时候才能变成猫猫呢，变成猫猫就不用努力了(=> <=)！」

于是天上传来一个声音：「如果你能答对以下若干个问题，你就可以变成猫猫。」

如果 XIAO7 没有如愿以偿地变成猫猫，她就会给大家的 Java 加难度，所以请你帮帮她(=^ ^=)!

具体来说，给定一个长度为 n 的序列 $x_1, x_2, \dots, x_n$ 和若干个问题，每个问题包含四个整数 a, b, c, d ，你需要找到一个最小的整数 $i \in [1, n]$ 满足：

$$a \times (x_i \oplus i)^2 + b \times (x_i \oplus i) \times x_i + c \times x_i^2 + d = 0$$

其中 $\oplus$ 表示按位异或运算，所有数按照 64 位补码进行运算。

为了防止奇怪的离线算法，输入数据进行了加密，假设你读到的四个整数为 a_0, b_0, c_0, d_0 ，则解密方法为

$a = a_0 + lastAns, b = b_0 + lastAns, c = c_0 + lastAns, d = d_0 + lastAns$ ，其中 $lastAns$ 为前一个问题的答案，对于第一个问题 $lastAns = 0$ 。

输入格式

第一行一个整数 n ($1 \leq n \leq 10^5$)，表示序列的长度。

第二行 n 个整数 $x_1, x_2, \dots, x_n$ ($-10^6 \leq x_i \leq 10^6$)，表示给定的序列元素。

接下来若干行，每行四个整数 a_0, b_0, c_0, d_0 ($-10^6 \leq a_0, b_0, c_0 \leq 10^6, -10^{18} \leq d_0 \leq 10^{18}$) 表示一个问题，对于每个问题保证至少存在一个 i 满足条件。

在你回答正确的前提下，当解密出的 a, b, c, d 均为 0 时一定是最后一个问题，此时所有数据都已成功读入。**你不需要对最后一个问题进行输出。**

保证问题的数量不超过 4×10^5 。

输出格式

若干行，第 i 行一个正整数表示第 i 个问题的答案。

注意，解密出 0 0 0 0 的最后一个问题不需要输出。

```

#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 500005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
ll getll(){
    char c;bool flag=0;ll num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}

```

```

int a[N], b[N], c[N], ans[N];
ll d[N], x[N];
int main(){

    int n = getint(), m = 0;
    for(int i = 1; i <= n; i ++){
        x[i] = getint();

        while(~scanf("%d", &a[++ m]))
            b[m] = getint(), c[m] = getint(), d[m] = getll();
        m --;

        ll j = -a[m --];

        for(int i = m; i; i --){
            ans[i] = j;
            ll A = 1ll * (x[j] ^ j) * (x[j] ^ j);
            ll B = 1ll * (x[j] ^ j) * x[j];
            ll C = 1ll * x[j] * x[j];
            ll t1 = a[i] * A + b[i] * B + c[i] * C + d[i];
            ll t2 = A + B + C + 1;

            j = -t1 / t2;

        }

        for(int i = 1; i <= m; i ++){
            printf("%d\n", ans[i]);
        }
        return 0;
    }
}

```

I 异或游戏

the_ignorant 的单词游戏进行得不是很顺利，于是为了提高数学水平，他又想了一个异或游戏。

他想要找出一个序列，这个序列由 n 个互不相同的非负整数组成，且这 n 个非负整数的按位异或值为 0。

可以证明，在 $n \geq 3$ 时，至少存在一个满足以上条件的序列。但 the_ignorant 不满足于此，他想找出所有满足以上条件的序列中，序列中最大数的最小值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，一行一个正整数 n ($3 \leq n \leq 10^6$)，表示序列的长度。

输出格式

对于每组数据，输出一行一个数，代表所有可能序列中，序列中最大数的最小值。

```

#include<bits/stdc++.h>
using namespace std;
int solve(int x)
{
    int res=0;
    while(x>0)
    {
        x>>=1;
        res++;
    }
}

```

```

        return res;
    }
    int main()
    {
        int t,n;
        scanf("%d",&t);
        while(t--)
        {
            scanf("%d",&n);
            if(n%4==0)
            {
                printf("%d\n",n-1);
            }
            else if(n%4==3)
            {
                printf("%d\n",n);
            }
            else if(n%4==1)
            {
                printf("%d\n",n);
            }
            else if(n%4==2)
            {
                if(solve(n+2)>solve(n))
                {
                    printf("%d\n",n+3);
                }
                else
                {
                    printf("%d\n",n+2);
                }
            }
        }
    }
}

```

J 小水獭和二叉树

小水獭正在学习二叉树，它希望世界上所有的树都是二叉树。

具体来说，小水獭有一棵 n 个结点的树 T ，每条边上有一个非负长度。小水獭想要构造一棵 m 个结点的树 T' ，其满足如下限制：

- $n \leq m \leq 2n$ 。
- 每条边上有一个非负长度。
- 任意节点度数不超过 3。
- 对任意节点 $1 \leq i < j \leq n$ ， i 和 j 在 T 和 T' 上的最短路距离相同。

你能帮助小水獭实现它的愿望嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($2 \leq n \leq 100$)，表示树 T 的节点数。

接下来 $n - 1$ 行，每行三个整数 x, y, z ($1 \leq x, y \leq n$, $x \neq y$, $0 \leq z \leq 100$)，表示 T 中存在边 (x, y) 且长度为 z 。

保证给出的边集构成一棵树。

输出格式

对于每组数据，首先输出一行一个正整数 m ($n \leq m \leq 2n$)，表示树 T' 的节点数。

接下来输出 $m - 1$ 行，每行三个整数 x, y, z ($1 \leq x, y \leq m$, $x \neq y$, $0 \leq z \leq 100$)，表示 T' 中存在边 (x, y) 且长度为 z 。

如果有多种可能的答案，你可以输出任意一种。

C2

A 平面最近点对 I

wzf 教给了莫卡很多计算几何的知识，并丢给她一道题作为课后习题。

二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，定义两点 (x_i, y_i) 和 (x_j, y_j) 的距离为 $\max(|x_i - x_j|, |y_i - y_j|)$ 。

wzf 要求莫卡求出任意两点间距离的最小值，即：

$$\min_{1 \leq i < j \leq n} \max(|x_i - x_j|, |y_i - y_j|)$$

但是这道题对莫卡来说太难了，于是她想请你帮她解决这道难题。

输入格式

第一行一个正整数 t ($1 \leq t \leq 30$)，表示数据组数。

对于每组数据，一行一个正整数 n ($2 \leq n \leq 10^3$)，表示点的数量。

接下来 n 行，第 i ($1 \leq i \leq n$) 行两个整数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 10^5$)，表示对应点的坐标。

输出格式

对于每组数据，输出一行一个非负整数，表示任意两点间距离的最小值。

```
#include<bits/stdc++.h>
#define maxn 1005
using ll = long long;
using namespace std;
int t, n, x[maxn], y[maxn];

int main() {
    scanf("%d",&t);
    while(t--) {
        scanf("%d",&n);
```

```

        for(int i = 1; i <= n; i++) scanf("%d", x + i), scanf("%d", y + i);
        int ans = 200000;
        for(int i = 1; i <= n; i++) for(int j = 1; j < i; j++) ans = min(ans,
max(abs(x[i] - x[j]), abs(y[i] - y[j])));
        printf("%d\n", ans);
    }
}

```

B 简单的堆

本题必须使用 C 语言提交。

题目描述

本题是堆的模板题，你的任务是实现一个大根堆，即根节点为所有元素最大值的堆，需要支持插入元素、查询堆顶元素、删除堆顶元素三个操作。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示操作的数量。

接下来 n 行，每行为一个操作，格式如下：

- 1 x ：向堆中插入元素 x ($1 \leq x \leq 10^9$)。
- 2：删除堆顶元素。
- 3：查询堆顶元素。

数据保证后两种操作时堆非空。

输出格式

对于每次查询堆顶元素时，输出一行一个正整数，表示此时堆顶元素的值。

在所有操作结束后，将堆中的元素从大到小依次输出到一行中。

```

#include<stdio.h>
#define maxn 1000005
#define inf 0x3f3f3f3f
#define ls(x) (x << 1)
#define rs(x) (x << 1 | 1)

int n, sz, a[maxn];

void swap(int *a, int *b) {
    int t = *a;
    *a = *b, *b = t;
}

void maxHeapify(int x) {
    int l = ls(x), r = rs(x), larg;
    if(l <= sz && a[l] > a[x]) larg = l;
    else larg = x;
    if(r <= sz && a[r] > a[larg]) larg = r;
    if(larg != x) swap(&a[x], &a[larg]), maxHeapify(larg);
}

void increaseKey(int i, int key) {
    a[i] = key;
    while(i > 1 && a[i / 2] < a[i]) {
        swap(&a[i / 2], &a[i]);
        i = i / 2;
    }
}

```

```

    }
}

void insert(int x) {
    sz += 1;
    a[sz] = -inf;
    increaseKey(sz, x);
}

int top() {
    return a[1];
}

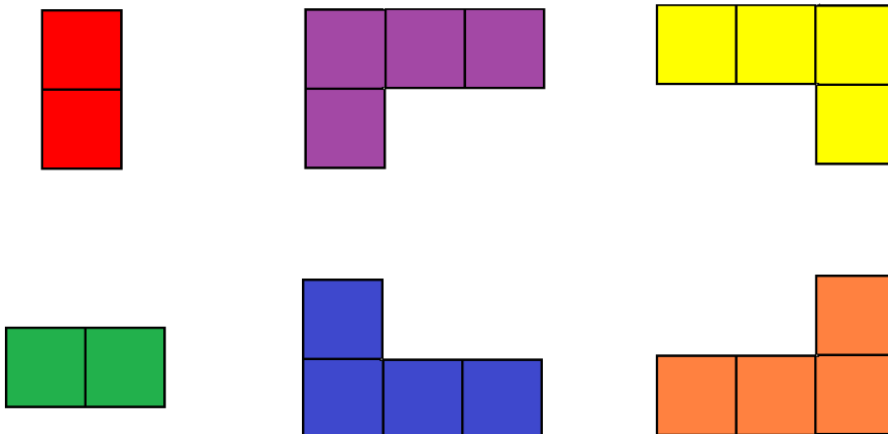
void pop() {
    if(sz < 1) exit(1);
    a[1] = a[sz];
    sz -= 1;
    maxHeapify(1);
}

int main() {
    scanf("%d", &n);
    int i;
    for(i = 1; i <= n; i++) {
        int op, x;
        scanf("%d", &op);
        if(op == 1) {
            scanf("%d", &x);
            insert(x);
        }
        if(op == 2) pop();
        if(op == 3) printf("%d\n", top());
    }
    while(sz) {
        printf("%d ", top()), pop();
    }
}

```

C 简单的铺瓷砖

有一个宽度为 2，长度为 n 的地板需要贴上瓷砖，有以下 6 种形状的瓷砖可供选择：



瓷砖不能**旋转、翻转、重叠、切割**，且必须恰好把地板**铺满**，可以结合输入输出样例理解。

the_ignorant 想知道一共有多少种不同的铺瓷砖方案，两种方案不同当且仅当存在一个位置上的瓷砖颜色不同。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^6$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，表示地板的长度。

输出格式

对于每组数据，输出一行一个非负整数，表示贴满瓷砖的方案数对 998 244 353 取模后的结果。

```
#include<bits/stdc++.h>
#define maxn 1000005
using ll = long long;
using namespace std;
const int mod = 998244353;

inline ll read() {
    ll x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

int t, tot;
int f[maxn];

int main() {
    f[0] = 1;
    for(int i = 1; i <= 1000000; i++) {
        f[i] = f[i - 1];
        if(i > 1) f[i] = (f[i] + f[i - 2]) % mod;
        if(i > 2) f[i] = (f[i] + 411 * f[i - 3]) % mod;
```

```

        if(i > 3) f[i] = (f[i] + 211 * f[i - 4]) % mod;
    }
    t = read();
    while(t--) printf("%d\n", f[read()]);
}

```

D 小水獭和随机树（简单版）

小水獭正在观摩一棵 n 个节点的树，其中每个点的权值来自一个随机数生成器，树的结构也在代码中直接钦定。它想知道对于每个点来说，所有与其距离小于 k 的祖先中（包括其本身），权值最大点的权值是多少？它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
int fa[maxn];
unsigned a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 2; i <= n; i++) fa[i] = i - 1;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        for(int i = 1; i <= n; i++){
            unsigned mx = 0;
            int x = i;
            for(int j = 1; j <= k; j++){
                mx = max(mx, a[x]);
                if(x == 1) break;
                x = fa[x];
            }
            ans += mx ^ i;
        }
        printf("%u\n", ans);
    }
}

```

其中 `fa[i]` 表示编号为 `i` 的节点的父节点编号。

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行三个整数 $n, k, seed$ ($1 \leq n \leq 10^7$, $1 \leq k \leq n$, $1 \leq seed < 2^{32}$)，表示树的节点数、距离参数和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand() {
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

```

```

}

int t, n, k;
int fa[maxn];
unsigned a[maxn];
int q[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 1; i <= n; i++) fa[i] = i - 1;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        int l = 1, r = 0;
        for(int i = 1; i <= n; i++) {
            while(l <= r && a[i] >= a[q[r]]) r--;
            q[++r] = i;
            if(i - k == q[l]) l++;
            ans += a[q[l]] ^ i;
        }
        printf("%u\n", ans);
    }
}

```

E XIAO7 和兔子

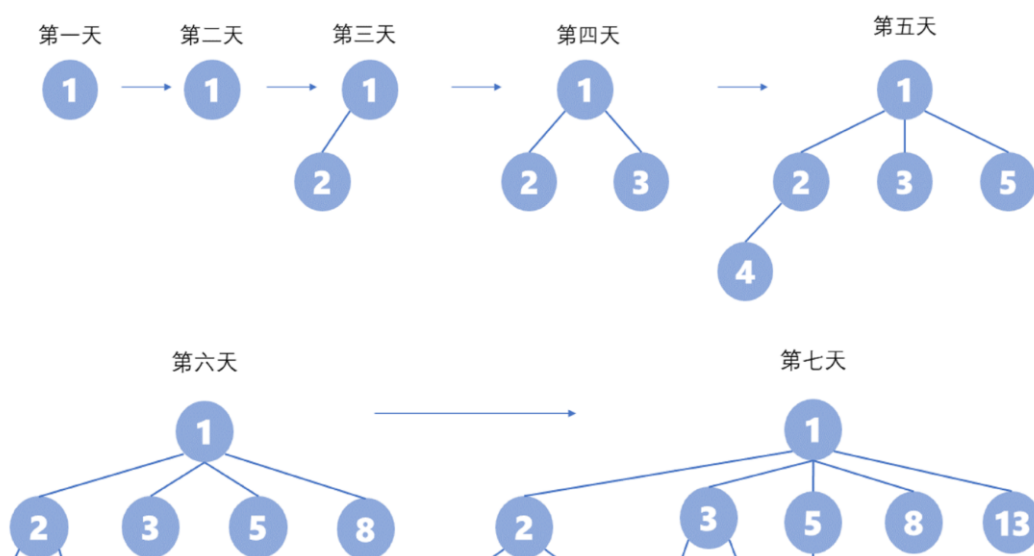
即使上周才下了猫粮雨，XIAO7 的猫粮还是不够吃了，于是她决定在猫娘乐园里养一些兔子吃。

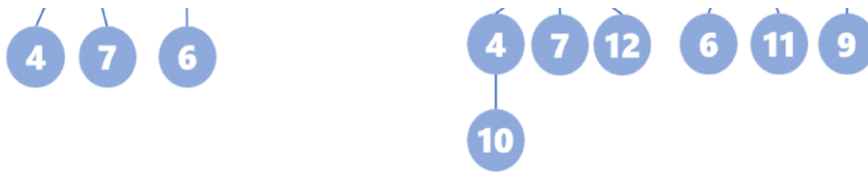
众所周知，兔子的繁殖速率很快，能很好地解决猫粮不够的问题，本题中**假设兔子不会死亡**。

第一天，XIAO7 有一对出生一天的幼年小兔子，小兔子会在**出生后的第二天**起变得成熟，并生育一对新的兔子（即在第 x 天生出的兔子，从 $x + 2$ 天起开始生育）。

XIAO7 懒得给兔子们起名字，因此她决定按兔子的出生顺序给每对兔子编号，假设最初的第一对兔子的编号为 1，如果有两对兔子在同一天出生，那么**父母编号越小的兔子编号越大**。

前 7 天的情况如下图所示：





猫娘「花叶」对此非常难过：「兔兔这么可爱，我不要吃兔兔~」

XIAO7 告诉她「如果你能答对以下若干个问题，我就把兔子们都放了。」

请你帮帮猫娘「花叶」和可爱的兔子们！

具体来说，XIAO7 想知道在 $998^{244}353$ 天后编号为 a 和编号为 b 的兔子的**最近公共祖先**的编号是多少。最近公共祖先是指两对兔子所共有的祖先中，离他们的距离之和最近的一对兔子。

以上图举例来说：

- 编号为 10 和 12 的兔子的最近公共祖先编号为 2。
- 编号为 6 和 13 的兔子的最近公共祖先编号为 1。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个正整数 a, b ($1 \leq a, b \leq 2^{61}$)，表示询问的两对兔子的编号。

输出格式

对于每组数据，输出一行一个正整数，表示询问的两对兔子的最近公共祖先编号。

```
#include<iostream>
#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 90;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
ll getll() {
    char c;
    bool flag=0;
    ll num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
ll f[N + 5];
ll getfa(ll x) {
    for(int i = N; i; i --)
        if(f[i] < x)
            return f[i - 1] - (x - f[i]) + 1;
    return 1;
}
```

```

}
int main() {
    f[1] = f[2] = 1;
    for(int i = 3; i <= N; i++)
        f[i] = f[i - 1] + f[i - 2];

    int q = getint();
    while(q --) {
        ll x = getll(), y = getll();
        while(x != y)
            x > y ? x = getfa(x) : y = getfa(y);
        cout << x << endl;
    }

    return 0;
}

```

F 小水獭和离散对数

小水獭正在学习离散对数，它正在关注如下问题：

- 给定正整数 $n = 4\,179\,340\,454\,199\,820\,289$ 、 $m = 3$ 和 k ，对于 $i = 1, 2, \dots, 10^5$ ，它想找到最小正整数的 $x \geq k$ 使得 i 在无前导 0 的十进制表示下是 $m^x \bmod n$ 的后缀，或判断这样的 x 不存在。

举例来说，当 $k = 1, i = 11$ 时， $x = 151$ 满足条件，因为 $3^{151} \bmod n = 1\,869\,392\,813\,447\,903\,111$ ，11 是它的后缀，而 $1, 2, \dots, 150$ 均不满足条件。

小水獭数学不太好，希望你能帮它编写一个求解器，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行一个正整数 k ($1 \leq k \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 10^5 个整数，对于第 i ($1 \leq i \leq 10^5$) 个整数：

- 如果对于 i 存在满足条件的 x ，输出 x 。
- 否则，输出 -1 。

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

typedef long long ll;

const ll p = 4179340454199820289ll;

ll fpow(ll x, int y) {
    ll ans = 1;
    while(y) {
        if(y & 1) ans = (__int128)ans * x % p;
        x = (__int128)x * x % p, y >>= 1;
    }
    return ans;
}

int t, k;
int a[maxn];

```

```

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &k);
        for(int i = 1; i <= 1e5; i++) a[i] = 0;
        int cnt = 0;
        ll now = fpow(3, k);
        while(cnt < 1e5) {
            ll x = now;
            int y = 0;
            for(int i = 1; i <= 1e5; i *= 10) {
                if(!x) break;
                y += i * (x % 10), x /= 10;
                if(y > 1e5) break;
                if(y && !a[y]) a[y] = k, cnt++;
            }
            now = (__int128)now * 3 % p, k++;
        }
        for(int i = 1; i <= 1e5; i++) printf("%d ", a[i]);
        puts("");
    }
}

```

G 莫卡和计算机网络

wzf 带领莫卡参观了 CVBBBDN 的专属机房，机房的座位可以被视作一个 $n \times n$ 的网格图，一些座位放置了计算机，剩余的座位是空的。

机房的计算机网络结构比较特殊，两个计算机（假设为计算机 i 和计算机 j ，且 $i \neq j$ ）可以通信当且仅当满足下列两个条件中**任意一个**：

- 两台计算机位置相邻，即假设两个计算机的分别位于第 r_i 行第 c_i 列和第 r_j 行第 c_j 列，则有 $|r_i - r_j| + |c_i - c_j| \leq 1$ 。
- 存在 k ($k \neq i, k \neq j$)，满足计算机 i 能够和计算机 k 通信且计算机 j 能够和计算机 k 通信。

参观结束后，莫卡想知道，如果在每一个空位置放上一台计算机，它能够和多少台计算机通信？

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 200$)，表示机房座位的行数和列数。

接下来 n 行，第 i ($1 \leq i \leq n$) 行一个长度为 n 的 01 串，如果第 i 个串的第 j ($1 \leq j \leq n$) 位为 1，则代表机房第 i 行第 j 列的座位已经放置了一台计算机，否则代表该座位为空。

输出格式

对于每组数据，第一行输出一个非负整数 m ，代表空位置的个数。

第二行输出 m 个整数，代表每个空位置放上计算机能够通信的计算机数量**从小到大排序**后的结果。

注意，如果空位置的数量为 0，你需要输出一个额外的空行。

```

#include<bits/stdc++.h>
#define maxn 505
using namespace std;

inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
    }
}

```

```

        ch = getchar();
    }
    return x * f;
}

int t, n, mp[maxn][maxn], col[maxn][maxn], cnt[maxn * maxn], now;
char s[maxn];

int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};

void dfs(int x, int y) {
    if(col[x][y]) return ;
    col[x][y] = now;
    cnt[now]++;
    for(int i = 0; i < 4; i++) {
        int xx = x + dx[i], yy = y + dy[i];
        if(xx < 1 || xx > n || yy < 1 || yy > n) continue;
        if(mp[xx][yy] == 0) continue;
        if(col[xx][yy]) continue;
        dfs(xx, yy);
    }
}

int main() {
    t = read();
    while(t--) {
        n = read();
        for(int i = 1; i <= n; i++) {
            scanf("%s", s + 1);
            for(int j = 1; j <= n; j++) mp[i][j] = s[j] - '0', cnt[(i - 1) * n +
j] = col[i][j] = 0;
        }
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(mp[i][j] == 1
&& !col[i][j])
            now++, dfs(i, j);
        vector<int> ans;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(!mp[i][j]) {
            set<int> s;
            int tot = 0;
            for(int k = 0; k < 4; k++) {
                int xx = i + dx[k], yy = j + dy[k];
                if(xx < 1 || xx > n || yy < 1 || yy > n) continue;
                if(mp[xx][yy] == 0) continue;
                s.insert(col[xx][yy]);
            }
            for(auto x: s) tot += cnt[x];
            ans.push_back(tot);
        }
        sort(ans.begin(), ans.end());
        printf("%d\n", ans.size());
        for(auto x: ans) printf("%d ", x);
        puts("");
    }
}

```

H XIAO7 和逗猫棒

本题提交显示 Other Error 代表 Wrong Answer 。

题目描述

猫娘「雪风」想要穿裙子！

XIAO7 告诉她：「如果你能答对以下若干个问题，我就给你买裙子。」

请你帮帮可爱的「雪风」！

具体来说，XIAO7 有 n 根逗猫棒，逗猫棒有两个属性：颜色和长度。XIAO7 想知道，能否用三根颜色互不相同的逗猫棒组成一个**非退化三角形**。

设三角形的三边长度为 $a \leq b \leq c$ ，该三角形是非退化三角形当且仅当 $a + b > c$ 。

输入格式

第一行一个正整数 n ($1 \leq n \leq 2 \times 10^5$)，表示逗猫棒的数量。

接下来 n 行，第 i ($1 \leq i \leq n$) 行两个正整数 c_i, l_i ($1 \leq c_i, l_i \leq 10^9$)，表示编号为 i 的逗猫棒的颜色和长度。

输出格式

若存在三根颜色互不相同的逗猫棒可以组成一个非退化三角形，输出一行三个互不相同的正整数 i, j, k ($1 \leq i, j, k \leq n$)，表示三根逗猫棒的编号。如果有多种可能的答案，你可以输出任意一种。

否则，输出 I would like to see xf wear a skirt every day !。

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 200005;
const int M = 55;
const int inf = 0x3f3f3f3f;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&& c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
const int cid[3][2] = {{0, 1}, {0, 2}, {1, 2}};
int color[N], len[N], id[N], mx[3];
bool cmp(int i, int j) {
    return len[i] < len[j];
}
void add(int x) {
    for(int i = 0; i < 3; i++)
        if(color[mx[i]] == color[x]) {
            len[mx[i]] < len[x] ? mx[i] = x : 1;
            return ;
        }
    sort(mx, mx + 3, cmp);
    len[mx[0]] < len[x] ? mx[0] = x : 1;
    return ;
}
bool check(int i, int j, int k) {
```

```

        return color[i] != color[k] && color[j] != color[k] && len[i] + len[j] >
len[k];
    }
    int main() {
        int n = getint();
        if(n < 3) {
            puts("I would like to see xf wear a skirt every day !");
            return 0;
        }
        for(int i = 1; i <= n; i++)
            scanf("%d%d", &color[i], &len[i]), id[i] = i;
        sort(id + 1, id + n + 1, cmp);
        add(id[1]), add(id[2]);
        for(int i = 3; i <= n; i++) {
            for(int j = 0; j < 3; j++)
                if(check(mx[cid[j][0]], mx[cid[j][1]], id[i])) {
                    printf("%d %d %d\n", mx[cid[j][0]], mx[cid[j][1]], id[i]);
                    return 0;
                }
            add(id[i]);
        }
        puts("I would like to see xf wear a skirt every day !");
    }
}

```

I Bellalabella 和生日礼物

本题提交显示 Other Error 代表 Wrong Answer。

题目描述

Bellalabella 最近又要过生日了，她收到了来自 n 位同学的礼物。这些礼物大多都是相同类型的礼物，第 i 个同学送的礼物为第 a_i 种礼物。Bellalabella 想知道第 $l, l+1, \dots, r$ 位同学送的礼物中有哪种礼物出现不少于 $\left\lceil \frac{r-l+1}{k} \right\rceil$ 次，其中 $\lceil x \rceil$ 表示不小于 x 的最小整数。

Bellalabella 会进行 q 次上述询问。

如果有多种礼物满足条件，Bellalabella 只想知道其中一个；如果没有满足条件的礼物，你也需要告知 Bellalabella 这一点。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示收到的礼物个数。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$)，含义同题目描述。

第三行一个正整数 q ($1 \leq q \leq 10^4$)，表示询问次数。

接下来 q 行，第 i ($1 \leq i \leq q$) 行包含三个整数 l_i, r_i, k_i ($1 \leq l_i \leq r_i \leq n, 1 \leq k_i \leq 4$)，表示第 i 个询问的参数。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 若存在满足条件的礼物种类，输出该礼物种类。如果有多种可能的答案，你可以输出任意一种。
- 否则，输出 -1 。

```

#include<bits/stdc++.h>
#define maxn 1000005
using namespace std;

inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
}

```

```

    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

vector<int> app[maxn];
int a[maxn], n, q;

int main() {
    srand(time(0));
    n = read();
    for(int i = 1; i <= n; i++) a[i] = read(), app[a[i]].push_back(i);
    q = read();
    for(int i = 1; i <= q; i++) {
        int l = read(), r = read(), k = read(), ok = (r - l + 1 + k - 1) / k;
        int ans = -1;
        for(int t = 1; t <= 60; t++) {
            int x = a[rand() % (r - l + 1) + l];
            if(lower_bound(app[x].begin(), app[x].end(), r + 1) -
upper_bound(app[x].begin(), app[x].end(), l - 1) >= ok)
                ans = x;
        }
        printf("%d\n", ans);
    }
}

```

J 小水獭和神秘动物

小水獭正在沉迷「神秘动物在哪里」，无心学习离散数学，请你帮它求出满足如下条件的方阵 $\{a_{i,j}\}_{n \times n}$ 数量：

- $a_{i,j} \in \{0, 1\}$ 对 $1 \leq i, j \leq n$ 成立。
- $a_{i,i} = 1$ 对 $1 \leq i \leq n$ 成立。
- $a_{i,j} = 1 \wedge a_{j,k} = 1 \Rightarrow a_{i,k} = 1$ 对 $1 \leq i, j, k \leq n$ 成立。
- 此外，有 m 个额外限制，第 i ($1 \leq i \leq m$) 个限制形如 $a_{x_i, y_i} = z_i$ 。

可以证明，在本题数据范围限制下，答案不超过 10^7 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 40$)，表示数据组数。

对于每组数据，一行两个非负整数 n, m ($1 \leq n \leq 7, 0 \leq m \leq n(n-1)$)，表示方阵的大小和额外限制数量。

接下来 m 行，第 i ($1 \leq i \leq m$) 行三个非负整数 x_i, y_i, z_i ($1 \leq x_i, y_i \leq n, x_i \neq y_i, 0 \leq z_i \leq 1$)，表示第 i 个限制的参数。

保证 $x_i = x_j \wedge y_i = y_j \Leftrightarrow i = j$ 对 $1 \leq i, j \leq n$ 成立，且 t 组数据的答案之和不超过 3×10^7 。

输出格式

对于每组数据，输出一行一个非负整数，表示满足条件的方阵数量。

```

#include <bits/stdc++.h>
#define maxn 10

using namespace std;

int t, n, m;

```

```

int ans;
vector<pair<int, int> > v;
int tag[maxn][maxn];
int in1[maxn], out1[maxn], in0[maxn], out0[maxn];

void dfs(int i) {
    if(i == v.size()) {
        ans++;
        return;
    }
    int x = v[i].first, y = v[i].second;
    if(tag[x][y] != 1 && !(out1[x] & in1[y])) {
        out0[x] ^= 1 << y;
        in0[y] ^= 1 << x;
        dfs(i + 1);
        out0[x] ^= 1 << y;
        in0[y] ^= 1 << x;
    }
    if(tag[x][y] != 0 && !(out0[x] & out1[y]) && !(in1[x] & in0[y])) {
        out1[x] ^= 1 << y;
        in1[y] ^= 1 << x;
        dfs(i + 1);
        out1[x] ^= 1 << y;
        in1[y] ^= 1 << x;
    }
}

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d", &n, &m);
        for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) tag[i][j] = -1;
        while(m--) {
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            tag[x - 1][y - 1] = z;
        }
        v.clear();
        for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) if(i ^ j)
v.push_back({i, j});
        ans = 0, dfs(0);
        printf("%d\n", ans);
    }
}

```

C3

A 进厂切钢条

互联网寒冬袭来，Zhoues 打算提前进厂，体验一下流水线的感觉是什么样的。

Zhoues 给一家钢条厂发送了一份自己的简历，希望可以寒假在这里实习。厂里负责招聘的工作人员正好最近也遇到了一个难题，需要一个会动态规划算法的同学帮帮忙。他恰好看到 Zhoues 的简历说自己熟练掌握《算法导论》上的动态规划问题，于是打算用这个难题考一考 Zhoues，问题如下：

- 给定一段长度为 n 英寸的钢条和一个价格表，该价格表表示长度为 i ($i = 1, 2, \dots, n$) 英寸的钢条的价格为 p_i 。求钢条切割方案，使得总销售价格最大，注意钢条的长度必须为整英寸。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^4$)，表示钢条的总长度。

第二行 n 个正整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 10^7$)，表示钢条的价格。

输出格式

一行一个正整数，表示最大总销售价格。

```
#include<bits/stdc++.h>
#define maxn 100005
typedef long long ll;
using namespace std;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}
ll dp[maxn];
int main(){
    int n = read();
    for(int i = 1; i <= n; i++) dp[i] = read();
    for(int i = 2; i <= n; i++){
        for(int j = 1; j < i; j++) dp[i] = max(dp[i], dp[j] + dp[i - j]);
    }
    printf("%lld\n", dp[n]);
    return 0;
}
```

B 进厂添括号

Zhoues 成功进厂了，现在被通知如果想晋升进行高级流水线技师，需要进行一场额外的算法考核。Zhoues 希望你们可以帮助他通过该考核，因为他真的很想当高级技师！题目内容如下：

- 给定 n 个矩阵的链乘 $A_1 A_2 \cdots A_n$ ，你需要在不改变矩阵链顺序的前提下给矩阵添加 $n - 1$ 对合法的括号，使得矩阵相乘运算次数最少。

你只需要输出最少运算次数。

输入格式

第一行一个整数 n ($1 \leq n \leq 300$)，表示矩阵的个数。

第二行 $n + 1$ 个正整数 $a_1, a_2, \dots, a_{n+1}$ ($1 \leq a_i \leq 10^3$)，表示矩阵 A_i 的行数和列数为 a_i 和 a_{i+1} 。

输出格式

一行一个正整数，表示最少运算次数。

```
#include<bits/stdc++.h>
#define maxn 310
#define INF LLONG_MAX
typedef long long LL;
using namespace std;
LL m[maxn][maxn],s[maxn][maxn];
LL a[maxn],n;
```

```

void Matrixchain() {
    for(LL l=2; l<=n; l++)
        for(LL i=1; i<=n-l+1; i++) {
            LL j = i+l-1;
            m[i][j] = INF;
            for(LL k = i; k<j; k++) {
                LL tmp = m[i][k] + m[k+1][j] + a[i-1] * a[k] * a[j];
                if(tmp<m[i][j])    m[i][j] = tmp,s[i][j] = k;
            }
        }
}

int main() {
    scanf("%lld",&n);
    for(LL i=0; i<=n; i++)
        scanf("%lld",a+i);
    for(LL i=0; i<=n; i++)
        m[i][i] = 0;
    Matrixchain();
    printf("%lld\n", m[1][n]);
}

```

C 进厂体验流水线

Zhoues 现在是高级技师哇！招聘人员让 Zhoues 明天立马过来报道并体验厂里车间的流水线装配环节。

流水线车间的负责人也想考一下 Zhoues，具体来说，此车间有 2 条流水线，每条流水线有 m 个装配站，编号都为 $1, 2, \dots, m$ ，每条流水线同样编号的装配站都执行相同的功能。拼装一个成品要按顺序经过 m 个装配站才能加工完成，经过第 i 条流水线的第 j 个装配站要花费 $p_{i,j}$ 的时间，从第 i 条流水线移动到第 j 条流水线要花费 $t_{i,j}$ 的时间，请问制造一个成品最少时间是多少，而且具体在流水线装配的过程是什么呢？

输入格式

第一行一个正整数 m ($2 \leq m \leq 100$)，表示每条流水线有 m 个装配站。

接下来两行，每行 m 个正整数 $p_{i,1}, p_{i,2}, \dots, p_{i,m}$ ($0 \leq p_{i,j} \leq 100$)，表示经过第 i 条流水线的第 j 个装配站要花费的时间。

接下来两行，每行两个正整数 $t_{i,1}, t_{i,2}$ ($0 \leq t_{i,j} \leq 10$)，表示从第 i 条流水线移动到第 j 条流水线所花费的时间。

输出格式

第一行一个非负整数，表示制造一个成品的最少时间。

接下来 m 行，表示一个成品在流水线装配的具体过程，要求按装配站编号从小到大的顺序输出，输出样式形如 `Station%d, Line%d`，数据保证方案唯一。

```

#include <iostream>
#include <cstring>
#include <climits>
#include <vector>
using namespace std;
#define maxn 100005
typedef long long ll;
int m;
ll dp[3][maxn];
int line[3][maxn];
int work[3][maxn];
int to[3][3];
int l;
vector<pair<int, int>> v;

inline void init()
{
    for (int i = 1; i <= 2; ++i)
    {

```

```

        for (int j = 1; j <= m; ++j)
        {
            dp[i][j] = LLONG_MAX;
        }
    }
}

inline ll buildDP()
{
    for (int i = 1; i <= 2; ++i)
    {
        dp[i][1] = work[i][1];
    }

    // 每一列
    for (int j = 2; j <= m; ++j)
    {
        // 每一条流水线行数
        for (int i = 1; i <= 2; ++i)
        {
            // 前一条流水线行数
            for (int k = 1; k <= 2; ++k)
            {
                if (dp[i][j] > work[i][j] + dp[k][j - 1] + to[k][i])
                {
                    dp[i][j] = work[i][j] + dp[k][j - 1] + to[k][i];
                    line[i][j] = k;
                }
            }
        }
    }

    ll result = LLONG_MAX;
    for (int i = 1; i <= 2; ++i)
    {
        result = min(dp[i][m], result);
    }

    if (result == dp[1][m])
    {
        l = 1;
    }
    else
    {
        l = 2;
    }
    return result;
}

void searchDP()
{
    v.push_back(make_pair(0, 0));
    for (int j = m; j >= 1; j--)
    {
        v.push_back(make_pair(j, 1));
        l = line[l][j];
    }
    for (int j = m; j >= 1; j--)
    {

```

```

        printf("Station%d, Line%d\n", v[j].first, v[j].second);
    }
}

int main()
{
    scanf("%d", &m);
    init();
    for (int i = 1; i <= 2; ++i)
    {
        for (int j = 1; j <= m; ++j)
        {
            scanf("%d", &work[i][j]);
        }
    }
    for (int i = 1; i <= 2; ++i)
    {
        for (int j = 1; j <= 2; ++j)
        {
            scanf("%d", &to[i][j]);
        }
    }
    printf("%lld\n", buildDP());
    searchDP();
}

```

D 小水獭和最大值 I

小水獭正在学习动态规划。

对于给定的整数序列 $a_1, a_2, \dots, a_n$ ，小水獭想知道不相邻子序列元素和的最大值是多少。形式化地说，对于所有非空整数序列 $x_1, x_2, \dots, x_m$ ，满足 $1 \leq x_i + 1 < x_{i+1} \leq n$ 对 $1 \leq i < m$ 成立，求解 $\sum_{i=1}^m a_{x_i}$ 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示序列的长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，表示序列的元素。

输出格式

对于每组数据，输出一行一个整数，表示不相邻子序列元素和的最大值。

```

#include <bits/stdc++.h>
#define maxn 1000086

using namespace std;

typedef long long ll;

int t, n;
int a[maxn];
ll f[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);

```

```

    for(int i = 1;i <= n;i++) scanf("%d", &a[i]);
    f[1] = a[1];
    if(n >= 2) f[2] = max(a[1], a[2]);
    for(int i = 3;i <= n;i++){
        f[i] = max({f[i - 1], f[i - 2] + a[i], 1ll * a[i]});
    }
    printf("%lld\n", f[n]);
}
}

```

E 小水獭和最大值 II

小水獭正在学习动态规划。

给定一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，小水獭初始位于 $(1,1)$ ，目标位置为 (n,m) ，它每次可以向下移动或向右移动，即从 (x,y) 移动到 $(x+1,y)$ 或 $(x,y+1)$ ，但不能移动到矩阵之外，它想知道经过位置元素之和的最大值是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($2 \leq n, m \leq 10^3$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($-10^9 \leq a_{i,j} \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个整数，表示经过位置元素之和的最大值。

```

#include <bits/stdc++.h>
#define maxn 1086

using namespace std;

typedef long long ll;

int t, n, m;
int a[maxn][maxn];
ll f[maxn][maxn];

int main(){
    memset(f, -0x3f, sizeof(f)), f[0][1] = 0;
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1;i <= n;i++) for(int j = 1;j <= m;j++) scanf("%d", &a[i][j]);
        for(int i = 1;i <= n;i++) for(int j = 1;j <= m;j++){
            f[i][j] = max(f[i - 1][j], f[i][j - 1]) + a[i][j];
        }
        printf("%lld\n", f[n][m]);
    }
}

```

F XIAO7 和数集（简单版）

由于所有的喵币都分给「雪风」和「星瑶」了，猫娘「凉宫秋小七」对此很不高兴又哭又闹，XIAO7 只好告诉她，若你能答对以下问题，我就把上一题扔掉的喵币捡回来偷偷给你。

具体来说，对于不含前导零的正整数 x ，定义 $S(x)$ 表示 x 的十进制表示下的每一位数字组成的集合，例如：

- $S(11523) = \{1, 2, 3, 5\}$
- $S(67654) = \{4, 5, 6, 7\}$

对于给定的 n ，XIAO7 想知道有多少对正整数 (x, y) 满足 $1 \leq x < y \leq n$ 且 $S(x) = S(y)$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int num[N];
int S(int x){
    int state = 0;
    for(; x; x /= 10)
        state |= 1 << (x % 10);
    return state;
}
int main(){

    int q = getint();

    while(q --) {
        int n = getint();
        assert(1 <= n && n <= 1000000);

        ll ans = 0;
        memset(num, 0, sizeof num);
        for(int i = 1; i <= n; i++){
            int s = S(i);
            ans += num[s], num[s] ++;
        }
        cout << ans << endl;
    }
```

```
}  
}
```

G 7sozx 特有的字符串

7sozx 特别喜欢字符串 $T = 1145141919810$ 。

对于给定的字符串 S ，7sozx 想让你帮它求出一个 S 的划分 $s_1, s_2, \dots, s_m$ ，满足：

- 将 $s_1, s_2, \dots, s_m$ 顺序拼接得到的字符串和 S 相同。
- $\sum_{i=1}^m f(s_i, T)$ 最大。

其中 $f(a, b)$ 表示字符串 a 和 b 相同的位置数量。形式化地说， $f(a, b)$ 是满足 $a_i = b_i$ 的 i 的数量，其中 $i = 1, 2, \dots, \min(|a|, |b|)$ 。例如：

- 当 $a = \underline{1234567}, b = \underline{1534967}$ 时， $f(a, b) = 5$ ，相同的位置用下划线标出。
- 当 $a = \underline{1234567}, b = \underline{153496789}$ 时， $f(a, b) = 5$ ，相同的位置用下划线标出。

你只需要输出 $\sum_{i=1}^m f(s_i, T)$ 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个十进制数字串 s ($1 \leq |s| \leq 10^5$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案。

```
#include <bits/stdc++.h>  
#define maxn 100086  
  
using namespace std;  
  
int t, n;  
char s[maxn];  
char T[14] = "1145141919810";  
int f[maxn][14];  
  
int main(){  
    scanf("%d", &t);  
    while(t--){  
        scanf("%s", s + 1);  
        n = strlen(s + 1);  
        memset(f, -0x3f, sizeof(f));  
        f[0][0] = 0;  
        for(int i = 1; i <= n; i++){  
            for(int j = 0; j < 14; j++){  
                if(j < 13) f[i][j + 1] = max(f[i][j + 1], f[i - 1][j] + (s[i] ==  
T[j]));  
                f[i][1] = max(f[i][1], f[i - 1][j] + (s[i] == T[0]));  
            }  
        }  
        int ans = 0;  
        for(int i = 0; i < 14; i++) ans = max(ans, f[n][i]);  
        printf("%d\n", ans);  
    }  
}
```

H XIAO7 和喵币

XIAO7 有 n 个喵币，面值分别为 $a_1, a_2, \dots, a_n$ 。XIAO7 要将它们分给「雪风」和「星瑶」两只猫娘，但如果两只猫娘分到的喵币总面值不同，她们就会打架，因此多余的不能均分的喵币只能被扔掉，XIAO7 想知道需要扔掉喵币的面值之和最少是多少。

注意，每张喵币的面值是固定的，不能进行拆分。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^3$)，表示喵币的数量。

第二行 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^4$)，表示喵币的面值。

保证 $\sum_{i=1}^n a_i \leq 10^4$ 。

输出格式

第一行一个非负整数，表示最少需要扔掉的喵币面值总和。

第二行若干个正整数，表示需要扔掉的喵币的序号。如果有多种可能的答案，你可以输出任意一种。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1005;
const int M = 10005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int a[N], sum[N], f[N][M];
vector<int> last;
void dfs(int n, int d){
    if(n < 1) return ;
    else if(f[n][d] == f[n - 1][d] + a[n]){
        last.push_back(n);
        dfs(n - 1, d);
    }
    else if(f[n][d] == f[n - 1][d + a[n]])
        dfs(n - 1, d + a[n]);
    else dfs(n - 1, abs(d - a[n]));
}
int main(){
    int n = getint();
    assert(n <= 1000);
    for(int i = 1; i <= n; i ++){
        sum[i] = sum[i - 1] + (a[i] = getint());
        assert(sum[n] <= 10000);
    }
}
```

```

memset(f, 0x3f, sizeof(f));
f[0][0] = 0;
for(int i = 1; i <= n; i ++){
    for(int j = 0; j <= sum[i - 1]; j ++){
        f[i][j] = min(f[i][j], f[i - 1][j] + a[i]);
        f[i][j + a[i]] = min(f[i][j + a[i]], f[i - 1][j]);
        f[i][abs(j - a[i])] = min(f[i][abs(j - a[i])], f[i - 1][j]);
    }

    cout << f[n][0] << endl;

    dfs(n, 0);
    for(auto v : last)
        printf("%d ", v);
}

```

I 任务达人莫卡 II

莫卡穿越到了一个新的世界并且遇到了一位名为任务达人的 NPC。

这个新的世界一共有 n 个城市，编号从 1 到 n ，莫卡起初位于编号为 k 的城市。第 i 天任务达人会给莫卡派送一个任务，该任务需要前往编号为 c_i 的城市完成，莫卡可以选择接受或者放弃这个任务，如果莫卡接受了任务，若莫卡当前即位于编号为 c_i 的城市，则可以获得 a_{c_i} 的收益，否则莫卡会前往编号为 c_i 的城市（在城市间移动是没有花费的），并获得 b_{c_i} 的收益，完成任务后莫卡会停留在编号为 c_i 的城市。如果莫卡放弃了任务，她将不会获得任何收益并会停留在当前所在城市。

莫卡想请你帮她计算出 m 天后自己可以获得的最大收益。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，代表数据组数。

对于每组数据，第一行两个正整数 n, k ($3 \leq n \leq 10^5$)，表示城市个数和起始城市编号。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)，含义同题目描述。

第三行 n 个正整数 $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$)，含义同题目描述。

第四行一个正整数 m ($1 \leq m \leq 10^5$)，表示执行任务的天数。

接下来 m 行，每行一个正整数 c_i ($1 \leq c_i \leq n$)，表示第 i 天任务的城市编号。

输出格式

对于每组数据，输出一行一个正整数，表示 m 天后能够获得的最大收益。

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long

multiset<ll> s;
const int N = 1e5+10;
ll a[N];
ll b[N];
ll dp[N];
int main(){
    int t;
    scanf("%d",&t);
    while(t--){
        s.clear();
        int n, m, k;
        scanf("%d %d", &n, &k);
        for (int i = 1; i <= n; i++) {
            scanf("%lld", a + i);

```

```

        dp[i] = -2e9;
    }
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &b[i]);
    }
    dp[k] = 0;
    scanf("%d", &m);
    for (int i = 1; i <= n; i++) {
        s.insert(dp[i]);
    }
    for (int i = 1; i <= m; i++) {
        int c;
        scanf("%d", &c);
        auto itt = s.find(dp[c]);
        s.erase(itt);
        auto it = s.end();
        it--;
        dp[c] = max(dp[c] + a[c], (*it) + b[c]);
        s.insert(dp[c]);
    }
    auto it = s.end();
    it--;
    printf("%lld\n", *it);
}
}

```

J 小水獭和电影院（简单版）

小水獭正在学习动态规划，但想去看个电影放松一下。

电影院可以被抽象为一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，如果电影院第 i 行第 j 列的座位有人，则 $a_{i,j} = 1$ ，否则 $a_{i,j} = 0$ 。

众所周知，为防控疫情，相邻座位不能同时有人。形式化地说， $(a_{i,j} = 1 \wedge a_{i',j'} = 1) \Rightarrow (|i - i'| + |j - j'| \neq 1)$ 。

小水獭想知道，有多少种不同的矩阵所对应的电影院是满足疫情防控要求的，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行两个正整数 n, m ($1 \leq nm \leq 400$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

```

#include <bits/stdc++.h>
#define maxn 20

using namespace std;

int t, n, m;
unsigned f[1 << maxn], g[1 << maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        if(n < m) swap(n, m);
        for(int i = 0; i < (1 << m); i++) f[i] = g[i] = 0;
    }
}

```

```

    f[0] = 1;
    for(int i = 0; i < n; i++) for(int j = 0; j < m; j++){
        for(int s = 0; s < (1 << m); s++){
            int ss = (s | (1 << j)) ^ (1 << j);
            g[ss] += f[s];
            if((j == 0 || !(s & (1 << (j - 1)))) && !(s & (1 << j))) g[ss |
(1 << j)] += f[s];
        }
        for(int s = 0; s < (1 << m); s++) f[s] = g[s], g[s] = 0;
    }
    unsigned ans = 0;
    for(int i = 0; i < (1 << m); i++) ans += f[i];
    printf("%u\n", ans);
}
}

```

A 图论选择练习题

A 图论选择题练习

时间限制：1000ms 内存限制：65536kb

通过率：200/201 (99.50%) 正确率：200/202 (99.01%)

选择题作答要求

请根据题目要求完成下列选择题，并将答案以每行一个只包含大写英文字母的、字母按照升序的字符串的格式输出，不要输出多余空格。假设有 n 道题目，那么 Special Judge 只会读取你输出的前 n 行内容，如果输出不足 n 行也会被判定为格式错误。

例如，有三道选择题，第一题答案为 BCE，第二题答案为 AC，第三题答案为 D，你应该输出以下答案。

```
BCE
AC
D
```

你可以使用如下的代码进行输出：

```
#include <stdio.h>

int main(){
    puts("BCE");
    puts("AC");
    puts("D");
}
```

只要满足上述格式要求，即返回 Accepted 结果，不返回真实评测结果，待比赛结束后正式评测使用每个人的最后一次 Accepted 的提交记录进行正式评测。

如果格式错误，会返回 Presentation Error 结果。

题目描述

第一题

下面关于 Dijkstra 算法不恰当的描述有_____。

- A. 采用 DP 方法。
- B. 采用分治方法。
- C. 采用贪心策略。
- D. 利用了边的松弛操作。
- E. 利用了优先队列。
- F. 顶点遍历是采用 DFS。

第二题

基于拓扑排序的最短路径算法适用于哪种性质的图_____。

- A. 有向、无回路、有负权边。
- B. 有向、无回路、无负权边。
- C. 有向、有回路、有负权边。
- D. 有向、有回路、无负权边。

第三题

Bellman-Ford、基于拓扑排序、采用 std::priority_queue 的 Dijkstra 三种算法的时间复杂度分别为（采用邻接表存储图，设点数和边数分别为 n 和 m ）_____。

- A. $O(nm)$ 、 $O(n^2)$ 、 $O(n+m)$ 。
- B. $O(n^2)$ 、 $O(nm)$ 、 $O(n+m)$ 。
- C. $O(nm)$ 、 $O(n+m)$ 、 $O(n+m)$ 。
- D. $O(nm)$ 、 $O(n+m)$ 、 $O(n\log n+m)$ 。
- E. $O(n^2)$ 、 $O(n+m)$ 、 $O(n\log n+m)$ 。
- F. $O(n^2)$ 、 $O(n+m)$ 、 $O(n^2)$ 。

B

B 运动会观赛安排

时间限制：1000ms 内存限制：65536kb

通过率：184/200 (92.00%) 正确率：184/726 (25.34%)

题目描述

钢条厂终于完成北航 70 周年校庆的大订单，因此也让员工休息几天。Zhoues 打算借这个空闲的时间回北航看看正在进行的运动会。

Zhoues 对所有的运动会比赛都很感兴趣，想为软件学院的运动健儿们加油鼓劲。Zhoues 一旦为某一场比赛加油鼓劲，他就必须全程看完这次比赛，但是很可惜的是有相当多的比赛项目的时间安排是重叠的，Zhoues 希望可以好好规划一下观赛的时间安排，让他在有限的时间内为尽可能多的比赛加油鼓劲，那么最多他可以为多少场比赛加油鼓劲呢？

形式化地说，共有 n 场比赛，第 i 场比赛的时间可以用左闭右开区间 $[x_i, y_i)$ 来表示，你需要选择尽可能多的比赛使得任意两个不同比赛对应的区间交集均为空。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示运动会的比赛数量。

接下来 n 行，每行两个整数 x_i, y_i ($1 \leq x_i < y_i \leq 10^6$)，表示第 i 场比赛的时间。

输出格式

一行一个正整数，表示 Zhoues 最多可以为运动会比赛加油鼓劲的次数。

样例输入

```
3
2 4
3 5
4 6
```

样例输出

```
2
```

样例解释

一种最优方案为选择第 1 场比赛和第 3 场比赛。

Hint

大胆贪心，小心求证！

经典贪心问题，本题的所有伪代码具体可以参考算法导论 P238 或者课件 PPT。

```
#include<bits/stdc++.h>
using namespace std;

pair<int,int>a[1000005];

int cmp(pair<int,int>a,pair<int,int>b){
    if(a.second!=b.second)return a.second<b.second;
    else return a.first<b.first;
}

int main(){
    using namespace std;
    int n;
    cin>>n;
    for(int i = 0;i < n;i++){
        cin>>a[i].first>>a[i].second;
    }
    sort(a,a+n,cmp);
    int ans = 1;
    int temp = a[0].second;
    for(int i=1;i<n;i++){
        if(a[i].first >= temp){
            temp = a[i].second;
            ans++;
        }
    }
```

```
}  
cout<<ans;  
}
```

C

C 偷钢条

时间限制：1000ms 内存限制：65536kb

通过率：144/194 (74.23%) 正确率：144/843 (17.08%)

题目描述

Zhoues 虽然是一名高级技师，但是其实他还暗中申请了安保工作，是一名光荣的安保大队队员。一天晚上，发现有一个人影在流水线附近，他悄悄的走进一看，发现是一个正在偷钢条的小偷！

小偷嘴里默念着：「我现在有一个最大承重为 k 吨的背包，现在流水线上有 n 根钢条，第 i 根钢条重量为 w_i 吨，价值为 v_i ，我最终的收益是偷到钢条的价值之和，我偷到钢条的总重量不能超过我背包的最大承重，那我最终可以获得的最大收益是多少呢？」

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的，于是他希望你帮忙计算一下小偷最终获得的最大收益是多少。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^3, 1 \leq k \leq 10^5$)，表示钢条的数量和小偷背包的最大承重。

接下来 n 行，每行两个正整数 v_i, w_i ($1 \leq v_i \leq 10^7, 1 \leq w_i \leq 10^6$)，表示第 i 根钢条的价值和重量。

输出格式

一行一个非负整数，表示小偷可以获得的最大收益。

输入样例

```
3 6  
1 1  
1 3  
2 4
```

输出样例

```
3
```

样例解释

一种最优方案为偷走第 1 根和第 3 根钢条。

Hint

注意是 n 根，不是 n 种。

经典动态规划问题，本题具体可以参考算法导论 P244 或者课件 PPT。

```
#include<iostream>  
#include<cstring>  
using namespace std;  
long long dp[200001];  
long long v[300001],w[300001];  
long long i,j,m,n;  
int main(){  
    scanf("%lld%lld",&n,&m);  
  
    dp[0]=0;  
    for(i=1;i<=n;i++){  
        scanf("%lld %lld",&v[i],&w[i]);  
    }  
    for(i=1;i<=n;i++){
```

```
        for(j=m;j>=w[i];j--){
            if(dp[j]<dp[j-w[i]]+v[i] )
                dp[j]=dp[j-w[i]]+v[i];
        }
    }
    printf("%lld\n",dp[m]);

    return 0;
}
```

D

D 偷沙砾

时间限制：1000ms 内存限制：65536kb

通过率：169/181 (93.37%) 正确率：169/331 (51.06%)

题目描述

Zhoues 虽然是一名高级技师，但是其实他还暗中申请了安保工作，是一名光荣的安保大队队员。一天晚上，发现有一个人在钢条厂原料区附近，他悄悄的走进一看，发现是一个正在偷沙砾的小偷！

小偷嘴里默念着：「我现在有一个最大承重为 k 吨的背包，现在原料区上有 n 种沙砾，第 i 种沙砾的总重量为 w_i 吨，总价值为 v_i ，如果偷走 j ($0 < j \leq w_i$) 吨第 i 种沙砾，那么我能获得的收益是 $\frac{jv_i}{w_i}$ ，我最终的收益是偷到沙砾的价值之和，我偷到沙砾的总重量不能超过我背包的最大承重，那我最终可以获得的最大收益是多少呢？」

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的，于是他希望你帮忙计算一下小偷最终获得的最大收益是多少。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^3, 1 \leq k \leq 10^5$)，表示沙砾的种数和小偷背包的最大承重。

接下来 n 行，每行两个正整数 v_i, w_i ($1 \leq v_i \leq 10^7, 1 \leq w_i \leq 10^6$)，表示第 i 种沙砾的总价值和总重量。

输出格式

一行一个保留三位小数的实数，表示小偷可以获得的最大收益。

输入样例

```
3 6
1 1
1 3
2 4
```

输出样例

```
3.333
```

输出样例

```
3.333
```

样例解释

一种最优方案为偷走 1 吨第 1 种沙砾、1 吨第 2 种沙砾和 4 吨第 3 种沙砾。

Hint

大胆贪心，小心求证！

注意是 n 种，不是 n 根。

经典贪心问题，本题具体可以参考算法导论 P244 或者课件 PPT。

```
#include<cstdio>
#include<algorithm>
using namespace std;
int i,j,cnt = 0,n;
```

```
double w,v,k,res,add;
struct node {
    double w;
    double rate;
    inline bool operator < (const node& o)const {
        return o.rate < rate;
    }
} Node[10007];
int main() {

    scanf("%d %lf",&n,&k);

    for(int i = 0; i<n; i++) {
        scanf("%lf %lf",&v,&w);
        Node[i].w = w;
        Node[i].rate = v/w;
    }
    sort(Node,Node+n);
    cnt = 0;
    res = 0.0;

    while(k>0.0 && cnt < n && Node[cnt].rate>0) {
        add = min(k , Node[cnt].w);
        res += add*Node[cnt++].rate;
        k-=add;
    }

    printf("%.3lf\n",res);

    return 0;
}
```

E

E 暗中加密通话

时间限制：1000ms 内存限制：65536kb

通过率：29/50 (58.00%) 正确率：29/161 (18.01%)

题目描述

Zhoues 虽然是一名高级技师，但是依旧摆脱不了摸鱼的习惯。他听说小水獭已经进厂切钢条了，就私下和小水獭商量看看能不能设计一套加密方式加密二人间的通话，这样就可以方便摸鱼啦！

最后二人决定使用哈夫曼编码来给通话加密。数据结构这门课程中介绍到：「哈夫曼编码可以构造出字符的最优二进制表示」。那么给定一个长度为 n 的字符串，能不能求出该字符串对应的二进制哈夫曼编码的总长度呢？

输入格式

一行一个仅包含大小写英文字母的非空字符串，长度不超过 10^7 。

输出格式

一行一个正整数，表示该字符串对应的二进制哈夫曼编码的总长度。

输入样例 1

abababac

输出样例 1

12

输入样例 2

vOEwOVQQtIPatwEZPPOKtApNDdpJEFQChpJwNqQAFDYvwlFMOpqKczuWOTBxCINaVadetEVCaIhHJKIcBoEfsUqRUBGHZucFZIU

voEwOVQQtIPatwEZPPOKtApMDdpjEFQChpJwNqqAFDYvwLFWMOpqXczuWotBxCiNavadetEVCaihHJkI
cBoEFsUqRubGHZucfZlU

输出样例 2

535

样例解释

第一个样例中，a 出现了 4 次，b 出现了 3 次，c 出现了 1 次，加密后 a,b,c 的哈夫曼编码分别为 0,11,10 或 0,10,11，但总编码长度均为 $1 \times 4 + 2 \times 3 + 2 \times 1 = 12$ 。

Hint

大胆贪心，小心求证！

经典优先队列贪心问题，具体可以参考算法导论 P245 页。

```
#include<bits/stdc++.h>
#define maxn 10000002
using namespace std;
char s[maxn];
int fre[100],ans,t,i,sum,a,b,l;
priority_queue <int,vector<int>,greater<int> > q;
int main() {
    scanf("%s",s);
    l = strlen(s);
    for(i=0; i<l; i++) {
        fre[s[i]-'A']++;
    }
    for(i=0; i<100; i++) {
        if(fre[i]!=0) {
            q.push(fre[i]);
            sum++;
        }
    }
    if(sum==1) {
        printf("%d",l);
        return 0;
    }
    while(q.size()>1) {
        a = q.top();q.pop();
        b = q.top();q.pop();
        ans+=(a+b);q.push(a+b);
    }
    printf("%d",ans);
}
```

F 上班 I

时间限制：2000ms 内存限制：65536kb

通过率：79/114 (69.30%) 正确率：79/329 (24.01%)

题目描述

观赛完运动会之后，Zhoues 需要立马回去钢条厂上班，由于北航到钢条厂的路径有很多条，他希望选择一条最短的！

形式化来说，给一个 n 个点 m 条边的带权无向图，求点 s 到点 t 的最短路长度。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 2.5 \times 10^3, 1 \leq m \leq 7 \times 10^3, 1 \leq s, t \leq n$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^3$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环。

输出格式

输出一行一个非负整数，表示点 s 到点 t 的最短路长度。

输入样例

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

输出样例

```
7
```

Hint

请注意重边和自环的处理，不需要对最短路算法进行优化就可以通过本题。

经典图论问题，该题所有伪代码可以见算法导论或 PPT。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0',c=getchar();
```

```

        return X*w;
    }

    const ll N=10000000+10,M=15000000+10;

    ll n,m,s,t;
    struct edge { ll v,w,nxt; } e[M];
    ll head[N];
    void addEdge(ll u,ll v,ll w) {
        static ll cnt=0;
        e[++cnt]=(edge){v,w,head[u]},head[u]=cnt;
    }

    priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
    ll dis[N];
    void dijkstra(ll s) {
        memset(dis,0x3f,sizeof(dis));
        dis[s]=0,Q.push(make_pair(0,s));
        while (!Q.empty()) {
            ll u=Q.top().second,d=Q.top().first; Q.pop();
            if (d!=dis[u]) continue;
            for (ll i=head[u];i;i=e[i].nxt) {
                ll v=e[i].v,w=e[i].w;
                if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
            }
        }
    }

    int main() {
        n=read(),m=read(),s=read(),t=read();
        for (ll i=1;i<=m;++i) {
            ll u=read(),v=read(),w=read();
            addEdge(u,v,w);
            addEdge(v,u,w);
        }
        dijkstra(s);
        printf("%lld",dis[t]);
        return 0;
    }

```

题目描述

小水獭正在浏览沙河校区的蓝园（Blue Graden）。

小水獭想将蓝园中 n 朵花发往学院路，这 n 朵花的重量分别为 $a_1, a_2, \dots, a_n$ ，一个快递可以运送任意一朵花，或运送两朵重量之和不超过 m 的花。

小水獭想知道，它最少需要发几个快递才能将所有的花发往学院路，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 10^9$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq m$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个正整数，表示最少需要的快递数量。

输入

2
5 5
1 2 3 4 5
2 3
2 2

输出

3
2

样例解释

第一组数据中，一种最优方案为 $[1, 4], [2, 3], [5]$ 。

第二组数据中，一种最优方案为 $[1], [2]$ 。

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
int a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        sort(a + 1, a + 1 + n);
        int ans = 0, l = 1, r = n;
        while(l <= r){
            if(l == r || a[l] + a[r] <= m) l++, r--;
            else r--;
        }
        ans++;
    }
    printf("%d\n", ans);
}
```

```
        ans++;
    }
    printf("%d\n", ans);
}
}
```

H

H XIAO7 和奇怪的伞

时间限制：1000ms 内存限制：65536kb

通过率：9/17 (52.94%) 正确率：9/53 (16.98%)

题目描述

10^9 只猫猫站成了一排，其中前 m 只是属于 XIAO7 的。这时突然下雨了，XIAO7 变出了 n 张奇怪的大伞，其中第 i 张大伞能遮住第 $a_i, a_i + 1, \dots, b_i - 1, b_i$ 只猫猫，请问最少用多少张大伞才能遮住所有 XIAO7 的猫猫。

如果无论如何都不能遮住所有 XIAO7 的猫猫，请输出 $(=> <=)$ 。

输入格式

第一行两个整数 n, m ($1 \leq n \leq 10^6, 0 \leq m \leq 10^9$)，含义同题目描述。

接下来 n 行，每行两个正整数 a_i, b_i ($1 \leq a_i \leq b_i \leq 10^9$)，表示第 i 张大伞能遮住的猫猫范围。

输出格式

输出一行：

- 若存在一种方案可以遮住所有 XIAO7 的猫猫，输出一个非负整数，表示最少需要的伞的数量。
- 否则，输出 $(=> <=)$ 。

输入样例

```
3 5
1 2
2 4
3 6
```

输出样例

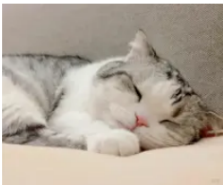
```
2
```

样例解释

一种最优方案为选择第 1 张伞和第 3 张伞。

Hint

XIAO7：发出喵喵的声音~



AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
```

```

#include<set>
#include<ctime>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
pi a[N];
int main(){
    int n = getint(), t = getint();
    for(int i = 1; i <= n; i ++){
        a[i].first = getint(), a[i].second = getint();
        sort(a + 1, a + n + 1);

        if(!t) {
            puts("0"); return 0;
        }

        int ed = 0, ans = 0;
        for(int i = 1; i <= n && ed < t;) {
            int lasted = ed + 1;
            for(; a[i].first <= lasted && i <= n; i ++){
                ed = max(ed, a[i].second);
            }
            if(lasted == ed + 1) break ;
            ans ++;
        }
        if(ed < t)
            puts("(=> <=)");
        else cout << ans << endl;
    }
}

```

I 莫卡的蛇蛇数列

时间限制：1000ms 内存限制：65536kb

通过率：5/7 (71.43%) 正确率：5/10 (50.00%)

题目描述

Serval 是加帕里幼儿园的新生，而莫卡是旭丘幼儿园的小朋友。

莫卡有一个长度为 n 的数列 $a_1, a_2, \dots, a_n$ ，Serval 想将这个数列变为**蛇蛇数列**。对于一个数列 $a_1, a_2, \dots, a_n$ ，Serval 定义这个数列是蛇蛇数列当且仅当这个数列满足以下条件：

- 对于满足 $2k \leq n$ 的所有正整数 k ，有 $a_{2k} > a_{2k-1}$ 。
- 对于满足 $2k + 1 \leq n$ 的所有正整数 k ，有 $a_{2k} > a_{2k+1}$ 。

为了将莫卡的数列变为蛇蛇数列，Serval 每次可以选择某个 a_i ，花费 1 的代价将 a_i 变为 $a_i + 1$ 或者 $a_i - 1$ 。需要注意的是，蛇蛇数列中的元素不必一定为正整数。

聪明的 Serval 当然知道将莫卡的数列变为蛇蛇数列的最小代价，但是莫卡并不知道。你能帮莫卡求出最小代价吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示莫卡的数列长度。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)，表示莫卡的数列元素。

输出格式

对于每组数据，输出一行一个非负整数，表示将莫卡的数列变为蛇蛇数列的最小代价。

输入样例

```
2
3
1 2 3
4
```

输出样例

```
2
6
```

样例解释

第一组数据中，可以选择 a_3 ，对其进行 -1 操作 2 次，最后的序列为 $[1, 2, 1]$ 。

第二组数据中，可以先选择 a_2 ，对其进行 $+1$ 操作 4 次，再选择 a_4 ，对其进行 $+1$ 操作 2 次，最后的序列为 $[4, 5, 3, 4]$ 。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        int n,i,x;
        long long Ans=0;
        scanf("%d",&n);
        scanf("%d",&x);
        for (i=2;i<=n;i++){
            int y;
            scanf("%d",&y);
            if (i&1){
                int t=max(y-x+1,0);
                y-=t;
                Ans+=t;
            }else{
                int t=max(x+1-y,0);
                y+=t;
                Ans+=t;
            }
        }
    }
}
```

```
        x=y;
    }
    printf("%lld\n",Ans);
}
return 0;
}
```

J

J 小水獭和最大值 III

时间限制：1000ms 内存限制：65536kb

通过率：2/4 (50.00%) 正确率：2/6 (33.33%)

题目描述



小水獭正在学习动态规划。

给定一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，小水獭初始位于 $(1,1)$ ，目标位置为 (n,m) ，它每次可以向下移动或向右移动，即从 (x,y) 移动到 $(x+1,y)$ 或 $(x,y+1)$ ，但不能移动到矩阵之外。它一共经过了 $n+m-1$ 个位置，将这些位置的元素进行按位与运算得到 A ，求 A 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n, m \leq 200$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($0 \leq a_{i,j} < 2^{30}$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示 A 的最大值。

输出格式

对于每组数据，输出一行一个非负整数，表示 A 的最大值。

输入样例

```
2
2 2
1 2
3 4
2 2
7 4
3 1
```

输出样例

```
0
1
```

```
#include <stdio.h>
#include <string.h>
#define maxn 205

int n, m;
int a[maxn][maxn];
int tag[maxn][maxn], g[maxn][maxn];
int f[maxn][maxn];

int solve(){
```

```

    if(tag[1][1]) return 0;
    f[1][1] = 1;
    int i, j;
    for(i = 1; i <= n; i++){
        for(j = 1; j <= m; j++){
            if(tag[i][j]) continue;
            f[i][j] |= f[i - 1][j] | f[i][j - 1];
        }
    }
    return f[n][m];
}

int main(){
    int t, ans, i, j, x;
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        memset(tag, 0, sizeof(tag));
        ans = 0;
        for(i = 1; i <= n; i++){
            for(j = 1; j <= m; j++){
                scanf("%d", &a[i][j]);
            }
        }
        for(x = 1 << 29; x >= 1){
            memset(f, 0, sizeof(f));
            memset(g, 0, sizeof(g));
            for(i = 1; i <= n; i++){
                for(j = 1; j <= m; j++){
                    if(tag[i][j]) continue;
                    if(!(a[i][j] & x)){
                        g[i][j] = tag[i][j] = 1;
                    }
                }
            }
            if(solve()){
                ans += x;
            }else{
                for(i = 1; i <= n; i++){
                    for(j = 1; j <= m; j++){
                        if(g[i][j]) tag[i][j] = 0;
                    }
                }
            }
        }
        printf("%d\n", ans);
    }
}

```

K XIAO7 和奇怪的猫窝

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/6 (83.33%) 正确率: 5/20 (25.00%)

题目描述

XIAO7 购买了一个奇怪的猫窝，虽然它能够装下全部的猫猫，但是太拥挤了还是会影响猫猫的心情。

第 i 只猫猫单独住进猫窝的快乐程度是 a_i ，除它本身每多住进来一只猫猫，她的快乐程度就会减去 b_i 。

现在 XIAO7 想知道选择哪些猫猫，能使猫窝里所有猫的快乐程度之和最大。

输入格式

第一行一个整数 n ($1 \leq n \leq 10^3$)，表示猫猫的数量。

接下来 n 行，每行两个整数 a_i, b_i ($1 \leq a_i, b_i \leq 10^9$)，含义同题目描述。

输出格式

第一行一个正整数表示最大的快乐程度之和。

第二行一个正整数表示选中的猫猫数量，如果有多种可能的答案，请输出猫猫数量的最大值。

输入样例

```
5
10 2
10 2
10 2
10 2
9 3
```

输出样例

```
18
3
```

样例解释

选择 1, 2, 3 号猫猫，每只猫猫的快乐程度都是 $10 - 2 \times 2 = 6$ ，快乐程度之和为 18。

Hint

XIAO7: 发出喵喵的声音~



AUTHOR: YUKI

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2000;
char s[N];
long long a[N];
struct fgo{
    int x;
    int y;
}f[N];
bool cmp(long long x,long long y)
{
    return x>y;
}
int main()
{
```

```

int t=1,n;
while(t--)
{
    scanf("%d",&n);
    for(int i=0;i<n;i++)
    {
        scanf("%d%d",&f[i].x,&f[i].y);
    }
    long long ans=0;
    int ansi=1;
    for(int i=n;i>=1;i--)
    {
        long long tmp=0;
        for(int j=0;j<n;j++)
        {
            a[j]=(long long)f[j].x-(long long)f[j].y*(i-1);
        }
        sort(a,a+n,cmp);
        for(int j=0;j<i;j++)
        {
            tmp+=a[j];
        }
        if(ans<=tmp)
        {
            ans=tmp;
            ansi=i;
        }
    }
    printf("%lld\n",ans);
    printf("%d",ansi);
}
}

```

L

L XIAO7 和奇怪的排列

时间限制: 1000ms 内存限制: 204800kb

通过率: 1/1 (100.00%) 正确率: 1/2 (50.00%)

题目描述

对于 1 到 n 的排列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_n$, 定义 $\text{MAX}(a, b) = \sum_{i=1}^n \max(a_i, b_i)$ 。对于给定的非负整数 K , 若 $\text{MAX}(a, b) \leq K$, XIAO7 则认为 (a, b) 是一对奇怪的排列。

给定整数 n 和 K , 求有多少对 1 到 n 的奇怪的排列。由于答案可能很大, 你只需要输出答案对 998 244 253 取模后的结果。

输入格式

一行两个整数 n, K ($1 \leq n \leq 50, 0 \leq K \leq 2500$), 含义同题目描述。

输出格式

一行一个非负整数, 表示答案对 998 244 253 取模后的结果。

输入样例

2 3

输出样例

2

输入样例

2 3

输出样例

2

样例解释

1 到 2 的排列 $[1, 2]$ 和 $[2, 1]$ 两种, 对于 4 对排列来说:

- $\text{MAX}([1, 2], [1, 2]) = 1 + 2 = 3$;
- $\text{MAX}([1, 2], [2, 1]) = 2 + 2 = 4$;
- $\text{MAX}([2, 1], [1, 2]) = 2 + 2 = 4$;
- $\text{MAX}([2, 1], [2, 1]) = 2 + 1 = 3$;

因此满足 $\text{MAX}(a, b) \leq 3$ 的奇怪的排列有两对。

Hint

XIAO7: 发出喵喵的声音~



AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<assert.h>
using namespace std;
typedef long long ll;
```

```

typedef pair<int, ll> pi;
const int N = 55;
const int inf = 0x3f3f3f3f;
const int mod = 998244253;
int getint(){
    char c; bool flag=0; int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9'){num=num*10+c-48; c=getchar();}
    return flag ? -num : num;
}
void add(int &a, int b) {
    a += b; if(a >= mod) a -= mod;
}
void sub(int &a, int b) {
    a -= b; if(a < 0) a += mod;
}
int dp[N][N][N * N];
int main(){

    int n = getint(), k = getint();
    int mid = (n + 1) >> 1, len = n >> 1;
    int Max = (n & 1) * mid + (mid + 1 + n) * len;
    dp[1][0][n] = n;
    dp[0][1][2 * n] = n * (n - 1);
    for(int i = 1; i < n; i++) {
        int val = n - i;
        for(int full = 0; full <= i; full++) {
            int half = i - full; //half = (2 * i - full * 2) / 2
            int empty = n - full - 2 * half;
            if(empty < 0) continue;
            for(int sum = 0; sum <= Max; sum++) if(dp[full][half][sum]) {
                if(full + 1 <= n && sum + val <= Max) {
                    //两个i放成一个新的full
                    add(dp[full + 1][half][sum + val],
                        1ll * dp[full][half][sum] * empty % mod);
                    //一个放half, 一个放empty, 生成一个full
                    add(dp[full + 1][half][sum + val],
                        1ll * dp[full][half][sum] * empty % mod * half * 2 % mod);
                }
                //两个i都放在half, 生成两个full, 减少一个half
                if(full + 2 <= n && half)
                    add(dp[full + 2][half - 1][sum],
                        1ll * dp[full][half][sum]
                        * (1ll * half * half % mod) % mod);
                //两个分别放在不同的empty中, 增加一个half
                if(half + 1 <= n && sum + 2 * val <= Max)
                    add(dp[full][half + 1][sum + 2 * val],
                        1ll * dp[full][half][sum] *
                        (1ll * empty * (empty - 1) % mod) % mod);
            }
        }
    }

    int ans = 0;
    for(int i = 0; i <= k; i++)

```

```
        add(ans, dp[n][0][i]);
    printf("%d\n", ans);
}
```

C5

A

A 小水獭和最长最短路

时间限制：2000ms 内存限制：65536kb

通过率：161/177 (90.96%) 正确率：161/422 (38.15%)

题目描述



小水獭发现一种奇特的路径，名为最长最短路。

具体来说，给定一个 n 个点 m 条边的无向连通图，设节点为 $1, 2, \dots, n$ ， $d(i, j)$ 表示节点 i 到节点 j 的最短路长度，记 $M = \max_{1 \leq i, j \leq n} d(i, j)$ ，请你按字典序从小到大输出所有点对 (i, j) 满足 $d(i, j) = M$ 。

(i, j) 的字典序小于 (i', j') 当且仅当 $i < i'$ 或 $(i = i') \wedge (j < j')$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 300, n - 1 \leq m \leq \frac{n(n-1)}{2}$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^6$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证给出的图没有重边和自环，且是连通的。

输出格式

对于每组数据，假设有 k 个点对满足条件，先输出一行两个正整数 M 和 k ，再输出 k 行，每行两个正整数表示对应的点对。你需要保证输出的顺序满足按字典序从小到大。

输入样例

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

输出样例

```
2 2
2 3
3 2
1 6
1 2
1 3
2 1
2 3
3 1
3 2
```

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

```
#include <bits/stdc++.h>
#define maxn 305

using namespace std;

int t, n, m;
int d[maxn][maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) d[i][j] = i == j ? 0
: 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            d[x][y] = d[y][x] = z;
        }
        for(int k = 1; k <= n; k++) for(int i = 1; i <= n; i++) for(int j = 1; j <=
n; j++) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        int M = 0, k = 0;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) M = max(M, d[i][j]);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(d[i][j] == M)
k++;
        printf("%d %d\n", M, k);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(d[i][j] == M)
printf("%d %d\n", i, j);
    }
}
```

B 计算几何大难题

时间限制: 1000ms 内存限制: 65536kb

通过率: 123/176 (69.89%) 正确率: 123/788 (15.61%)

题目描述

给定平面上三个点，求这三个点围成的三角形的面积。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，共三行，每行两个整数 ($0 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。保证三点不共线。

输出格式

对于每组数据，输出一行一个浮点数，保留一位小数，表示三个点围成的三角形面积。

输入样例

```
1
0 0
0 1
1 0
```

输出样例

```
0.5
```

Hint

本题采用浮点数会有精度问题，请考虑其他做法。

```
#include <bits/stdc++.h>

typedef long long ll;

using namespace std;

inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int x[3], y[3];

void solve(){
    for(int i = 0; i < 3; i++){
        x[i] = read(), y[i] = read();
    }
    for(int i = 1; i < 3; i++) y[i] -= y[0], x[i] -= x[0];
    ll ans = abs(1ll * y[2] * x[1] - 1ll * y[1] * x[2]);
    printf("%lld.%.1d\n", ans / 2, ans & 1 ? 5 : 0);
    assert(ans != 0);
}

int main(){
    int t = read();
    while(t--){
```

```
        solve();
    }
    return 0;
}
```

C

C 这是一道凸包模板题

时间限制: 1000ms 内存限制: 65536kb

通过率: 171/172 (99.42%) 正确率: 171/299 (57.19%)

题目描述

如题，这是一道凸包的模板题，在二维平面中有 n 个点，请你求出这 n 个点构成凸包的周长。

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^3$)，表示平面中点的个数。

接下来 n 行，每行两个整数 x_i, y_i ($0 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。

保证任意三点不共线。

输出格式

对于每组数据，输出一行一个浮点数，表示 n 个点构成凸包的周长。当且仅当你的答案和正确答案的绝对误差或相对误差不超过 10^{-6} 时，你的答案会被视为正确，即假设你的答案为 a ，正确答案为 b ，当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$ 时你的答案会被视为正确答案。

```
2
4
0 0
0 1
1 0
1 1
5
0 0
1 2
-2 -1
-1 3
0 2
```

```
4.0000000000
10.8313095581
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2000;
int a[N];
const double eps = 1e-8;
const double Pi = acos(-1.0);
int dcmp(const double x) {
    if(fabs(x)<=eps)return 0;
    return x>0?1:-1;
}
```

```

struct Point {
    double x,y;
    Point() {}
    Point(double x,double y):x(x),y(y) {}
    Point operator + (const Point&a)const {
        return Point(x+a.x,y+a.y);
    }
    Point operator - (const Point&a)const {
        return Point(x-a.x,y-a.y);
    }
    Point operator * (const double&a)const {
        return Point(x*a,y*a);
    }
    Point operator / (const double&a)const {
        return Point(x/a,y/a);
    }
    double operator * (const Point&a)const {
        return x*a.x+y*a.y;
    }
    double operator ^ (const Point&a)const {
        return x*a.y-y*a.x;
    }
    bool operator < (const Point&a)const {
        return (x!=a.x)?x<a.x:y<a.y;
    }
    bool operator == (const Point&a)const {
        return !dcmp(x-a.x)&&!dcmp(y-a.y);
    }
} p[N],s[N];
typedef Point Vector;
double lth(const Vector a) {
    return sqrt(a*a);
}
double S_(const Point a,const Point b,const Point c) {
    return ((b-a)^(c-a))/2;
}
int build(int n,Point *p,Point *s) {
    sort(p,p+n);
    int m = 0;
    for(int i = 0; i < n; i++) {
        while(m>1&&((s[m]-s[m-1])^(p[i]-s[m-1]))<=0)m--;
        s[++m] = p[i];
    }
    int lim = m;
    for(int i = n-2; i >= 0; i--) {
        while(m>lim&&((s[m]-s[m-1])^(p[i]-s[m-1]))<=0)m--;
        s[++m] = p[i];
    }
    if(m!=1)m--;
    return m;
}
int main() {
    int t=1,n;
    long long x1,y1,x2,y2,x3,y3;
    scanf("%d",&t);

```

```

while(t--){
    scanf("%d",&n);
    for(int i=0; i<n; i++){
        scanf("%lf%lf",&p[i].x,&p[i].y);
    }
    int m=build(n,p,s);
    s[0]=s[m];
    double ans=0;
    for(int i=1; i<=m; i++){
        ans+=lth(s[i]-s[i-1]);
    }
    printf("%.10lf",ans);
}
}

```

D

D Template of Geometry

时间限制: 1000ms 内存限制: 65536kb

通过率: 6/16 (37.50%) 正确率: 6/62 (9.68%)

题目描述

Bazoka13 在整理 [We will fail] 队的计算几何模板, 但是准备考研占据了 Bazoka13 大量时间, 于是他请莫卡来帮他整理关于直线和线段位置关系部分的模板。然而莫卡对计算几何完全不了解, 于是她想请聪明的你帮她实现对直线和线段位置关系的判断。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$), 表示数据组数。

对于每组数据, 共两行, 每行五个整数 $type, x_s, y_s, x_e, y_e$ ($type \in \{1, 2\}$, $-10^5 \leq x_s, y_s, x_e, y_e \leq 10^5$), 分别表示两条直线或线段; 其中后四个整数代表二维平面上两个点 $S(x_s, y_s)$ 和 $E(x_e, y_e)$ 的坐标, 保证两点坐标不同; 如果 $type = 1$, 表示该行数据为一条过点 S 和点 E 的直线, 如果 $type = 2$, 表示该行数据为一条以点 S 和点 E 为端点的线段。

输出格式

对于每组数据, 输出一行:

- 若为两条直线, 如果两条直线平行不共线, 输出 `Parallel`; 如果共线, 输出 `Coincide`; 如果相交, 输出 `Intersect`。
- 若为一条直线和一条线段, 如果非严格相交 (即在线段的端点处相交、或共线), 输出 `Not strict`; 如果严格相交, 输出 `Strict`; 如果不相交, 输出 `Don't intersect`。
- 若为两条线段, 如果非严格相交 (即在线段的端点处相交、或共线), 输出 `Not strict`; 如果严格相交, 输出 `Strict`; 如果不相交, 输出 `Don't intersect`。

输入样例

```

4
1 0 0 2 0
2 1 0 3 0
1 0 0 2 0
1 -1 -1 1 2
2 0 0 2 0
2 1 0 3 0
1 0 0 2 0
1 1 0 3 0

```

```

Not strict
Intersect
Not strict
Coincide

```

```

#include<bits/stdc++.h>
using namespace std;
#define inf 1e100
#define ll long long
#define eps 1e-8
//ÓÃÓÚ,;µãÊÝÕÝ,°ÀÐ¶İİf¬,ù¼ÝİâÄ¿¼«¶ÊÐÐ,Ä
const double pi = acos(-1.0);//Ô²ÖÜÂÊ
int sgn(ll x) {
    if(x==0)return 0;
    return x<0?-1:1;
}
//Ê¹ÓÃPointÊ±x¢Ôâ²¿·Ö°ÊÝÊÇ·µ»ØÐÃPoint¶ø·ÇÐÐ,Ä±¼ÉíÖµ
struct Point {
    ll x,y;
    /*¹¹Ôì°ÊÝ*/
    Point() {}
    Point(ll xx,ll yy) {
        x=xx;
        y=yy;
    }
    /*ÖÖÖÖ»Ð@µãµÄ»ù´;ÔÊÊã·û*/
    bool operator == (Point b)const {
        return sgn(x-b.x) == 0 && sgn(y-b.y) == 0;
    }
    bool operator < (Point b)const {
        return sgn(x-b.x)== 0?sgn(y-b.y)<0:x<b.x;
    }
    Point operator -(const Point &b)const {
        return Point(x-b.x,y-b.y);
    }
    Point operator +(const Point &b)const {
        return Point(x+b.x,y+b.y);
    }
    //²æ»ý
    ll operator ^(const Point &b)const {
        return x*b.y - y*b.x;
    }
    //µã»ý
    ll operator *(const Point &b)const {
        return x*b.x + y*b.y;
    }

    void input() {
        scanf("%lld%lld",&x,&y);
    }
};
struct Line {
    //Ã½µãÊ·¶Ö±İß
    Point s,e;
    Line() {}
    Line(Point ss,Point ee) {
        s=ss;
        e=ee;
    }

```

```

}
void input() {
    s.input();
    e.input();
}
//µãðúîßŋîéî
bool checkPS(Point p) {
    return sgn((p-s)^(e-s)) == 0 && sgn((p-s)*(p-e)) <= 0;
}
//ö±îßÆ½ðð
bool parallel(Line v) {
    return sgn((e-s)^(v.e-v.s)) == 0;
}
//µã°íö±îß'øîµ
//1  ôú×ó²à
//2  ôúôð²à
//3  ôúö±îßéî
int relation(Point p) {
    int c = sgn((p-s)^(e-s));
    if(c < 0)return 1;
    else if(c > 0)return 2;
    else return 3;
}
//îßŋîîà½»
//2  'æ·ŋîîà½»
//1  ·ç¹æ·ŋîîà½»
//0  ²»îà½»
int checkSS(Line v) {
    int d1 = sgn((e-s)^(v.s-s));
    int d2 = sgn((e-s)^(v.e-s));
    int d3 = sgn((v.e-v.s)^(s-v.s));
    int d4 = sgn((v.e-v.s)^(e-v.s));
    if( (d1^d2)==-2 && (d3^d4)==-2 )return 2;
    return (d1==0 && sgn((v.s-s)*(v.s-e))<=0) ||
           (d2==0 && sgn((v.e-s)*(v.e-e))<=0) ||
           (d3==0 && sgn((s-v.s)*(s-v.e))<=0) ||
           (d4==0 && sgn((e-v.s)*(e-v.e))<=0);
}
//ö±îß°íîßŋîîà½»
//2  'æ·ŋîîà½»
//1  ·ç¹æ·ŋîîà½»
//0  ²»îà½»
int checkLS(Line v) {
    int d1 = sgn((e-s)^(v.s-s));
    int d2 = sgn((e-s)^(v.e-s));
    if((d1^d2)==-2) return 2;
    return (d1==0||d2==0);
}
//Á½ö±îß'øîµ
//0  Æ½ðð
//1  öð°î
//2  îà½»
int checkLL(Line v) {
    if((*this).parallel(v))
        return v.relation(s)==3;
}

```

```

        return 2;
    }
};

int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int type1,type2,xs,ys,xs,ys,xs,ys,xs,ys;
        Line l1, l2;
        scanf("%d%d%d%d",&type1,&xs,&ys,&xe,&ye);
        l1 = Line(Point(xs,ys),Point(xe,ye));
        scanf("%d%d%d%d",&type2,&xs,&ys,&xe,&ye);
        l2 = Line(Point(xs,ys),Point(xe,ye));
        if(type1 == 1 && type2 == 1) {
            if(l1.checkLL(l2) == 0) {
                puts("Parallel");
            } else if(l1.checkLL(l2) == 1) {
                puts("Coincide");
            } else {
                puts("Intersect");
            }
        }
        else if(type1 == 1) {
            if(l1.checkLS(l2) == 2) {
                puts("Strict");
            } else if(l1.checkLS(l2) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
        else if(type2 == 1) {
            if(l2.checkLS(l1) == 2) {
                puts("Strict");
            } else if(l2.checkLS(l1) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
        else {
            if(l1.checkSS(l2) == 2) {
                puts("Strict");
            } else if(l1.checkSS(l2) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
    }
}

```

题目描述

XIAO7 不想努力了，于是她决定抓几只草兽来帮忙干活。

二维平面上有 n 只草兽，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 。

XIAO7 有一颗智慧宝珠，可以对一个圆形范围内的草兽进行计数。她会进行 q 次询问，每次询问会给定三个参数 X, Y, R ，你需要求解有多少草兽其坐标 (x_i, y_i) 满足 $(x_i - X)^2 + (y_i - Y)^2 \leq R^2$ 。

输入格式

第一行两个正整数 n, q ($1 \leq n, q \leq 5 \times 10^4$)，表示草兽的数量和询问的次数。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示草兽的坐标。可能有多个草兽坐标相同。

接下来 q 行，每行三个整数 X_i, Y_i, R_i ($1 \leq X_i, Y_i \leq 10^9, 1 \leq R_i \leq 10$)，表示第 i 次询问的参数。

输出格式

输出 q 行，第 i 行一个非负整数，表示第 i 次询问的答案。

```
5 3
0 0
1 0
1 1
1 2
2 2
1 1 1
3 3 2
3 2 2
```

```
3
1
2
```

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 100005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
```

```

}
map<pi, int> point;
ll sqr(int x) {
    return 1ll * x * x;
}
ll dis(int x1, int y1, int x2, int y2) {
    return sqr(x1 - x2) + sqr(y1 - y2);
}
int main(){

    int n = getint(), q = getint();

    for(int i = 1; i <= n; i ++) {
        int x = getint(), y = getint();

        if(point.count(make_pair(x, y)))
            point[make_pair(x, y)] ++;
        else
            point[make_pair(x, y)] = 1;
    }

    while(q --) {
        int x = getint(), y = getint(), r = getint(), ans = 0;

        for(int i = x - r; i <= x + r; i ++)
            for(int j = y - r; j <= y + r; j ++)
                if(dis(i, j, x, y) <= sqr(r) && point.count(make_pair(i, j))) {

                    ans += point[make_pair(i, j)];
                }
        printf("%d\n", ans);
    }
}

```

F

F 上班 III

时间限制：1000ms 内存限制：65536kb

通过率：55/81 (67.98%) 正确率：55/306 (17.97%)

10
4957541
4957260
4957192

题目描述

观赛完运动会之后，Zhoues 需要立马回去钢厂上班，由于北航到钢厂的路径有很多条，他希望选择一条最短的！

但是由于最近海淀区疫情比较严重，钢厂要求回去上班的员工必须做一次核酸检测，因此 Zhoues 必须要在 k 个核酸检测点 $s_1, s_2, \dots, s_k$ 中选取一个核酸检测点做完核酸之后才可以上班。

形式化地说，给定一个 n 个点 m 条边的无向图和 k 个指定节点，要求在至少经过一个指定节点的情况下，求点 1 到点 n 的最短路长度。

输入格式

第一行三个正整数 n, m, k ($1 \leq n \leq 10^5, 1 \leq m \leq 5 \times 10^5, 1 \leq k \leq n$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^3$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

接下来 k 行，每行一个正整数 s_i ($1 \leq s_i \leq n$)，表示第 i 个指定节点。保证这 k 个节点不同。

图中有可能存在重边和自环，保证至少存在一条从点 1 经过指定节点并到点 n 的路径。

输出格式

一行一个非负整数，表示在至少经过一个指定节点的情况下，点 1 到点 n 的最短路长度。

```
7 6 1
1 2 1
1 3 2
2 4 2
2 5 5
3 6 3
3 7 4
4
```

12

样例解释

在必须经过点 44 的情况下，最短路径为 $1 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 7$ 。

Hint

单源最短路 $\rightarrow$ 「双源」最短路。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0', c=getchar();
    return x*w;
}

const ll N=1000000+10, M=5000000+10;

ll n,m,k,t,ans = LLONG_MAX;
struct edge { ll v,w,nxt; } e[M];
ll head[N];
void addEdge(ll u,ll v,ll w) {
```

```

static ll cnt=0;
e[++cnt]=(edge){v,w,head[u]},head[u]=cnt;
}

priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
ll dis[N];
ll dis2[N];
void dijkstra(ll s) {
    memset(dis,0x3f,sizeof(dis));
    dis[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
        if (d!=dis[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
        }
    }
}
void dijkstra2(ll s) {
    memset(dis2,0x3f,sizeof(dis2));
    dis2[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
        if (d!=dis2[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis2[u]+w<dis2[v])
dis2[v]=dis2[u]+w,Q.push(make_pair(dis2[v],v));
        }
    }
}
int main() {
    n=read(),m=read(),k=read();
    for (ll i=1;i<=m;++i) {
        ll u=read(),v=read(),w=read();
        addEdge(u,v,w);
        addEdge(v,u,w);
    }
    dijkstra(1);
    dijkstra2(n);
    for (ll i=0;i<k;++i) {
        int t = read();
        ans = min(ans,dis[t] + dis2[t]);
    }
    printf("%lld",ans);
    return 0;
}

```

G 小水獭和贝叶斯网络

时间限制：2000ms 内存限制：65536kb

通过率：4/6 (66.67%) 正确率：4/13 (30.77%)

题目描述



小水獭接触到了贝叶斯网络，它现在有一个有向图，希望求出一个边数最多的无环生成子图作为贝叶斯网络。可以证明这个问题是 **NP** 的，因此小水獭希望你得到一个近似比为 $\frac{1}{2}$ 的算法。

具体来说，给定一个 n 个点 m 条边的有向图，请你保留全部的节点和至少 $\left\lceil \frac{m}{2} \right\rceil$ 条边，使得图中无环。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 $u_i \rightarrow v_i$ 。

保证给出的图没有重边和自环。

输出格式

对于每组数据，先输出一行一个正整数 k ($\left\lceil \frac{m}{2} \right\rceil \leq k \leq m$)，表示你所保留的边数。

接下来 k 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示保留的第 i 条边为 $u_i \rightarrow v_i$ 。

如果有多种可能的答案，你可以输出任意一种。

输入样例

```
2
4 4
1 2
1 3
1 4
2 4
4 4
1 2
2 3
3 4
4 1
```

输出样例

```
4
1 2
1 3
1 4
2 4
3
1 2
2 3
4 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        vector<pair<int, int> > v[2];
        while(m--){
```

```
        int x, y;
        scanf("%d%d", &x, &y);
        v[x < y].push_back({x, y});
    }
    if(v[0].size() < v[1].size()) swap(v[0], v[1]);
    printf("%d\n", v[0].size());
    for(auto it : v[0]) printf("%d %d\n", it.first, it.second);
}
}
```

H

H 平面最近点对 II

时间限制：2000ms 内存限制：204800kb

通过率：124/154 (80.52%) 正确率：124/427 (29.04%)

题目描述

Zhoues 发现二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，定义两点 (x_i, y_i) 和 (x_j, y_j) 的欧几里得距离为 $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。Zhoues 要求同学们求出任意两点间最小欧几里得距离，即：

$$\min_{1 \leq i < j \leq n} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

输入格式

第一行一个正整数 n ($2 \leq n \leq 5 \times 10^5$)，表示点的数量。
接下来 n 行，每行两个至多六位小数的浮点数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 10^6$)，表示第 i 个点的坐标。

输出格式

输出一行一个浮点数，保留四位小数，表示任意两点间最小欧几里得距离。

输入样例

```
3
0.00 0.00
2.00 2.00
1.00 3.00
```

输出格式

输出一行一个浮点数，保留四位小数，表示任意两点间最小欧几里得距离。

输入样例

```
3
0.00 0.00
2.00 2.00
1.00 3.00
```

输出样例

```
1.4142
```

Hint

经典分治问题，本题的所有伪代码具体可以参考算法导论 P613、课件 PPT 和仓库中 C2-A 的题解。

```
#include<iostream>
#include<cstdio>
#include<cstring>
#include<cmath>
#include<algorithm>
```

```

using namespace std;
struct Point{double x,y;}a[2000000],tmp[2000000];
int n,b[2000000];
const double inf=0x7fffffff;
double sqr(double x){return x*x;}
double dis(int l,int r){return sqr(a[l].x-a[r].x)+sqr(a[l].y-a[r].y);}
int cmp(const Point &a,const Point &b){return a.x==b.x?a.y<b.y:a.x<b.x;}
void merge(int l,int r)
{
    int mid=(l+r)>>1,i=l,j=mid+1;
    for(int k=l;k<=r;k++)
    {
        if(i<=mid&&(j>r||a[i].y<a[j].y))tmp[k]=a[i++];
        else tmp[k]=a[j++];
    }
    for(int i=l;i<=r;i++)a[i]=tmp[i];
}
double solve(int l,int r)
{
    if(l>=r)return inf;
    if(l+1==r){if(a[l].y>a[r].y)swap(a[l],a[r]);return dis(l,r);}
    int mid=(l+r)>>1,t=a[mid].x,cnt=0;
    double d=min(solve(l,mid),solve(mid+1,r));
    merge(l,r);
    for(int i=l;i<=r;i++)
        if(sqr(a[i].x-t)<d)
            b[++cnt]=i;
    for(int i=1;i<=cnt;i++)
        for(int j=i+1;j<=cnt&&sqrt(a[b[j]].y-a[b[i]].y)<d;j++)
            d=min(d,dis(b[j],b[i]));
    return d;
}
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)scanf("%lf%lf",&a[i].x,&a[i].y);
    sort(a+1,a+n+1,cmp);
    printf("%.4lf",sqrt(solve(1,n)));
}

```

I XIAO7 和猫瘟公司

时间限制：3000ms 内存限制：131072kb

通过率：4/7 (57.14%) 正确率：4/13 (30.77%)

题目描述

XIAO7 的猫娘乐园里发生了猫瘟病毒泄露，她需要尽快逃出去！

猫娘乐园里一共有 n 栋猫舍，猫舍之间有 m 条单向的道路，每条道路都有一个长度和年龄。雪风所在的 1 号猫舍发生了病毒泄露，位于 n 号猫舍的 XIAO7 在得知情况后决定抛弃猫娘乐园逃跑，总共需要 T 秒才能彻底逃离。

为了帮助 XIAO7 成功逃离，你需要拆除猫娘乐园的部分道路，使得从 1 号猫舍到 n 号猫舍的最短路长度至少为 T （或使得 1 号猫舍无法到达 n 号猫舍）。拆除道路对猫娘乐园的破坏程度定义为被拆除的道路中最古老的一条路的年龄。XIAO7 希望在能成功逃离的基础上对猫娘乐园的破坏程度尽可能小，请你帮她！

输入格式

第一行三个整数 n, m, T ($1 \leq n \leq 5 \times 10^4, 1 \leq m \leq 2 \times 10^5, 0 \leq T \leq 10^9$)，含义同题目描述。

接下来 m 行，每行四个正整数 u_i, v_i, l_i, a_i ($1 \leq u_i, v_i \leq n, 1 \leq l_i, a_i \leq 10^9$)，表示第 i 条边 $u_i \rightarrow v_i$ 的长度和年龄分别为 l_i 和 a_i 。

图中有可能存在重边和自环，保证至少存在一条从 1 号猫舍到 n 号猫舍的路径。

输出格式

输出一行：

- 若需要拆除道路则输出一个正整数，表示最小破坏程度。
- 否则，输出 `0 v 0` 和从 1 到 n 的最短路长度（显然，它应该 $\geq T$ ）。

输入样例 1

```
5 6 12
1 2 5 10
2 3 6 20
1 4 4 15
4 3 8 25
3 5 2 18
4 5 7 12
```

输出样例 1

```
12
```

输入样例 2

```
3 2 10
1 2 8 30
2 3 6 45
```

输出样例 2

```
0 v 0 14
```

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<ll, int> pi;
const int N = 100005;
const int M = 2000005;
const ll inf = 1ll << 60;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int n, m, T, cnt, fir[N], tar[M], nxt[M], w[M], age[M], A[M];
ll dis[N]; bool vis[N];
priority_queue<pi, vector<pi>, greater<pi> > q;
```

```

void link(int a, int b, int c, int A) {
    tar[++ cnt] = b, w[cnt] = c, age[cnt] = A;
    nxt[cnt] = fir[a], fir[a] = cnt;
}

bool dij(int x, int ed, int mx) {
    for(int i = 1; i <= n; i ++)
        dis[i] = inf, vis[i] = 0;
    q.push(make_pair(dis[x] = 0, x));
    while(! q.empty()) {
        x = q.top().second, q.pop();
        if(vis[x]) continue ;

        vis[x] = 1;
        for(int i = fir[x]; i; i = nxt[i])
            if(age[i] > mx && dis[tar[i]] > dis[x] + w[i]) {
                dis[tar[i]] = dis[x] + w[i];
                q.push(make_pair(dis[tar[i]], tar[i]));
            }
    }
    return dis[ed] >= T;
}

int main(){

    n = getint(), m = getint(), T = getint();
    for(int i = 1; i <= m; i ++) {
        int x = getint(), y = getint();
        int len = getint(), age = getint();
        link(x, y, len, A[i] = age);
    }
    sort(A + 1, A + m + 1);
    if(dij(1, n, -1)) {
        printf("o v o %lld\n", dis[n]);
        return 0;
    }

    int l = 1, r = m;
    while(l < r) {
        int mid = (l + r) >> 1;
        if(dij(1, n, A[mid]))
            r = mid;
        else l = mid + 1;
    }
    cout << A[l] << endl;
}

```

题目描述

莫卡学会了一个新的魔法——姆修派斯光线!

莫卡来到了练习场练习这个魔法。练习场可以视为一个无穷大的二维平面，并且上面分布着 n 个练习人偶。姆修派斯光线比较奇特，当有两个不同的练习人偶和莫卡在一条直线上时，莫卡可以释放该魔法并摧毁这两个练习人偶。

莫卡在练习场中标出了 q 个位置，每次练习莫卡会在当前位置不断地释放魔法并摧毁练习人偶，直到释放条件无法满足为止。之后练习人偶会在原来的位置恢复，莫卡会传送到下一个位置继续练习。

莫卡想知道，对于每个位置，她最多能够释放几次魔法？

输入格式

第一行一个整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^3$)，表示练习人偶数量。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示第 i 个练习人偶的位置。

接下来一行一个正整数 q ($1 \leq q \leq 100$)，表示莫卡练习位置的数量。

接下来 q 行，每行两个整数 X_i, Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$)，代表第 i 个练习位置的坐标。

输出格式

对于每组数据，输出 q 行，第 i 行一个非负整数，表示第 i 个练习位置最多能够释放几次魔法。

输入样例

```
1
4
2 2
-2 -2
2 -2
-2 2
2
0 0
2 0
```

输出样例

```
2
1
```

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
int sgn(ll x) {
    if(x==0)return 0;
    return x<0?-1:1;
}
struct Point {
    ll x,y;
    Point() {}
    Point(ll xx,ll yy) {
        x=xx;
        y=yy;
    }
    void input() {
        scanf("%lld %lld",&x,&y);
    }
    bool operator ==(const Point &p)const {
        return sgn(x-p.x)==0&&sgn(y-p.y)==0;
    }
    bool operator !=(const Point &p)const {
        return sgn(x-p.x)!=0||sgn(y-p.y)!=0;
    }
};
```

```

    }
    bool operator < (const Point &p)const {
        return sgn(x-p.x)<0||(sgn(x-p.x)==0&&sgn(y-p.y)<0);
    }
    Point operator + (const Point &p)const {
        return {x+p.x,y+p.y};
    }
    Point operator - (const Point &p)const {
        return {x-p.x,y-p.y};
    }
    Point operator * (ll k)const {
        return {x*k,y*k};
    }
    Point operator / (const ll &k)const {
        return {x/k,y/k};
    }
    ll dot(Point p) const {
        return p.x*x+p.y*y;
    }
    ll cross(Point p) const {
        return x * p.y - y * p.x;
    }
};

ll cross(Point p1,Point p2,Point p3) {
    return (p2-p1).cross(p3-p1);
}

ll dot(Point p1,Point p2,Point p3) {
    return (p2-p1).dot(p3-p1);
}

struct Frac {
    ll a,b;
    Frac() {}
    Frac(ll aa,ll bb) {
        if(aa!=0&&bb!=0) {
            ll g=__gcd(abs(aa),abs(bb));
            a=aa/g;
            b=bb/g;
        }
        if(aa==0&&bb!=0) {
            a=0;
            b=bb/abs(bb);
        }
        if(aa!=0&&bb==0) {
            b=0;
            a=aa/abs(aa);
        }
    }
};

map<pair<ll,ll>,int> mp;
vector<Point> ps;
int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int n;

```

```

    ps.clear();
    scanf("%d",&n);
    for(int i=1; i<=n; i++) {
        Point tmp;
        tmp.input();
        ps.push_back(tmp);
    }
    int q;
    scanf("%d",&q);
    while(q--) {
        Point p;
        p.input();
        mp.clear();
        for(auto i:ps) {
            Frac f=Frac(i.x-p.x,i.y-p.y);
            mp[ {f.a,f.b} ]++;
        }
        ll ans=0;
        for(auto i:ps) {
            Frac f=Frac(i.x-p.x,i.y-p.y);
            int sz=mp[ {-f.a,-f.b} ]+mp[ {f.a,f.b} ];
            if(sz>=2)ans=ans+sz/2;
            mp[ {-f.a,-f.b} ]=0;
            mp[ {f.a,f.b} ]=0;
        }
        printf("%lld\n",ans);
    }
}

```

C6

A

A Keyboard Magic Power

时间限制：1000ms 内存限制：65536kb

通过率：180/194 (92.78%) 正确率：180/457 (39.39%)

题目描述

the_ignorant 在写编程题时，总喜欢用键盘上的一些神秘代码来辅助编程，比如 Ctrl-C，Ctrl-V，Ctrl-F。其中，Ctrl-F 的功能是在现有的字符串 s 中查找输入字符串 t 出现的所有位置（非常适合查找原题）。

他在使用 Ctrl-F 时，却发现键盘失灵了。但万幸的是，他知道键盘中蕴含着一种神秘力量（Keyboard Magic Power）能替代 Ctrl-F 解决问题。他希望你帮帮他，去发掘出这种神秘力量吧！

输入格式

第一行一个非空字符串 s ，表示匹配串（主串）。

第二行一个非空字符串 t ，表示模式串（子串）。

两个字符串的长度均不超过 10^6 ，且由可见字符组成。

输出格式

一行从小到大若干个非负数，表示每次匹配成功后首字符的下标位置，其中下标从 0 开始。

输入样例 1

```
aaaaa
aa
```

输出样例 1

```
0 1 2 3
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2e6;
char s[N],t[N];
int a[N];
int nxt[N];
void kmp(int n,char *a,int m,char *b) {
    int i,j;
    for(nxt[0]=j=-1,i=1; i<n; nxt[i++]=j) {
        while(~j&&a[j+1]!=a[i])j=nxt[j];
        if(a[j+1]==a[i])j++;
    }
    for(j=-1,i=0; i<m; i++) {
        while(~j&&a[j+1]!=b[i])j=nxt[j];
        if(a[j+1]==b[i])j++;
        if(j==n-1)printf("%d ",i-n+1),j=nxt[j];
    }
}
int main() {
    scanf("%s",s);
    scanf("%s",t);
    int len1=strlen(t),len2=strlen(s);
    assert(len1>=1&&len1<=1e6);
    assert(len2>=1&&len2<=1e6);
    kmp(len1,t,len2,s);
}
```

Hint: KMP

B

B 小水獭和三点共线

时间限制: 1000ms 内存限制: 65536kb

通过率: 170/188 (90.43%) 正确率: 170/520 (32.69%)

题目描述



小水獭正在补习计算几何。

二维平面上有 n 个不同的点，小水獭请你帮它判断是否存在三点不共线？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^5$)，含义同题目描述。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。

保证任意两点坐标不同。

输出格式

对于每组数据，输出一行一个字符串：

- 如果存在三点不共线，输出 `how?`；
- 否则，输出 `boo how! boo how!`。

...

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n;
int x[maxn], y[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d%d", &x[i], &y[i]);
        bool tag = false;
        for(int i = 3; i <= n; i++){
            if(1ll * (x[i] - x[1]) * (y[i] - y[2]) != 1ll * (x[i] - x[2]) *
(y[i] - y[1])){
                tag = true;
                break;
            }
        }
        puts(tag ? "how?" : "boo how! boo how!");
    }
}
```

hint: 计算几何

时间限制：1000ms 内存限制：65536Kb

通过率：84/120 (70.00%) 正确率：84/269 (31.23%)

题目描述

the_ignorant 最近玩了一款名叫 **hsah** 的格斗游戏，它有一个奇怪的机制：输入一套连招后，系统会依次判断这套连招的前 i 个操作和后 i 个操作是否相同，每相同一次，伤害就会降低一次。
the_ignorant 希望你能告诉他，输入一套连招后，哪些 i 会使得伤害降低。

形式化地说，给定字符串 s ，请你求出对于 $i = 1, 2, \dots, |s|$ 而言，长度为 i 的前缀和长度为 i 后缀是否相同。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母、大写字母和数字组成的字符串 s ($1 \leq |s| \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行从小到大若干个正整数，表示符合条件的 i 。

输入样例

```
2
aaaaa
hsah
```

输出样例

```
1 2 3 4 5
1 4
```

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 2e6;
const int P=31,D=1000173169;
int n,i,p[N],f[N];
char a[N];
int Hash(int l,int r) {
    return (ll)(f[r]-(ll)f[l-1]*p[r-l+1]%D+D)%D;
}
int main() {
    int t;
    scanf("%d",&t);
    assert(t>=1&&t<=10);
    while(t--) {
        scanf("%s",a+1);
        int n=strlen(a+1);
        assert(n>=1&&n<=1e6);
        for(p[0]=i=1; i<=n; i++) {
            p[i]=(ll)p[i-1]*P%D;
        }
        for(i=1; i<=n; i++) {
            f[i]=(ll)((ll)f[i-1]*P+a[i])%D;
        }
        for(i=1; i<=n; i++) {
            if(Hash(1,i)==Hash(n-i+1,n)) {
                printf("%d ",i);
            }
        }
        puts("");
    }
}
```

hint: hash公式

D

D BellalaBella 和回文串

时间限制: 1000ms 内存限制: 65536kb

通过率: 177/185 (95.68%) 正确率: 177/362 (48.90%)

题目描述

Bellalabella 今天学习了回文串的内容，一个字符串是回文串当且仅当将其翻转后和原串相同。

Bellalabella 想知道对于一个字符串 S ，它的最长回文子串长度是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母组成的字符串 S ，含义同题目描述。

保证对于所有数据 $|S|$ 的和不超过 2×10^3 。

输出格式

对于每组数据，输出一行一个正整数，表示串 S 的最长回文子串长度。

输入样例

2
aba
aabb

输出样例

3
2

```
#include<bits/stdc++.h>
#define inf 0x7fffffff
#define RG register int
using namespace std;
typedef long long ll;
const ll mod = 998244353;
const long double eps = 1e-11;
//using namespace std;
inline ll read(){
    ll x=0,f=1;char ch=getchar();
    while(ch>'9' || ch<'0'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0' && ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
int n,w,h;
char ch[110005],pat[220005];
int pos,r[220005],ans;
int main(){
    int i,j,u,v;
    int t = read();
    while(t--){
        scanf("%s",ch+1);
        n=strlen(ch+1);
        pat[0]='@';
        ans = 0;
        for(int i = 0; i <= 2 * n + 2; i++) r[i] = 0;
        for(i=1;i<=n;i++) pat[2*i-1]=ch[i],pat[2*i]='@';
        pos=0,r[0]=0;
        for(i=1;i<=2*n;i++){
            if(i<=pos+r[pos]) r[i]=min(pos+r[pos]-i,r[pos*2-i]);
            while(i>r[i]&&pat[i+r[i]+1]==pat[i-r[i]-1]) r[i]++;
        }
        ans = max(ans,r[pos]);
    }
    printf("%d\n",ans);
}
```

```

        if(i+r[i]>pos+r[pos]) pos=i;
        ans=max(ans,r[i]);
    }
    printf("%d\n", ans);
}
return 0;
}

```

最长回文子串

E

E 小水獭和后缀数组

时间限制: 1000ms 内存限制: 65536kb

通过率: 27/49 (55.10%) 正确率: 27/104 (25.96%)

题目描述



小水獭正在预习字符串。

给定字符串 s ，设它的长度为 $|s|$ ， s_i 表示 s 的第 i 个字符。

s 有 $|s|$ 个后缀，第 i ($1 \leq i \leq |s|$) 个后缀为 $s_i s_{i+1} \cdots s_{|s|}$ ，小水獭想知道这 $|s|$ 个后缀中，字典序最小的后缀是哪一个呢？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母组成的字符串 s ($1 \leq |s| \leq 10^5$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个字符串，表示字典序最小的后缀。

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int a[maxn], b[maxn], c[maxn];
int SA[maxn], rnk[maxn];
void build(char *s, int n, int m) {
    int *x = a, *y = b;
    s[n + 1] = 0;
    for (int i = 0; i <= n * 2 && i < maxn; i++) a[i] = b[i] = SA[i] = rnk[i] = 0;
    for (int i = 1; i <= m; i++) c[i] = 0;
    for (int i = 1; i <= n; i++) c[x[i] = s[i]]++;
    for (int i = 1; i <= m; i++) c[i] += c[i - 1];
    for (int i = n; i; i--) SA[c[x[i]]--] = i;
    for (int j = 1, p = 0; j <= n && p != n + 1; j <= 1) {
        p = 0;
        for (int i = n - j + 1; i <= n; i++) y[++p] = i;
        for (int i = 1; i <= n; i++) if (SA[i] > j) y[++p] = SA[i] - j;
        for (int i = 1; i <= m; i++) c[i] = 0;
        for (int i = 1; i <= n; i++) c[x[y[i]]]++;
        for (int i = 1; i <= m; i++) c[i] += c[i - 1];
    }
}

```

```
        for (int i = n; i; i--) SA[c[x[y[i]]]--] = y[i];
        std::swap(x, y); x[SA[1]] = 1; p = 2;
        for (int i = 2; i <= n; i++) x[SA[i]] = (y[SA[i]] == y[SA[i - 1]] &&
y[SA[i] + j] == y[SA[i - 1] + j]) ? p - 1 : p++;
        m = p;
    }
    for (int i = 1; i <= n; i++) rnk[SA[i]] = i;
}

int t, n;
char s[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        build(s, n, 256);
        printf("%s\n", s + SA[1]);
    }
}
```

F

F BellalaBella 和多项式

时间限制：1000ms 内存限制：65536kb
通过率：13/14 (92.86%) 正确率：13/23 (56.52%)

题目描述

Bellalabella 在数学课上学习了函数的知识。由于 Bellalabella 只上了初中，她只接触到了一元多项式的知识。

老师想要 Bellalabella 对于给定的多项式 $f(x) = \sum_{i=0}^{n-1} a_i x^i$ 求出其除以另一个多项式 $g(x) = x + c$ 的商 $h(x) = \sum_{i=0}^{n-2} b_i x^i$ 和余数 r ，但是 Bellalabella 实在是太笨了，她只能寻求来自小水獭幼儿园的你解答这个问题。

形式化地说，设 $f(x) = h(x) \times (x + c) + r$ ，给定 $f(x)$ 的各项系数和常数 c ，请你求出 $h(x)$ 的各项系数和常数 r 。

输入格式

第一行两个整数 n, c ($1 \leq n \leq 10^6, 0 \leq |c| \leq 10^9$)，含义同题目描述。

第二行 n 个整数 $a_0, a_1, \dots, a_{n-1}$ ($0 \leq |a_i| \leq 10^9$)，含义同题目描述。

输出格式

一行 n 个整数 $b_0, b_1, \dots, b_{n-2}, r$ ，数据保证这 n 个整数的绝对值不超过 10^9 。

输入样例

3 1
1 2 1

输出样例

1 1 0

样例解释

$f(x) = 1 + 2x + x^2 = (x + 1) \times (x + 1) = h(x) \times (x + 1) + 0$ ，因此 $b_0 = b_1 = 1, r = 0$ 。

```
#include <cstdio>
#include <algorithm>
#include <cstring>

using namespace std;
```

```

const int N = 1e6 + 5;

int n;
long long c, a[N], b[N], r;

void solve()
{
    scanf("%d%lld", &n, &c);
    for (int i = 1; i <= n; i++)
        scanf("%lld", &a[i]);
    for (int i = n; i > 1; i--)
    {
        b[i - 1] = a[i];
        a[i] -= b[i - 1];
        a[i - 1] -= c * b[i - 1];
    }
    r = a[1];
    for (int i = 1; i < n; i++)
        printf("%lld ", b[i]);
    printf("%lld\n", r);
}

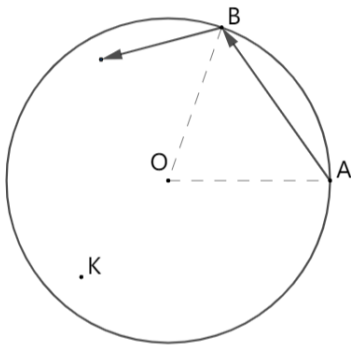
int main()
{
    int T = 1;
    // scanf("%d", &T);
    while (T--)
        solve();
    return 0;
}

```

G

题目描述

Zhoues 有一块神奇的圆形镜子，里面有一束永不消失的光。



光从圆 O 上一点 A 出发，在镜子内部不断沿逆时针反射。Zhoues 想知道在给定 $\angle ABO$ 的情况下，这一束光经过多少次反射后可以回到点 A 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^3$)，表示数据组数。

对于每组数据，一行两个正整数 a, b ($1 \leq a, b < 2^{31}$, $\frac{a}{b} < 90$)，表示 $\angle ABO$ 的角度制大小为 $\frac{a}{b}$ 度。

输出格式

对于每组数据，输出一行一个正整数，表示反射次数。

...

```
#include <bits/stdc++.h>
using namespace std;
int t;
long long a,b,n,m;

int main()
{
    scanf("%d",&t);
    while(t--)
    {
        scanf("%d%d",&a, &b);
        n=180*b-2*a;
        m=360*b;
        printf("%lld\n",m / __gcd(n,m) - 1);
    }
    return 0;
}
```

H 小水獭和科学抗疫 (O 版)

时间限制: 3000ms 内存限制: 65536kb

通过率: 1/2 (50.00%) 正确率: 1/3 (33.33%)

题目描述



小水獭幼儿园出现了疫情!

小水獭幼儿园可以被抽象为一个数轴, 其中共有 n 名学生, 第 i 名学生位于坐标 x_i , 任意两名学生的坐标均不相同。对于每名学生均有 $\frac{1}{2}$ 的概率被感染, 且每名学生是否被感染相互独立。

基于最新的科学研究, 如果第 i 名学生和第 j 名学生都被感染 ($i < j$), 那么需要额外投入 $f(|x_i - x_j|)$ 的核酸费用, 其中 $f(n) = n!$ 。设核酸费用之和的数学期望为 C , 容易证明 $4C$ 是一个整数, 请你求解 $4C$ 对 998244353 取模后的结果。

同学们众志成城, 根据科学精准防控, 一定能战胜疫情!

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正整数 n ($2 \leq n \leq 10^5$), 含义如题目所示。

第二行 n 个正整数 $x_1, x_2, \dots, x_n$ ($1 \leq x_i \leq 10^5$), 含义如题目所示。保证 $x_i \neq x_j$ 对 $1 \leq i < j \leq n$ 成立。

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998244353 取模后的结果。

🐼 \ +V/DI

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

const double pi = acos(-1);
const int p = 998244353;

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
    return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
}

int pos[maxn];

void FFT(Complex *a, int lim, int type){
    for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
    for(int mid = 1; mid < lim; mid <= 1){
        Complex wn(cos(pi / mid), type * sin(pi / mid));
        for(int i = mid < 1, j = 0; j < lim; j += i){
```

```

        Complex w(1, 0);
        for(int k = 0; k < mid; k++, w = w * wn){
            Complex x = a[j + k], y = w * a[j + k + mid];
            a[j + k] = x + y;
            a[j + k + mid] = x - y;
        }
    }
}
if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
}

int t, n;
Complex a[maxn], b[maxn];
const int m = 1e5;

int main(){
    scanf("%d", &t);
    while(t--){
        for(int i = 0; i < maxn; i++) a[i] = b[i] = {0, 0};
        scanf("%d", &n);
        while(n--){
            int x;
            scanf("%d", &x);
            a[x].x++, b[m - x].x++;
        }
        int lim = 1;
        while(lim <= m * 2) lim <<= 1;
        for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
        FFT(a, lim, 1), FFT(b, lim, 1);
        for(int i = 0; i < lim; i++) a[i] = a[i] * b[i];
        FFT(a, lim, -1);
        int ans = 0;
        for(int i = m - 1, sum = 1; ~i; i--, sum = 111 * sum * (m - i) % p){
            ans = (ans + 111 * sum * (int)(a[i].x + 0.5)) % p;
        }
        printf("%d\n", ans);
    }
}

```

I 小水獭和 -300IQ

时间限制: 1000ms 内存限制: 65536kb

通过率: 3/4 (75.00%) 正确率: 3/9 (33.33%)

题目描述



小水獭的 IQ 是 -300, 你能帮它提升 IQ 嘛?

给定字符串 s , 设它的长度为 $|s|$, s_i 表示 s 的第 i 个字符。

定义 $S_i = s_1 s_2 \cdots s_{i-1} s_{i+1} \cdots s_{|s|-1} s_{|s|}$ ($1 \leq i \leq |s|$), 相当于把 s 第 i 个字符删掉得到的字符串。

现在, 请你对 $S_1, S_2, \cdots, S_{|s|}$ 按字典序从小到大排序。特别地, 如果两个字符串 $S_i = S_j$ ($i < j$), 令 S_i 的排名靠前。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 一行一个由小写字母组成的字符串 s ($2 \leq |s| \leq 10^5$), 含义同题目描述。

输出格式

对于每组数据, 输出一行 $|s|$ 个正整数, 第 i ($1 \leq i \leq |s|$) 个正整数表示 S_i 的排名。

样例输入

```
#include <stdio.h>
#include <string.h>

#define maxn 100086

char s[maxn], a[maxn];
int rk[maxn];

int main()
{
    int t, n, tmp, i, j, l, r, x;
    scanf("%d", &t);
    while (t--)
    {
        scanf("%s", s + 1), n = strlen(s + 1);
        for (i = n; i; i--)
        {
            if (s[i] == s[i + 1])
                a[i] = a[i + 1];
            else
                a[i] = s[i + 1];
        }
        l = 1, r = n;
        for (i = 1; i <= n; i++)
        {
            if (s[i] > a[i])
                rk[i] = l++;
            else
                rk[i] = r--;
        }
        x = 0;
        for (i = 1; i <= n + 1; i++)
        {
            if (s[i] ^ s[i - 1])
            {
                if (x && rk[x] > rk[i - 1])
```

```

        {
            for (j = 1; j < i - x + 1 - j; j++)
            {
                tmp = rk[x - 1 + j];
                rk[x - 1 + j] = rk[x - 1 + i - x + 1 - j];
                rk[x - 1 + i - x + 1 - j] = tmp;
            }
        }
        x = i;
    }
}
for (i = 1; i <= n; i++)
    printf("%d ", rk[i]);
puts("");
}
return 0;
}

```

J

J 这是一道凸包？

时间限制：500ms 内存限制：65536kb

通过率：1/2 (50.00%) 正确率：1/3 (33.33%)

题目描述

Bellaiabella 刚刚学习了计算几何中的凸包知识，她想到了这样一个问题。

二维平面中有 n 个点，第 i 个点的坐标为 (x_i, y_i) ，权值为 w_i 。Bellalabella 可以选择一个点集 S ，设 m 和 M 分别为点集 S 中点的横坐标最小值和最大值，此方案的权值为 S 中所有点与 $(m, 0), (M, 0)$ 构成的凸包面积减去选取点的权值和。特别地，如果 $S = \emptyset$ 则权值为 0。

Bellalabella 想知道所有选取方案中最大权值为多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 5 \times 10^3$)，含义同题目描述。

接下来 n 行，每行三个非负整数 x_i, y_i, w_i ($0 \leq x_i, y_i, w_i \leq 10^4$)，含义同题目描述。

可能存在两个不同的点坐标相同。

输出格式

对于每组数据，输出一行一个浮点数，保留一位小数，表示最大权值。

输入样例

```

1
3
1 2 1
2 4 2
3 2 1

```

输出样例

```

2.0

```

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
int n,k;
const int N = 5010;
vector<tuple<ll,ll,ll>> tmp;
long double dp[N];
long double calc(int l,int r) {

```

```

        return 1.0L*(get<1>(tmp[l-1])+get<1>(tmp[r-1]))*(get<0>(tmp[r-1])-get<0>
(tmp[l-1]))/2-get<2>(tmp[r-1]);
    }
    int main() {

        int t;
        scanf("%d", &t);
        while(t--) {
            scanf("%d",&n);
            for(int i = 1; i <= n; i++) dp[i] = 0;
            tmp.clear();
            for(int i=0; i<n; i++) {
                ll x,w,c;
                scanf("%lld %lld %lld",&x,&c,&w);
                tmp.push_back(make_tuple(x,c,w));
            }
            sort(tmp.begin(),tmp.end());
            for(int i=1; i<=n; i++) {
                dp[i]=-get<2>(tmp[i-1]);
            }
            for(int i=1; i<=n; i++) {
                for(int j=1; j<i; j++) {
                    dp[i]=max(dp[i],dp[j]+ calc(j,i));
                }
            }
            long double tmpa=0;
            for(int i=1; i<=n; i++) {
                tmpa=max(tmpa,dp[i]);
            }
            printf("%.1Lf\n",tmpa);
        }
    }
}

```

C7

A

第一题

对于排序算法和时间复杂度分析，下列说法正确的是__。

- A. 根据排序元素所在位置的不同，排序分内排序和外排序。
- B. 可以使用 Amortized Analysis（平摊分析）对归并排序进行复杂度分析。
- C. 可以使用概率分析方法对快速排序进行复杂度分析。
- D. 堆排序是一种稳定的排序算法。
- E. 当上下限表达式相等时，可以使用 O 表示法来描述算法代价。

第二题

对于动态规划，下列说法正确的是__。

- A. 动态规划可以解决最优二叉搜索树的问题，其中最优二叉搜索树指的是最坏查找时间最小的二叉搜索树。
- B. 可以使用动态规划解决的问题只需要满足最优化原理即可，不需要考虑是否有后效性。
- C. Strassen 矩阵乘法问题可以使用动态规划来解决。
- D. 动态规划通常以自底向上的方式求解最优解。

第三题

对于 $\mathbf{NP}$ 问题，下列说法正确的是__。

- A. $\mathbf{NP}$ 问题都是不可能解决的问题。
- B. $\mathbf{P}$ 问题包含在 $\mathbf{NP}$ 问题中。
- C. $\mathbf{NP}$ 完全问题是 $\mathbf{P}$ 问题的子集。
- D. $\mathbf{NP}$ 问题包含在 $\mathbf{P}$ 问题中。

第四题

对 n ($5 \times 10^3 < n < 10^4$) 个点的点集，采用 Graham's scan 算法求凸包，下列说法正确的是__。

- A. Graham's scan 算法的计算复杂度为 $O(kn)$ ，其中 k 为凸包顶点个数。
- B. 可以用 The accounting method 方法有效分析 Graham's scan 的计算复杂度。
- C. 不可以用 Aggregate analysis 方法有效分析 Graham's scan 的计算复杂度。
- D. 凸包顶点个数 k 约为 $\frac{n}{5}$ 时，Graham's scan 算法比 Jarvis's march 更快。

B

B XIAO7 和搬砖

时间限制：1000ms 内存限制：65536kb

通过率：37/106 (34.91%) 正确率：37/525 (7.05%)

题目描述

XIAO7 准备放弃猫娘乐园专心搬砖。

有 N 种砖头，第 i 种砖头有 m_i 块，每一块的价值为 d_i ，每一个单位时间（从 1 开始编号）XIAO7 都会搬一块砖，直到无砖可搬为止。

同时，有一个得分系数 F ，初始时为 1。搬一块砖的得分为当时的得分系数乘以对应砖的价值。

此外，有 T 个时间分割点，每经过一个时间分割点，即从时间分割点的下一个单位时间开始， F 就会增加 1。

请你告诉 XIAO7 她能够获得的最大得分是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 N ($1 \leq N \leq 10^5$)，含义同题目描述。

接下来 N 行，每行两个正整数 m_i, d_i ($1 \leq m_i \leq 10^5, 1 \leq d_i \leq 10^3$)，含义同题目描述。

接下来一行一个正整数 T ($1 \leq T \leq 10^5$)，含义同题目描述。

接下来一行 T 个互不相同的正整数 p_i ($1 \leq p_i \leq 10^9$)，表示时间分割点，即从第 $p_i + 1$ 个单位时间开始 F 会增加 1。

输出格式

对于每组数据，输出一行一个正整数，表示最大得分。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
#include<assert.h>
using namespace std;
typedef long long ll;
```

```

typedef pair<ll, ll> pi;
const int N = 100005;
const ll inf = 1ll << 60;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
pi a[N];
ll t[N];
int main() {

    int n;
    int tt;
    scanf("%d", &tt);
    while(tt--) {
        scanf("%d", &n);
        assert(n <= 1e5);
        for(int i = 1; i <= n; i++) {
            a[i].second = getint();
            a[i].first = getint();
        }
        sort(a + 1, a + n + 1);

        int T = getint();
        for(int i = 1; i <= T; i++) {
            t[i] = getint();
        }
        sort(t + 1, t + T + 1), t[T + 1] = inf;

        ll ans = 0, sum = 0;
        for(int i = 1, j = 1; i <= n; ) {
            if(sum + a[i].second <= t[j]) {
                ans += 1ll * j * a[i].first * a[i].second;
                sum += a[i].second, i++;
            } else {
                ll m = t[j] - sum;
                a[i].second -= m;
                ans += 1ll * j * a[i].first * m;
                sum += m, j++;
            }
        }

        cout << ans << endl;
    }

    return 0;
}

```

C Bellalabella 与中位数

时间限制: 2000ms 内存限制: 65536kb
通过率: 135/172 (78.49%) 正确率: 135/411 (32.85%)

题目描述

Bellalabella 学到了中位数的知识, Bellalabella 想知道对于一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 所有前缀的中位数是多少。

由于 Bellalabella 学艺不精, 不知道长度为偶数的序列的中位数的定义, 因此她只需要你求出长度为奇数的所有前缀的中位数。

一个长度为 $2n - 1$ 的序列的中位数是将这个序列从小到大排序后第 n 个位置的数, 例如 $[1, 3, 2]$ 中位数为 2 , $[2, 3, 2]$ 中位数为 2 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正奇数 n ($1 \leq n \leq 10^5$), 含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^5$), 含义同题目描述。

对于得分占比 50% 的测试点, 保证 $0 \leq a_i \leq 20$ 。

输出格式

对于每组数据, 输出一行 $\frac{n+1}{2}$ 个整数, 第 i 个数表示长度为 $2i - 1$ 的前缀的中位数。

输入样例

```
2
5
3 4 5 1 2
13
13 25 19 11 29 19 17 17 5 9 11 11 3
```

输出样例

```
3 4 3
13 19 19 19 17 17 13
```

```
#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 1000005
#define ls (tot << 1)
#define rs (tot << 1 | 1)
#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acos(-1.0L);
const int mod = 998244353;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n, a[maxn];
priority_queue<int> q;
priority_queue<int, vector<int>, greater<int>> p;

void solve(){
    while(!q.empty())q.pop();
```

```

while(!p.empty())p.pop();
n = read();
for(int i = 1; i <= n; i++) {
    a[i] = read();
    q.push(a[i]);
    if(i & 1){
        while(q.size() > p.size()) p.push(q.top()), q.pop();
        while(q.size() < p.size()) q.push(p.top()), p.pop();
        printf("%d ", q.top());
    }
}
puts("");
}

int main(){

    int t = read();
    while(t--){
        solve();
    }

    return 0;
}

```

D

D Bellalabella 与「三分之一」

时间限制: 1000ms 内存限制: 65536kb

通过率: 1/16 (6.25%) 正确率: 1/55 (1.82%)

题目描述

Bellalabella 在自己进行游戏，游戏规则如下：

- 游戏总共进行 n 轮，每一轮都由 Bellalabella 先手。
- 初始 Bellalabella 有一个数字 0。
- 每一轮 Bellalabella 可以从 A, B, C 三个数中选择一个数，并把这个数加到自己的数字上。
- 设最终 Bellalabella 手中的数字设为 x 。
- 如果对于其他所有选择方式的出的结果 y 都有 $\left| x - \frac{n(A+B+C)}{3} \right| \leq \left| y - \frac{n(A+B+C)}{3} \right|$ ，那么 Bellalabella 就会获胜。

简单来说，即 Bellalabella 想要让自己最后的数字最接近 $\frac{n(A+B+C)}{3}$ ，Bellalabella 想知道自己最后的数字是多少才能赢，容易证明这个解是唯一的。

注意在 n 轮游戏中， A, B, C 三个数的值是固定的。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行四个正整数 n, A, B, C ($1 \leq n \leq 10^5$, $1 \leq A, B, C \leq 10^9$)，含义同题目描述。

对于得分占比 50% 的测试点保证 $1 \leq n \leq 100$ 且 $1 \leq A, B, C \leq 100$ 。

输出格式

对于每组数据，输出一行一个正整数，表示 Bellalabella 能够获得的最接近 $\frac{n(A+B+C)}{3}$ 的数为多少。

```

#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 100005
#define ls (tot << 1)
#define rs (tot << 1 | 1)

```

```

#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acos(-1.0L);
const int mod = 998244353;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

ll n, A, B, C;

void solve(){
    ll ans = 0;

    n = read(), A = read(), B = read(), C = read();
    if(A > B) swap(A, B);if(A > C) swap(A, C);if(B > C) swap(B, C);
    ll sum = n * (A + B + C);
    for(int i = 0; i <= n; i++){
        ll mn = 1ll * A * i + 1ll * B * (n - i);
        if(mn * 3 >= sum || B == C){
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
        }else{
            ll delta = (C - B) * 3, nd = sum - mn * 3;
            ll down = nd / delta, up = down + 1;
            if(down >= n - i) down = up = n - i;
            mn = 1ll * A * i + 1ll * B * (n - i - down) + 1ll * C * down;
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
            mn = 1ll * A * i + 1ll * B * (n - i - up) + 1ll * C * up;
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
        }
    }
    printf("%lld\n", ans);
}

int main(){
    int t = read();
    while(t--){
        solve();
    }

    return 0;
}

```

E

本题采用「炉石传说」背景，无论您是否玩过这个游戏，请完全以本题规则为准。

小水獭和奥布莱克斯每人各有 7 个随从，编号为 1 到 7，每个随从有两个基本属性：攻击力和生命值；以及可能拥有的三种特殊能力：**嘲讽**、**圣盾**、**亡语**，这三种能力会在下方解释清楚。

每个随从从受到伤害时生命值会减少相应的数值，如果一个随从的生命值小于等于 0，则该随从死亡。

小水獭和奥布莱克斯会轮流执行下列操作，直到其中一方（或两方）的全部随从死亡，小水獭先手：

- 选择己方攻击力最高的一个未死亡随从，如果有多个随从攻击力相同，选择编号小的那一个；记这个随从的编号为 x 。
- 如果对方有具有嘲讽能力的未死亡随从，则选择对方生命值最高的一个具有嘲讽能力的未死亡随从，否则选择对方生命值最高的一个未死亡随从，如果有多个随从生命值相同，选择编号小的那一个；记这个随从的编号为 y 。
- 对 x 号随从造成等同于 y 号随从从攻击力的伤害，对 y 号随从造成等同于 x 号随从从攻击力的伤害。

对于上述三种特殊能力的解释：

- 具有嘲讽能力的随从，会被对方优先攻击，已在上述游戏规则阐述过。
- 具有圣盾能力的随从，第一次受到伤害时生命值不会减少，并失去圣盾能力。
- 具有亡语能力的随从，会在死亡后额外产生一个和它编号相同的随从，该随从的攻击力和生命值会在输入中给出，且不具有任何特殊能力。

莫卡正在和教练同学学习术学，于是她想问问你，在上述游戏过程中，存不存在一个时刻（包括开始时刻和结束时刻），使得场上未死亡随从数量大于 2，且通过名为**教科书般的毁灭**的下列操作可以使得场上不存在任何未死亡随从。

- 对所有随从造成 1 点伤害，如果这次操作使得至少一个随从死亡，重复这个操作。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，共 14 行，分别描述小水獭的 1 到 7 号随从以及奥布莱克斯的 1 到 7 号随从。

每行 6 个整数 a, b, c, d, e, f ($1 \leq a, b \leq 20, 0 \leq c, d \leq 1, 0 \leq e, f \leq 5$)，具体说明如下：

- a 表示该随从的攻击力。
- b 表示该随从的生命值。
- 如果 $c = 1$ 说明该随从具有嘲讽能力，否则该随从没有嘲讽能力。
- 如果 $d = 1$ 说明该随从具有圣盾能力，否则该随从没有圣盾能力。
- 如果 $e = f = 0$ 说明该随从不具有亡语能力，否则该随从具有亡语能力，且死亡后产生随从的攻击力和生命值分别为 e 和 f ，保证两者同时为 0 或同时非 0。

输出格式

对于每组数据，输出一行一个整数，如果存在一个时刻，使得场上未死亡随从数量大于 2，且通过教科书般的毁灭可以使得场上不存在任何未死亡随从，输出 1；否则，输出 0。

```
#include <stdio.h>
#define max(a, b) (a > b ? a : b)

struct Minion
{
    int a, b, c, d, e, f;
} A[2][10];

int check()
{
    int c[30] = {0}, mx = 0, i, j, cnt = 0;
    for (i = 0; i <= 1; i++)
    {
        for (j = 1; j <= 7; j++)
        {
            if (A[i][j].b <= 0) continue;
            cnt++;
            c[A[i][j].b + A[i][j].d] = 1;
            c[A[i][j].b + A[i][j].d + A[i][j].f] = 1;
            mx = max(mx, A[i][j].b + A[i][j].d + A[i][j].f);
        }
    }
    if (cnt <= 2) return 0;
    for (i = 1; i <= mx; i++)
        if (!c[i]) return 0;
    return 1;
}

int main()
{
    int t, i, j, suc, x, y;
```

```

scanf("%d", &t);
while (t--)
{
    for (i = 0; i <= 1; i++)
        for (j = 1; j <= 7; j++) scanf("%d%d%d%d%d", &A[i][j].a, &A[i][j].b, &A[i][j].c, &A[i][j].d, &A[i][j].e, &A[i][j].f);
    suc = check();
    for (i = 0; !suc; i ^= 1)
    {
        x = y = 0;
        for (j = 1; j <= 7; j++)
        {
            if (A[i][j].b > 0 && (!x || A[i][j].a > A[i][x].a))
            {
                x = j;
            }
        }
        for (j = 1; j <= 7; j++)
        {
            if (A[i ^ 1][j].b > 0 && (!y || A[i ^ 1][j].c > A[i ^ 1][y].c || A[i ^ 1][j].c == A[i ^ 1][y].c && A[i ^ 1][j].b > A[i ^ 1][y].b))
            {
                y = j;
            }
        }
        if (!x || !y) break;
        if (A[i][x].d)
        {
            A[i][x].d = 0;
        }
        else
        {
            A[i][x].b -= A[i ^ 1][y].a;
        }
        if (A[i ^ 1][y].d)
        {
            A[i ^ 1][y].d = 0;
        }
        else
        {
            A[i ^ 1][y].b -= A[i][x].a;
        }
        if (A[i][x].b <= 0 && A[i][x].e)
        {
            A[i][x].a = A[i][x].e, A[i][x].b = A[i][x].f;
            A[i][x].c = A[i][x].d = A[i][x].e = A[i][x].f = 0;
        }
        if (A[i ^ 1][y].b <= 0 && A[i ^ 1][y].e)
        {
            A[i ^ 1][y].a = A[i ^ 1][y].e, A[i ^ 1][y].b = A[i ^ 1][y].f;
            A[i ^ 1][y].c = A[i ^ 1][y].d = A[i ^ 1][y].e = A[i ^ 1][y].f = 0;
        }
        suc |= check();
    }
    puts(suc ? "1" : "0");
}

```

```
    return 0;
}
```

F

题目描述



小水獭正在前往加帕里幼儿园学习构造题的相关技巧。

对于任意有限数集 A, B ，定义二者之和 $A + B$ 为：

$$A + B = \{x + y \mid x \in A, y \in B\}$$

例如，当 $A = \{1, 2\}, B = \{3, 4\}$ ，有 $A + B = \{4, 5, 6\}$ ，注意此处并非可重集，因此即使 $1 + 4 = 2 + 3 = 5$ ， $A + B$ 也仅包含一个 5。

记有限数集 A 中的元素个数为 $|A|$ 。

现给定整数集合 S ，记 $T = S + S$ ，请你求解 $|S + T|$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 $|S|$ ($1 \leq |S| \leq 8 \times 10^4 + 1$)，含义如题目所示。

第二行 $|S|$ 个互不相同的整数 $a_1, a_2, \dots, a_{|S|}$ ($0 \leq |a_i| \leq 4 \times 10^4$)，表示 S 中的元素。

输出格式

对于每组数据，输出一行一个正整数表示 $|S + T|$ 。

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

#define double long double

const double pi = acos(-1.0L);
const int p = 998244353;

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
    return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
}
```

```

}

int pos[maxn];

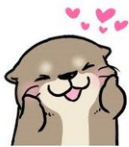
void FFT(Complex *a, int lim, int type){
    for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
    for(int mid = 1; mid < lim; mid <= 1){
        Complex wn(cos(pi / mid), type * sin(pi / mid));
        for(int i = mid < 1, j = 0; j < lim; j += i){
            Complex w(1, 0);
            for(int k = 0; k < mid; k++, w = w * wn){
                Complex x = a[j + k], y = w * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
    if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
}

int t, n;
Complex a[maxn];
const int m = 4e4;

int main(){
    scanf("%d", &t);
    while(t--){
        for(int i = 0; i < maxn; i++) a[i] = {0, 0};
        scanf("%d", &n);
        while(n--){
            int x;
            scanf("%d", &x);
            a[x + m].x++;
        }
        int lim = 1;
        while(lim <= m * 6) lim <= 1;
        for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
        FFT(a, lim, 1);
        for(int i = 0; i < lim; i++) a[i] = a[i] * a[i] * a[i];
        FFT(a, lim, -1);
        int ans = 0;
        for(int i = 0; i < lim; i++) ans += ((long long)(a[i].x + 0.5)) > 0;
        printf("%d\n", ans);
    }
}

```

题目描述



小水獭正在两个 $n \times m$ 的矩阵中随机游走，记这两个矩阵分别为 A 和 B 。

具体来说，小水獭会首先选中两个矩阵中的两个位置 (x_1, y_1) 和 (x_2, y_2) ，之后进行任意次如下四种操作之一：

- 将 (x_1, y_1) 变为 $(x_1 - 1, y_1)$ ，将 (x_2, y_2) 变为 $(x_2 - 1, y_2)$ 。
- 将 (x_1, y_1) 变为 $(x_1 + 1, y_1)$ ，将 (x_2, y_2) 变为 $(x_2 + 1, y_2)$ 。
- 将 (x_1, y_1) 变为 $(x_1, y_1 - 1)$ ，将 (x_2, y_2) 变为 $(x_2, y_2 - 1)$ 。
- 将 (x_1, y_1) 变为 $(x_1, y_1 + 1)$ ，将 (x_2, y_2) 变为 $(x_2, y_2 + 1)$ 。

任意时刻，小水獭需要保证 $1 \leq x_1, x_2 \leq n$ 且 $1 \leq y_1, y_2 \leq m$ ，同时 $A_{x_1, y_1} = B_{x_2, y_2}$ 。

记 S 为小水獭在 A 所经过位置对应元素组成的集合，那么最理想情况下 S 中元素数量 $|S|$ 的最大值是多少呢？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq nm \leq 10^5$)，含义如题目所示。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个正整数 $A_{i,j}$ ($1 \leq A_{i,j} \leq nm$)，表示第一个矩阵 A 中的元素。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个正整数 $B_{i,j}$ ($1 \leq B_{i,j} \leq nm$)，表示第二个矩阵 B 中的元素。

保证 $A_{i,j} = A_{i',j'} \Leftrightarrow (i = i' \wedge j = j')$ 且 $B_{i,j} = B_{i',j'} \Leftrightarrow (i = i' \wedge j = j')$ 。

输出格式

对于每组数据，输出一行一个正整数最理想情况下 $|S|$ 的最大值。

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
int a[maxn];
pair<int, int> b[maxn], c[maxn];
bool vis[maxn];

int id(int x, int y){
    return (x - 1) * m + y;
}

const int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++) scanf("%d", &a[id(i, j)]);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++){
            int x;
            scanf("%d", &x);
            b[x] = {i, j};
        }
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++){
            c[id(i, j)] = {i - b[a[id(i, j)]]>.first, j - b[a[id(i, j)]]>.second};
        }
        for(int i = 1; i <= n * m; i++) vis[i] = false;
        int ans = 0;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++) if(!vis[id(i, j)]){
            int sum = 0;
```

```

queue<pair<int, int> > q;
vis[id(i, j)] = true, q.push({i, j});
while(!q.empty()){
    int x = q.front().first, y = q.front().second;q.pop();
    sum++;
    for(int k = 0;k < 4;k++){
        int xx = x + dx[k], yy = y + dy[k];
        if(xx >= 1 && xx <= n && yy >= 1 && yy <= m){
            if(c[id(x, y)] == c[id(xx, yy)] && !vis[id(xx, yy)])
vis[id(xx, yy)] = true, q.push({xx, yy});
        }
    }
}
ans = max(ans, sum);
}
printf("%d\n", ans);
}
}

```

H

小水獭定义了一种奇怪的最短路。

形式化来说，给定一个 n 个点 m 条边的带权无向连通图，对于一条至少经过一条边的路径，设其所有边的权值之和为 A ，边权中的最大值和最小值为 B 和 C ，则定义这条路径的权值为 $A - B + C$ 。

小水獭想知道，从节点 1 到其它每个节点在本题路径权值定义下的最短路长度，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($2 \leq n \leq 10^5$, $n - 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $0 \leq w_i \leq 10^4$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证图是连通的，且没有自环和重边。

输出格式

对于每组数据，输出一行 $n - 1$ 个非负整数，表示从节点 1 到节点 $2, 3, \dots, n$ 在本题路径权值定义下的最短路长度。

输入样例

```

2
7 6
1 2 1
2 3 2
1 4 2
4 5 1
1 6 3
6 7 3
4 6
1 2 1
1 3 2
1 4 3
2 3 4
2 4 5
3 4 6

```

输出样例

```

1 2 2 2 3 6
1 2 2

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<pair<int, int> > v[maxn];
int d[maxn][4];
bool vis[maxn][4];

```

```

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) v[i].clear();
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            v[x].push_back({y, z}), v[y].push_back({x, z});
        }
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>,
greater<tuple<int, int, int>>> q;
        for(int i = 1; i <= n; i++) for(int j = 0; j < 4; j++) d[i][j] = 0x3f3f3f3f,
vis[i][j] = false;
        d[1][0] = 0, q.push(make_tuple(0, 1, 0));
        while(!q.empty()){
            int x = get<1>(q.top()), y = get<2>(q.top()); q.pop();
            if(vis[x][y]) continue;
            vis[x][y] = true;
            for(auto it : v[x]){
                int to = it.first, co = it.second;
                if(d[to][y] > d[x][y] + co){
                    d[to][y] = d[x][y] + co;
                    q.push(make_tuple(d[to][y], to, y));
                }
                if(!(y & 1) && d[to][y | 1] > d[x][y]){
                    d[to][y | 1] = d[x][y];
                    q.push(make_tuple(d[to][y | 1], to, y | 1));
                }
                if(!(y & 2) && d[to][y | 2] > d[x][y] + co * 2){
                    d[to][y | 2] = d[x][y] + co * 2;
                    q.push(make_tuple(d[to][y | 2], to, y | 2));
                }
                if(!(y & 3) && d[to][y | 3] > d[x][y] + co){
                    d[to][y | 3] = d[x][y] + co;
                    q.push(make_tuple(d[to][y | 3], to, y | 3));
                }
            }
        }
        for(int i = 2; i <= n; i++) printf("%d ", d[i][3]), assert(d[i][3] <
0x3f3f3f3f); puts("");
    }
}

```

题目描述



小水獭正在预习计算几何。

二维平面上有 n 个不同的点，小水獭请你帮它计算这 n 个点所组成的集合有多少个非空子集满足其中点构成的凸包（有可能退化为一条线段或一个点）严格不包含原点，即原点不能在凸包内或凸包上。

由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^5$)，含义同题目描述。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$, $|x_i| + |y_i| \neq 0$)，表示第 i 个点的坐标。

保证任意两点坐标不同。

对于得分占比 50% 的测试点，保证 $1 \leq n \leq 15$ 。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 998 244 353 取模后的结果。

```
#include <bits/stdc++.h>
#define maxn 200086

using namespace std;

typedef long long ll;

struct node{
    ll x, y;
    int f, id;
    void getf(){
        if(x >= 0 && y > 0) f = 1;
        else if(x < 0 && y >= 0) f = 2;
        else if(x <= 0 && y < 0) f = 3;
        else f = 4;
    }
    bool operator < (const node & a) const {return f < a.f || (f == a.f && x * a.y > y * a.x) || (f == a.f && x * a.y == y * a.x && id < a.id);}
}a[maxn];

const int p = 998244353;

int t, n;
int x[maxn], y[maxn];
int pw[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        pw[0] = 1;for(int i = 1;i <= n;i++) pw[i] = pw[i - 1] * 2 % p;
        for(int i = 1;i <= n;i++) scanf("%lld%lld", &a[i].x, &a[i].y),
a[i].getf(), a[i].id = i;
        sort(a + 1, a + n + 1);
        for(int i = n + 1;i <= n * 2;i++) a[i] = a[i - n];
        int x = 1, ans = 0;
        for(int i = 1;i <= n;i++){
            while(x + 1 < i + n && (a[x + 1].x * a[i].y - a[i].x * a[x + 1].y == 0 && a[x + 1].f == a[i].f
                || a[x + 1].x * a[i].y - a[i].x * a[x + 1].y < 0)){
```

```

        x++;
    }
    ans = (ans + pw[x - i]) % p;
}
printf("%d\n", ans);
}
}

```

J

小水獭正在预习字符串。

给定 n 个字符串，小水獭想知道所有满足这 n 个字符串均是其子串的字符串中，最短字符串的长度是多少？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 18$)，含义同题目描述。

接下来 n 行，每行一个由小写字母组成的字符串 s_i ($1 \leq |s_i| \leq 10^4$)，表示第 i 个字符串。

对于得分占比 50% 的测试点，保证 $1 \leq n \leq 6$ 。

输出格式

对于每组数据，输出一行一个正整数，表示满足条件的最短字符串长度。

输入样例

```

2
4
aba
bab
bcc
cbaab
5
aaaaa
aaaa
aaa
aa
a

```

输出样例

```

9
5

```

```

#include <bits/stdc++.h>
#define maxn 10086

using namespace std;

char s[18][maxn];
int len[18];
int nxt[maxn];

int cal(int x, int y){
    for(int i = 1; i < len[x]; i++){
        int j = nxt[i - 1];
        while(j > 0 && s[x][i] != s[x][j]) j = nxt[j - 1];
        if(s[x][i] == s[x][j]) j++;
        nxt[i] = j;
    }
    int j = 0;
    for(int i = 0; i < len[y]; i++){
        while(j > 0 && s[x][j] != s[y][i]) j = nxt[j - 1];
        if(s[x][j] == s[y][i]) j++;
        if(j == len[x]) return -1;
    }
    return j;
}

```

```

int t, n;
int a[18][18];
int f[1 << 18][18];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 0; i < n; i++) scanf("%s", s[i]), len[i] = strlen(s[i]);
        vector<int> v;
        for(int i = 0; i < n; i++){
            bool tag = false;
            for(int j = 0; j < n; j++) if(i ^ j){
                a[j][i] = cal(i, j);
                if(a[j][i] == -1){
                    if(len[i] < len[j] || i < j){
                        tag = true;
                        break;
                    }
                }
            }
            if(tag) continue;
            v.push_back(i);
        }
        int m = v.size();
        for(int i = 0; i < (1 << m); i++) for(int j = 0; j < m; j++) f[i][j] = 1e9;
        for(int i = 0; i < m; i++) f[1 << i][i] = len[v[i]];
        for(int i = 0; i < (1 << m); i++) for(int j = 0; j < m; j++){
            for(int k = 0; k < m; k++) if(!(i & (1 << k))){
                f[i | (1 << k)][k] = min(f[i | (1 << k)][k], f[i][j] + len[v[k]]
- a[v[j]][v[k]]);
            }
        }
        int ans = 1e9;
        for(int i = 0; i < m; i++) ans = min(ans, f[(1 << m) - 1][i]);
        printf("%d\n", ans);
    }
}

```

E1

A 你还记得我吗~

本题必须使用 C 语言提交。

题目描述

给定整数序列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_m$ ，严格递增的非负整数序列 $A_1, A_2, \dots, A_n$ 和 $B_1, B_2, \dots, B_m$ ，求解如下多项式：

$$\left(\sum_{i=1}^n a_i x^{A_i}\right) + \left(\sum_{i=1}^m b_i x^{B_i}\right)$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($0 < |a_i| \leq 10^9$)，含义同题目描述。

第三行 n 个非负整数 $A_1, A_2, \dots, A_n$ ($0 \leq A_i \leq 10^9$)，含义同题目描述。

第四行一个正整数 m ($1 \leq m \leq 10^5$)，含义同题目描述。

第五行 n 个整数 $b_1, b_2, \dots, b_m$ ($0 < |b_i| \leq 10^9$)，含义同题目描述。

第六行 n 个非负整数 $B_1, B_2, \dots, B_m$ ($0 \leq B_i \leq 10^9$)，含义同题目描述。

保证 $A_i > A_{i-1}$ 对 $1 < i \leq n$ 成立， $B_i > B_{i-1}$ 对 $1 < i \leq m$ 成立， $A_i = B_j \Rightarrow a_i + b_j \neq 0$ 对 $1 \leq i \leq n, 1 \leq j \leq m$ 成立。

输出格式

对于每组数据，输出三行：

- 第一行一个正整数 k ，表示所得多项式在合并同类项后有 k 个系数非 0 项，并设所得多项式为：

$$\sum_{i=1}^k c_i x^{C_i}$$

其中 $C_i > C_{i-1}$ 对 $1 < i \leq k$ 成立。

- 第二行 k 个整数 $c_1, c_2, \dots, c_k$ ，含义同上式。
- 第三行 k 个非负整数 $C_1, C_2, \dots, C_k$ ，含义同上式。

输入样例

```
1
3
1 2 3
1 2 3
3
2 3 4
2 3 4
```

输出样例

```
4
1 4 6 4
1 2 3 4
```

Hint

宋老师：「这道题，大一每年都做，但通过率都不高。」

```
#include <stdio.h>
#define maxn 200086

int t, n, m, k;
int a[maxn], A[maxn], b[maxn], B[maxn], c[maxn], C[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        for(int i = 1; i <= n; i++) scanf("%d", &A[i]);
        scanf("%d", &m);
        for(int i = 1; i <= m; i++) scanf("%d", &b[i]);
```

```

for(int i = 1; i <= m; i++) scanf("%d", &B[i]);
int posa = 1, posb = 1;
k = 0;
while(posa <= n || posb <= m) {
    if(posa > n) c[++k] = b[posb], c[k] = B[posb], posb++;
    else if(posb > m) c[++k] = a[posa], c[k] = A[posa], posa++;
    else if(A[posa] == B[posb]) c[++k] = a[posa] + b[posb], c[k] =
A[posa], posa++, posb++;
    else if(A[posa] < B[posb]) c[++k] = a[posa], c[k] = A[posa], posa++;
    else c[++k] = b[posb], c[k] = B[posb], posb++;
}
printf("%d\n", k);
for(int i = 1; i <= k; i++) printf("%d ", c[i]);
puts("");
for(int i = 1; i <= k; i++) printf("%d ", c[i]);
puts("");
}
}

```

B 小水獭和签到题

题目描述



小水獭在数学课上学习到了勾股定理，即直角三角形的直角边平方和等于斜边的平方。

善于举一反三的小水獭想知道对于给定正整数的 z, k ，存在多少对正整数 (x, y) 使得 $x^k + y^k = z^k$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，一行两个正整数 z, k ($1 \leq z, k \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示满足 $x^k + y^k = z^k$ 的正整数对 (x, y) 的个数。

输入样例

```

2
2 1
5 2

```

输出样例

```

1
2

```

```

#include<stdio.h>
#include<math.h>
int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int k, z, index = 0;
        scanf("%d%d",&z,&k);
        if (k > 2)
            printf("0\n");
    }
}

```

```

else if( k == 1)
    printf("%d\n", z-1);
else {
    for(long long i = 1; i < z; i++) {
        double j = sqrt(1ll * z * z - i * i);
        if(j == (int)j) {
            index++;
        }
    }
    printf("%d\n", index);
}
}
return 0;
}

```

C 小水獭和多路归并

题目描述



小水獭正在学习归并排序，它觉得将两个序列进行归并太简单了，因此想要尝试将多个序列进行归并，即多路归并。

具体来说，给定 k 个长度为 n 的序列 $[a_{1,1}, a_{1,2}, \dots, a_{1,n}]$, $[a_{2,1}, a_{2,2}, \dots, a_{2,n}]$, $\dots$, $[a_{k,1}, a_{k,2}, \dots, a_{k,n}]$ ，满足每个序列均是严格递增的，且这 kn 个数互不相同。小水獭会重复进行如下操作直到所有序列均为空：

- 对于所有非空序列的首个元素，找到最小的那个元素，并将其从所在序列中删除。

小水獭会进行 kn 次操作，它想请你帮它写一个程序模拟这个过程，输出每一次操作删掉了哪个数。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 k, n ($2 \leq k \leq 10^5$, $1 \leq n \leq 10^5$, $2 \leq kn \leq 10^5$)，表示序列个数和序列长度。

接下来 k 行，第 i 行 n 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,n}$ ($-10^9 \leq a_{i,j} \leq 10^9$)，表示第 i 个序列的元素。

保证 $a_{i,j} > a_{i,j-1}$ 对 $1 \leq i \leq k, 1 < j \leq n$ 成立，且 $a_{i,j} = a_{x,y} \Leftrightarrow (i = x \wedge j = y)$ 。

输出格式

对于每组数据，输出一行 kn 个整数，第 i ($1 \leq i \leq kn$) 个整数表示第 i 次操作删除的数。

输入样例

```

1
2 2
-1 3
-2 4

```

输出样例

```

-2 -1 3 4

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, k, n;

int main() {
    scanf("%d", &t);

```

```

while(t--) {
    scanf("%d%d", &k, &n);
    vector<int> v(k * n);
    for(int i = 0; i < k * n; i++) scanf("%d", &v[i]);
    sort(v.begin(), v.end());
    for(auto x : v) printf("%d ", x);
    puts("");
}
}

```

D 小水獭和随机数

题目描述



小水獭正在学习排序，不过这回它所排序的数字来自一个随机数生成器，它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 20000086

using namespace std;

unsigned seed;

int Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed & ((1 << 24) - 1);
}

int t, n, a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%u", &n, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand();
        sort(a + 1, a + 1 + n);
        unsigned ans = 0;
        for(int i = 1; i <= n; i++) ans += a[i] ^ i;
        printf("%u\n", ans);
    }
}

```

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行两个正整数 $n, seed$ ($1 \leq n \leq 2 \times 10^7$, $1 \leq seed < 2^{32}$)，表示序列的长度和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

输入样例

```

2
3 5
5 7

```

输出样例

```

23518363
40839186

```

样例解释

样例中两组数据对应的初始序列分别为：[9085253, 83271, 14349837] 和 [9363911, 93636, 5545503, 14569699, 10466434]。

```

#include <bits/stdc++.h>
#define maxn 20000086

```

```

using namespace std;

unsigned seed;

int Rand() {
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed & ((1 << 24) - 1);
}

int t, n, a[maxn];
int num[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        memset(num, 0, sizeof(num));
        scanf("%d%u", &n, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand(), num[a[i]]++;
        unsigned ans = 0;
        int cnt = 0;
        for(int i = 0; i <= (1 << 24) - 1; i++) {
            while(num[i]--) cnt++, ans += i ^ cnt;
        }
        printf("%u\n", ans);
    }
}

```

E 小水獭和组合数

题目描述



小水獭正在学习「离散数学 3」，它接触到了组合数的定义，对于非负整数 n 和 m 有：

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}$$

其中 $n! = 1 \times 2 \times \dots \times n$ ，特别地， $0! = 1$ 。

由组合意义可得，当 $n \geq m$ 时组合数是一个正整数。

小水獭希望你帮他编写一个程序用以计算组合数，特别地，如果组合数的值大于 10^9 ，你只需要输出 -1 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个非负整数 n, m ($0 \leq m \leq n \leq 10^9$)，表示需要计算的组合数的两个参数。

输出格式

对于每组数据，输出一行一个整数：

- 如果 $\binom{n}{m} \leq 10^9$ ，输出 $\binom{n}{m}$ 。
- 否则，输出 -1 。

输入样例

```
2
4 2
100 50
```

输出样例

```
6
-1
```

```
#include <bits/stdc++.h>

using namespace std;

int t, n, m;

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d", &n, &m);
        m = min(m, n - m);
        long long x = 1;
        for(int i = 1; i <= m && x <= 1e9; i++) x *= n - i + 1, x /= i;
        printf("%d\n", x <= 1e9 ? x : -1);
    }
}
```

F 莫卡的二叉树魔法

题目描述

小水獭正在学习二叉树，它希望世界上所有的树都是二叉树。

为了实现这个目的，小水獭请求莫卡教自己二叉树魔法，使自己可以把世界上所有的树变成二叉树。

为了检验小水獭的魔法天赋，莫卡给了小水獭一棵有 n 个节点的二叉树，且根节点的编号为 1。这棵二叉树任意一个节点要么是白色，要么是黑色。之后莫卡会对这棵二叉树释放 q 次魔法，每次莫卡会选择一个节点，将以这个节点为根的子树内所有节点的颜色反转，即黑色变成白色，白色变成黑色。在莫卡的 q 次魔法全部释放完成之后，小水獭需要回答每个节点的颜色。

但是这个问题对于小水獭来说过于困难了，请聪明的你帮他回答这个问题吧。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示二叉树的节点数量。

第二行 $n - 1$ 个正整数，第 i ($1 \leq i \leq n - 1$) 个数表示编号为 $i + 1$ 的节点的父亲节点编号，数据保证是一棵二叉树。

第三行一个长度为 n 的 01 串，从左到右第 i ($1 \leq i \leq n$) 位如果为 0，表示编号为 i 的节点颜色为白色，否则为黑色。

第四行一个正整数 q ($1 \leq q \leq 10^5$)，表示魔法的释放次数。

接下来 q 行每行一个正整数 a_i ($1 \leq a_i \leq n$)，表示第 i 次魔法选择的节点编号。

输入格式

对于每组数据，输出一行一个长度为 n 的 01 串，表示 q 次魔法全部释放完成之后每个节点的颜色。从左到右第 i ($1 \leq i \leq n$) 位如果为 0，表示编号为 i 的节点颜色为白色，否则为黑色。

输入样例

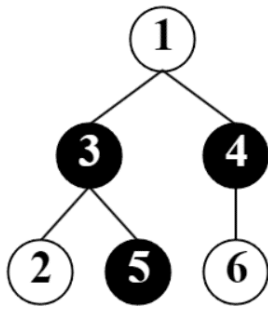
```
1
6
3 1 1 3 4
001110
3
1
3
2
```

输出样例

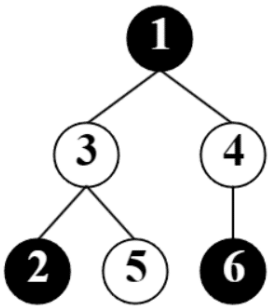
```
111011
```

样例解释

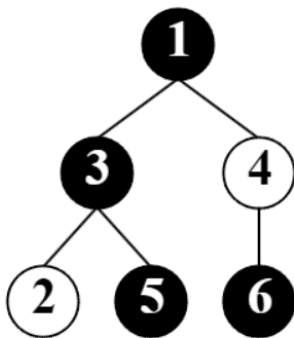
二叉树最初如下图所示：



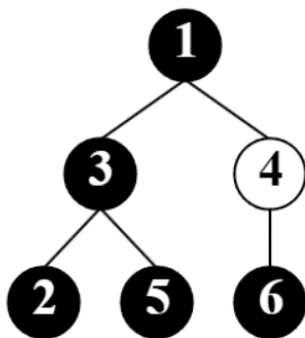
释放第 1 次魔法后，二叉树如下图所示：



释放第 2 次魔法后，二叉树如下图所示：



释放第 3 次魔法后，二叉树如下图所示：



因此最后的颜色序列为 111011。

```
#include<bits/stdc++.h>
#define MIN(a,b) a<b?a:b
using namespace std;
vector<int>a[100005];
int b[100005];
bool path[100005];
char s[100010];
```

```

void dfs(int u) {
    if(a[u].empty()) {
        return;
    }
    for(int i=0; i<a[u].size(); i++) {
        b[a[u][i]]^=b[u];
        dfs(a[u][i]);
    }
}

int main() {
    int n,q;
    int x;
    int T;
    scanf("%d", &T);
    while(T--) {
        scanf("%d",&n);
        for(int i = 1; i <= n; i++) a[i].clear(), b[i] = 0;
        for(int i=2; i<=n; i++) {
            scanf("%d",&x);
            a[x].push_back(i);
        }
        scanf("%s",s);
        scanf("%d",&q);
        while(q--) {
            scanf("%d",&x);
            b[x]^=1;
        }
        dfs(1);
        for(int i=0; i<n; i++) {
            printf("%d",(s[i]-'0')^b[i+1]);
        }
        puts("");
    }
}

```

G XIAO7 的猫粮

题目描述

成功变成猫猫的 XIAO7 开设了一家猫娘乐园，但由于猫猫太多，XIAO7 的猫粮不够吃了。

于是天上传来一个声音：「如果你能答对以下若干个问题，就会有足够多的猫粮从天而降。」

请你救救 XIAO7 和她的猫猫们！

具体来说，给定一个整数 n ，不妨将其看作一个十进制数字串，规定如下两种操作：

- 删除数字串中的任何一位数字（允许执行此操作而出现数字串为空的情况）。
- 在数字串中的最右侧添加一位数字。

以上操作可以按任意顺序执行任意次数。请问最少需要执行多少次操作才能使数字串对应的整数成为 2 的任意非负整数次幂（即 $2^i, i \in \mathbb{N}$ ）？

最终得到的数字串不能有前导零。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示最少操作次数。

输入样例

```
8
1256
701
12
95
512
7635
25765
100000
```

输出样例

```
1
2
1
3
0
4
2
5
```

样例解释

正确答案对应的操作种类和顺序可能不唯一。

- $n = 1256$, 答案为 1, 删除数字 1。
- $n = 701$, 答案为 2, 删除数字 7 和 0, 得 $1 = 2^0$ 。
- $n = 12$, 答案为 1, 在右侧添加数字 8, 得 $128 = 2^7$ 。
- $n = 95$, 答案为 3, 删除数字 9 和 5, 随后添加数字 8, 得 $8 = 2^3$ 。
- $n = 512$, 答案为 0, $512 = 2^9$, 无需执行操作。
- $n = 7635$, 答案为 4, 删除数字 3 和 5, 右侧添加数字 4, 删除数字 7, 得 $64 = 2^6$ 。
- $n = 25765$, 答案为 2, 删除数字 7 和最后一位数字 5, 得 $256 = 2^8$ 。
- $n = 100000$, 答案为 5, 删除所有的数字 0, 得 $1 = 2^0$ 。

Hint

XIAO7: 发出喵喵的声音~

AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
using namespace std;
typedef long long ll;
const int N = 55;
const int inf = 0x3f3f3f3f;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
int A[N], B[N];
int match(int x, ll target) {
    int n = 0, m = 0, i, j;
    while(x)
        A[++ n] = x % 10, x /= 10;
    while(target)
        B[++ m] = target % 10, target /= 10;

    for(i = m, j = n; i && j; i --, j --) {
        for(; j && B[i] != A[j]; j --);
        if(j <= 0) break ;
    }
}
```

```

int delete_num = n - (m - i);
int add_num = i;

return delete_num + add_num;
}
int main() {
    int T = getint();
    while(T --) {
        int n = getint(), ans = inf;
        for(int i = 0; i <= 60; i ++){
            ans = min(ans, match(n, 1ll << i));
        }
        cout << ans << endl;
    }
    return 0;
}

```

H 任务达人莫卡 I

题目描述



[高价收购致命的短枪，卖的密！]

莫卡穿越到了一个新的世界并且化身成了名为任务达人的 NPC。

现在莫卡手里有 n 个任务，编号为 $1, 2, \dots, n$ ，难度分别为 $a_1, a_2, \dots, a_n$ 。对于每一名前来领取任务的旅行者，莫卡会从中任意选取 3 个不同的任务，设编号为 i, j, k ($i \neq j, j \neq k, k \neq i$)。但是每个旅行者对任务的难度有独特的要求，他们会提出两个参数 l 和 r ，要求 $2 \cdot a_i \leq a_j \leq l \cdot a_i$ 且 $2 \cdot a_j \leq a_k \leq r \cdot a_j$ 。

现在莫卡想知道，对于每位旅行者，满足其难度要求的任务选择方案有多少种？由于答案可能很大，你只需要输出答案对 $10^9 + 7$ 取模后的结果。

两个选择方案（设任务编号分别为 i, j, k 和 i', j', k' ）被视作不同的方案当且仅当 $i \neq i'$ 或 $j \neq j'$ 或 $k \neq k'$ 。

输入格式

第一行一个正整数 n ($3 \leq n \leq 10^5$)，表示任务的数量。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^3$)，表示每个任务的难度。

第三行一个正整数 q ($1 \leq q \leq 10^5$)，表示旅行者的数量。

接下来 q 行每行两个正整数 l_i, r_i ($2 \leq l_i, r_i \leq 10^3$)，表示第 i 个旅行者提出的两个参数。

输出格式

输出 q 行，第 i 行一个非负整数，表示满足第 i 个旅行者难度要求的方案数对 $10^9 + 7$ 取模后的结果。

输入样例

```

4
1 2 2 4
1
2 3

```

输出样例

```

2

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

```

```

const int p = 1e9 + 7;

int n, q;
int a[maxn];

int main() {
    scanf("%d", &n);
    for(int i = 1; i <= n; i++) {
        int x;
        scanf("%d", &x);
        a[x]++;
    }
    for(int i = 1; i <= 1000; i++) a[i] += a[i - 1];
    scanf("%d", &q);
    while(q--) {
        int l, r;
        scanf("%d%d", &l, &r);
        int ans = 0;
        for(int i = 1; i <= 1000; i++) {
            int x = a[i] - a[i - 1];
            int L = (i + l - 1) / l, R = i / 2;
            if(L > R) continue;
            int sum1 = a[R] - (L ? a[L - 1] : 0);
            L = i * 2, R = i * r;
            if(L > 1000) continue;
            R = min(R, 1000);
            int sum2 = a[R] - a[L - 1];
            ans = (ans + 1ll * x * sum1 % p * sum2) % p;
        }
        printf("%d\n", ans);
    }
}

```

I XIAO7 的地图

题目描述

XIAO7 的猫娘乐园中有一条很长的几乎覆盖了整个园区的路径。

这天 Couteau 前来参观，XIAO7 却画不出地图，只能请你帮帮她 => <=。

猫娘乐园的地图是一种递归定义的分形图，下面给出大小为 1, 2, 3 的地图图示，相信聪明的你一定可以总结出规律来~

在下面的图示中，纯白色的格子表示空地，其他颜色的格子（灰色和各种蓝色）表示路径。

本题采用字符画的形式输出猫娘乐园的地图，其中路径用 ■ 表示，空地用 表示。

当猫娘乐园的大小为 1 时，对应的输出为：

```

***
 *
***

```

当猫娘乐园的大小为 2 时，对应的输出为：

```

* *****
* * *
*** ***
 *
*** *****
* * *
* *****

```

一行一个正整数 n ($1 \leq n \leq 10$)，表示猫娘乐园的大小。

按题目描述中的规定，输出对应大小猫娘乐园的字符画。

XIAO7: 发出呜喵的声音~



```
#include<iostream>
#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 3005;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}

bool mat[N][N], tep[N][N];
void setMat(int sx, int sy, int n) {
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            mat[sx + i][sy + j] = tep[1 + i][1 + j];
}

void dfs(int n, int m, int sx, int sy) {
```

```

if(n == 1) {
    mat[sx][sy] = mat[sx][sy + 1] = mat[sx][sy + 2] = 1;
    mat[sx + 1][sy] = mat[sx + 1][sy] = 0, mat[sx + 1][sy + 2] = 1;
    mat[sx + 2][sy] = mat[sx + 2][sy + 1] = mat[sx + 2][sy + 2] = 1;
    return ;
}

dfs(n - 1, m >> 1, sx, sy);

int sz = m >> 1;
for(int i = 1; i <= sz; i++)
    for(int j = 1; j <= sz; j++) {
        tep[i][j] = mat[sx + i - 1][sy + j - 1];
        mat[sx + i - 1][sy + j - 1] = 0;
    }

setMat(sx, sy + sz + 1, sz);
setMat(sx + sz + 1, sy + sz + 1, sz);

for(int i = 1; i <= m; i++)
    mat[sx + sz][i] = mat[i][sy + sz] = 0;
mat[sx][sy + sz] = mat[sx + m - 1][sy + sz] = mat[sx + sz][sy + sz + 1] = 1;

for(int i = 1; i <= sz/2; i++)
    for(int j = i; j < sz - i + 1; j++) {
        int r = tep[i][j];
        tep[i][j] = tep[sz - j + 1][i];
        tep[sz - j + 1][i] = tep[sz - i + 1][sz - j + 1];
        tep[sz - i + 1][sz - j + 1] = tep[j][sz - i + 1];
        tep[j][sz - i + 1] = r;
    }

setMat(sx, sy, sz);

for(int i = 1; i <= sz/2; i++)
    for(int j = 1; j <= sz; j++)
        swap(tep[i][j], tep[sz - i + 1][j]);

setMat(sx + sz + 1, sy, sz);
}

int main() {
    int n = getint(), m = 3;
    for(int i = 2; i <= n; i++)
        m = m << 1 | 1;
    dfs(n, m, 1, 1);

    for(int i = 1; i <= m; i++) {
        int j = m;
        for(; mat[i][j] == 0; j--);
        for(int k = 1; k <= j; k++)
            putchar(mat[i][k] ? '*' : ' ');
        putchar(10);
    }
    return 0;
}

```

J 小水獭和战狼

题目描述



小水獭正在沉迷电影「War Wolf」，所以它的算法作业只能请求你帮忙完成了~

具体来说，给定两个整数序列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_n$ ，定义整数序列 $i_1, i_2, \dots, i_m$ ($m \geq 1$) 是一个沃沃序列当且仅当其满足如下三个条件：

- $1 \leq i_1 < i_2 < \dots < i_m \leq n$
- $a_{i_1} < a_{i_2} < \dots < a_{i_m}$
- $b_{i_1} < b_{i_2} < \dots < b_{i_m}$

你需要对于 $j = 1, 2, \dots, n$ 求解有多少个沃沃序列满足 $i_m = j$ ，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，含义同题目描述。

第三行 n 个整数 $b_1, b_2, \dots, b_n$ ($-10^9 \leq b_i \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 n 个非负整数，第 j ($1 \leq j \leq n$) 个数表示满足 $i_m = j$ 的沃沃序列数量对 2^{32} 取模后的结果。

输入样例

```
2
3
1 2 3
2 1 3
3
1 2 3
3 2 1
```

输出样例

```
1 1 3
1 1 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

unsigned c[maxn];

void add(int x, unsigned y) {
    for(; x < maxn; x += x & -x) c[x] += y;
}

unsigned query(int x) {
    unsigned sum = 0;
    for(; x; x -= x & -x) sum += c[x];
    return sum;
}

int t, n;
int a[maxn], b[maxn];
unsigned f[maxn];
```

```

void solve(int l, int r) {
    if(l == r) {
        f[l]++;
        return;
    }
    int mid = l + r >> 1;
    solve(l, mid);
    vector<pair<int, int>> v;
    for(int i = l; i <= r; i++) v.push_back({a[i], -i});
    sort(v.begin(), v.end());
    for(auto it : v) {
        int i = -it.second;
        if(i <= mid) add(b[i], f[i]);
        else f[i] += query(b[i] - 1);
    }
    for(auto it : v) {
        int i = -it.second;
        if(i <= mid) add(b[i], -f[i]);
    }
    solve(mid + 1, r);
}

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        vector<int> v;
        for(int i = 1; i <= n; i++) scanf("%d", &b[i]), v.push_back(b[i]);
        sort(v.begin(), v.end());
        v.erase(unique(v.begin(), v.end()), v.end());
        for(int i = 1; i <= n; i++) b[i] = lower_bound(v.begin(), v.end(), b[i])
- v.begin() + 1;
        for(int i = 1; i <= n; i++) f[i] = 0;
        solve(1, n);
        for(int i = 1; i <= n; i++) printf("%u ", f[i]);
        puts("");
    }
}

```

E2

A 简单的可删堆

本题必须使用 C 语言提交。

题目描述

本题是堆的一种拓展应用，你的任务是实现一个小根堆，即根节点为所有元素最小值的堆，需要支持插入元素、查询堆顶元素、删除堆顶元素、删除堆中任意元素四个操作。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示操作的数量。

接下来 n 行，每行为一个操作，格式如下：

- 1 x : 向堆中插入元素 x ($1 \leq x \leq 10^9$)。
- 2: 删除堆顶元素。
- 3: 查询堆顶元素。
- 4 x : 删除堆中元素 x ($1 \leq x \leq 10^9$)，若有多个相同的元素，只需删除一个。

数据保证进行操作 2, 3 时堆非空，进行操作 4 时 x 必为堆中元素。

输出格式

对于每次查询堆顶元素时，输出一行一个正整数，表示此时堆顶元素的值。

在所有操作结束后，将堆中的元素从小到大依次输出到一行中。

输入样例

```
10
1 2
1 4
3
2
1 1
4 4
1 6
1 3
4 3
3
```

输出样例

```
2
1
1 6
```

Hint

堆的实现可以参考《算法导论》第三版的 84 页 - 93 页。

堆还有很多其他的实现方式，不同的堆有不同的性质，感兴趣的同学可以课后进行研究。

如果使用暴力删除的方式只可以获得部分分数，请思考复杂度正确的算法。

实在没有思路的同学可以考虑一下这道题的标题（所以究竟可删堆是什么呢？）。

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#include<stdlib.h>
#include<ctype.h>
#define pi acos(-1)
#define inf 2147483647
#define llinf 9223372036854775807
#define MIN(a,b) (a<b?a:b)
#define MAX(a,b) (a>b?a:b)

#define N 200010
void swap(int *a,int *b) {
    int t=*a;
    *a=*b;
    *b=t;
}
int heap_size=0;
int heap[N];

int heapDel_size=0;
int heapDel[N];
```

```

void put(int x) {
    int now,next,res;
    heap[++heap_size]=x;
    now=heap_size;
    while(now>1) {
        next=now>>1;
        if(heap[now]>=heap[next]) {
            return;
        } else {
            swap(&heap[now],&heap[next]);
            now=next;
        }
    }
}

int get() {
    if(heap_size<1)exit(-1);
    return heap[1];
}

int del() {
    if(heap_size<1)exit(-1);
    int res=heap[1];
    heap[1]=heap[heap_size--];
    int now=1,next;
    while(now*2<=heap_size) {
        next=now*2;
        if(next<heap_size&&heap[next+1]<heap[next]) {
            next++;
        }
        if(heap[now]<=heap[next]) {
            return res;
        }
        swap(&heap[now],&heap[next]);
        now=next;
    }
    return res;
}

void putDel(int x) {
    int now,next,res;
    heapDel[++heapDel_size]=x;
    now=heapDel_size;
    while(now>1) {
        next=now>>1;
        if(heapDel[now]>=heapDel[next]) {
            return;
        } else {
            swap(&heapDel[now],&heapDel[next]);
            now=next;
        }
    }
}

int getDel() {
    if(heapDel_size<1)return -2;
    return heapDel[1];
}

```

```

int delDel() {
    int res=heapDel[1];
    heapDel[1]=heapDel[heapDel_size--];
    int now=1,next;
    while(now*2<=heapDel_size) {
        next=now*2;
        if(next<heapDel_size&&heapDel[next+1]<heapDel[next]) {
            next++;
        }
        if(heapDel[now]<=heapDel[next]) {
            return res;
        }
        swap(&heapDel[now],&heapDel[next]);
        now=next;
    }
    return res;
}

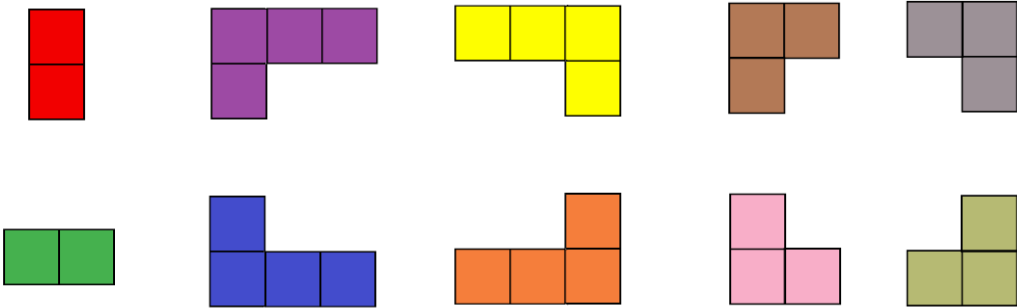
int main() {
    int n,op,x;
    scanf("%d",&n);
    while(n-->0) {
        scanf("%d",&op);
        if(op==1) {
            scanf("%d",&x);
            put(x);
        } else if(op==2) {
            while(get()==getDel()) {
                del();
                delDel();
            }
            del();
        } else if(op==3) {
            while(get()==getDel()) {
                del();
                delDel();
            }
            printf("%d\n",get());
        } else if(op==4) {
            scanf("%d",&x);
            putDel(x);
        }
    }
    while(heap_size>0) {
        if(get()==getDel()) {
            del();
            delDel();
        } else {
            printf("%d ",del());
        }
    }
    return 0;
}

```

B 比较简单的铺瓷砖

题目描述

有一个宽度为 2，长度为 n 的地板需要贴上瓷砖，有以下 10 种形状的瓷砖可供选择：



瓷砖不能旋转、翻转、重叠、切割，且必须恰好把地板铺满，可以结合输入输出样例理解。

the_ignorant 想知道一共有多少种不同的铺瓷砖方案，两种方案不同当且仅当存在一个位置上的瓷砖颜色不同。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^6$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，表示地板的长度。

输出格式

对于每组数据，输出一行一个非负整数，表示贴满瓷砖的方案数对 998 244 353 取模后的结果。

输入样例

```
5
1
2
3
4
5
```

输出样例

```
1
2
9
21
48
```

Hint

比较简单的递推，类似于斐波那契数。

P.S. 请枚举所有可能的情况，能过样例就证明你基本上对了。

P.P.S. 注意数据范围以及考虑递推与递归的区别。

```
#include<stdio.h>
#define ll long long int
#define M 1000100

ll MOD=998244353;
ll result[M]= {1, 1, 2, 9, 21, 48};
ll sum[M]= {1, 2, 4, 13, 34, 82};
int main() {
    int t, n, i, j;
    scanf("%d",&t);
    for(i=6; i<M; i++) {
        result[i] = (result[i-1] + result[i-2] + 4*result[i-3] + 2*result[i-4]
+2*sum[i-3])%MOD;
        sum[i] = (sum[i-1] + result[i])%MOD;
    }
    for(i=0; i<t; i++) {
        scanf("%d",&n);
```

```
printf("%11d\n",result[n]);
}
return 0;
}
```

C 小水獭和子集

本题提交显示 Other Error 代表 Wrong Answer。

题目描述



小水獭在预推免面试过程中被问到了一个难题，想要紧急求助你，你能帮帮它嘛？

具体来说，给定正整数 n 和 m ，你需要构造 m 个非空正整数集合 $S_1, S_2, \dots, S_m$ ，满足：

- $S_i \subseteq \{1, 2, \dots, n\}$ 对 $1 \leq i \leq m$ 成立。
- $S_i \not\subseteq S_j$ 对 $1 \leq i, j \leq m, i \neq j$ 成立。

当然，题目有可能是无解的，此时你需要告知小水獭。

输入格式

第一行一个正整数 t ($1 \leq t \leq 2^{11}$)，表示数据组数。

对于每组数据，一行两个正整数 n, m ($1 \leq n \leq 10, 1 \leq m \leq 2^n - 1$)，含义同题目描述。

输出格式

对于每组数据，输出若干行：

- 若题目有解，输出 m 行，第 i ($1 \leq i \leq m$) 行第一个整数 $|S_i|$ ($1 \leq |S_i| \leq n$)，表示 S_i 的大小，接下来 $|S_i|$ 个正整数，表示 S_i 中的元素。如果有多种可能的答案，你可以输出任意一种。
- 否则，输出一行 -1 。

输入样例

```
4
3 1
3 2
3 3
3 4
```

输出样例

```
1 1
1 1
1 2
1 1
1 1
1 2
1 3
-1
```

```
#include <bits/stdc++.h>

using namespace std;

int t, n, m;

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        int ans = 1;
        for(int i = 1;i <= (n + 1) / 2;i++) ans *= n - i + 1, ans /= i;
        if(ans < m){
            puts("-1");
        }
    }
}
```

```

        continue;
    }
    for(int i = 0; i < (1 << n) && m; i++) if(__builtin_popcount(i) == (n + 1)
/ 2){
        m--;
        printf("%d ", (n + 1) / 2);
        for(int j = 0; j < n; j++) if(i & (1 << j)) printf("%d ", j +
1); puts("");
    }
}
}
}

```

D 莫卡和皇帝

题目描述

蛇皇帝扔给莫卡一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 初始有 $a_i = i$, 你需要维护一个数据结构, 支持 m 次下列三种操作之一:

1. 给定正整数 l, r , 对任意 $l \leq i \leq r$, 将 a_i 变为 a_i^2 .
2. 给定正整数 l, r, k , 对任意 $l \leq i \leq r$, 将 a_i 变为 $k \cdot a_i$.
3. 给定正整数 l, r , 输出 $\left(\prod_{i=l}^r a_i\right) \bmod (r-l+1)!$.

输入格式

第一行两个正整数 n, m ($1 \leq n, m \leq 10^6$), 含义同题目描述。

接下来 m 行, 每行代表一个操作, 为以下三种形式之一:

- 1 l r
- 2 l r k
- 3 l r

保证 $1 \leq l \leq r \leq n$ 且 $1 \leq k \leq 100$.

输出格式

对于每个第三种操作, 输出一行一个非负整数, 表示 $\left(\prod_{i=l}^r a_i\right) \bmod (r-l+1)!$ 的值。

输入样例

```

2 1
3 1 2

```

输出样例

```

0

```

```

#include <stdio.h>
int main(){
    int n, m, op, l, k, r;
    scanf("%d%d", &n, &m);
    while(m--){
        scanf("%d", &op);
        switch (op){
            case 1:
                scanf("%d%d", &l, &r);
                break;
            case 2:
                scanf("%d%d%d", &l, &r, &k);
                break;
            case 3:
                scanf("%d%d", &l, &r);
                printf("0\n");
        }
    }
}

```

```
    }  
}  
return 0;  
}
```

E 奇怪的汉诺獭

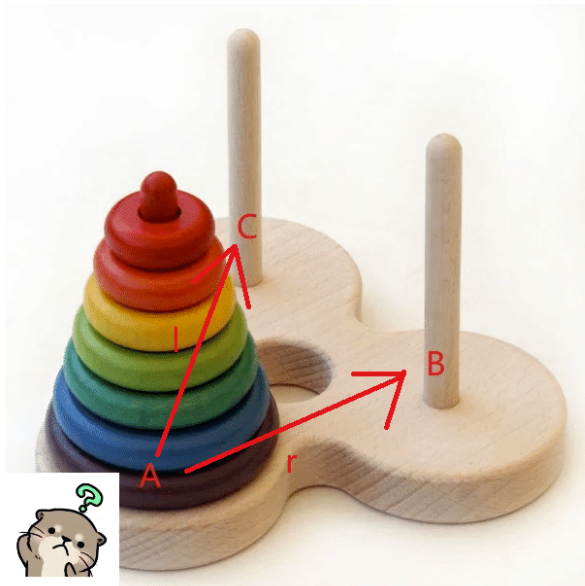
题目描述

小水獭学算法学累了，于是开始玩汉诺塔。它了解到，汉诺塔是一个有名的益智游戏，一共有三根柱子，其中一根柱子上从下往上按照大小顺序摆着 n 片圆盘，要求圆盘从下面开始按大小顺序重新摆放在另一根柱子上。并且规定，在小圆盘上不能放大圆盘，在三根柱子之间一次只能移动一个圆盘。

小水獭非常聪明，它很快明白了普通汉诺塔玩法，然而它还没有玩够，于是它构想了一个奇怪的汉诺塔。如下图所示，三柱均匀分布在一个圆形基座上，规定初始柱为 A 柱，目标柱为 C 柱，中间柱为 B 柱，且每个圆盘仅能依次顺时针旋转或逆时针旋转，其他规则和普通的汉诺塔相同。

举个例子，若圆盘 2 规定为仅能顺时针旋转，那么它仅能以 $A \rightarrow C \rightarrow B \rightarrow A$ 的顺序移动，若此时圆盘 2 在 A 柱，而圆盘 1 在 C 柱，则圆盘 2 无法移动，只能先设法移开圆盘 1 才能移动。

小水獭想问你如何以最少的操作次数，使所有圆盘从 A 柱移动到 C 柱。



输入格式

本题有多组数据，总组数不超过 10，请使用 EOF 判断读入结束。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 20$) 和一个长度为 n 的字符串 s ，其中 n 表示圆盘的数量， s 由 1 和 r 组成，第 i ($1 \leq i \leq n$) 代表第 i 小的圆盘所能旋转的方向， 1 代表顺时针方向 ($A \rightarrow C \rightarrow B \rightarrow A$)， r 代表逆时针旋转 ($A \rightarrow B \rightarrow C \rightarrow A$)。

输出格式

对于每组数据，输出一个操作方案。每个方案包含若干行操作，每步操作以形如 `move i from A to C` (意思是将第 i 小的圆盘从 A 柱移动到 C 柱) 的形式表现，在全部操作结束时，输出一个空行 (最后一组数据后有无空行不影响评测结果)。

输入样例

```
1 1  
1 r  
2 11
```

输出样例

```
move 1 from A to C  
  
move 1 from A to B  
move 1 from B to C  
  
move 1 from A to C  
move 1 from C to B  
move 2 from A to C  
move 1 from B to A  
move 1 from A to C
```

Hint

真是一个奇怪的汉诺獭，小水獭也表示很无奈。

```
#include <bits/stdc++.h>  
#define inf 1e18  
#define maxn 20005
```

```

typedef long long ll;
typedef unsigned long long ull;
using namespace std;
const ll mod = 998244353;
const double eps = 1e-4;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n;
char s[maxn];

char pan[5] = {' ', 'A', 'B', 'C'};

void solve(int n, int now, int to){
    if(n == 1){
        if(to == now % 3 + 1) {
            if(s[n] == 'r') printf("move 1 from %c to %c\n", pan[now], pan[to]);
            else printf("move 1 from %c to %c\n", pan[now], pan[6 - now - to]),
printf("move 1 from %c to %c\n", pan[6 - now - to], pan[to]);
        }
        else{
            if(s[n] == 'l') printf("move 1 from %c to %c\n", pan[now], pan[to]);
            else printf("move 1 from %c to %c\n", pan[now], pan[6 - now - to]),
printf("move 1 from %c to %c\n", pan[6 - now - to], pan[to]);
        }
        return ;
    }
    if(to == now % 3 + 1) {
        if(s[n] == 'r') solve(n - 1, now, 6 - now - to), printf("move %d from %c
to %c\n", n, pan[now], pan[to]), solve(n - 1, 6 - now - to, to);
        else {
            solve(n - 1, now, to);
            printf("move %d from %c to %c\n", n, pan[now], pan[6 - now - to]);
            solve(n - 1, to, now);
            printf("move %d from %c to %c\n", n, pan[6 - now - to], pan[to]);
            solve(n - 1, now, to);
        }
    }
    else{
        if(s[n] == 'l') solve(n - 1, now, 6 - now - to),printf("move %d from %c
to %c\n", n, pan[now], pan[to]), solve(n - 1, 6 - now - to, to);
        else {
            solve(n - 1, now, to);
            printf("move %d from %c to %c\n", n, pan[now], pan[6 - now - to]);
            solve(n - 1, to, now);
            printf("move %d from %c to %c\n", n, pan[6 - now - to], pan[to]);
            solve(n - 1, now, to);
        }
    }
}

```

```
int main(){

    while(scanf("%d%s", &n, s + 1) != EOF){
        solve(n, 1, 3);
        puts("");
    }
    return 0;
}
```

F 小水獭和子数组

题目描述



小水獭在夏令营机试过程中遇到了一个难题，想要紧急求助你，你能帮帮它嘛？

具体来说，给定一个长度为 n 的数组 $a_1, a_2, \dots, a_n$ ，你需要求解其中所有子数组的极差的和。

形式化地说，定义 $S(l, r)$ ($1 \leq l \leq r \leq n$) 表示 $a_l, a_{l+1}, \dots, a_r$ 这些元素所组成的集合， $\max S$ 表示集合 S 中的最大值， $\min S$ 表示集合 S 中的最小值。请你求解：

$$\sum_{i=1}^n \sum_{j=i}^n [\max S(i, j) - \min S(i, j)]$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示数组的长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，表示数组的元素。

输出格式

对于每组数据，输出一行一个整数，表示所有子数组的极差的和。

输入样例

```
2
5
1 2 3 4 5
5
233 233 233 233 233
```

输出样例

```
20
0
```

Hint

你可能需要使用 `__int128` 类型来存储最终的结果，但该类型无法使用常规的方式输出，这里给出一个该类型的输出实现：

```
void print(__int128 x){
    if(x < 10){
        printf("%d", (int)x);
        return;
    }
    print(x / 10);
    printf("%d", (int)(x % 10));
}
```

```
#include <bits/stdc++.h>
#define maxn 1000086
```

```
using namespace std;
```

```
int t, n;
int a[maxn];
int sk[maxn], top;
```

```

int l[maxn], r[maxn];

void print(__int128 x){
    if(x < 10){
        printf("%d", (int)x);
        return;
    }
    print(x / 10);
    printf("%d", (int)(x % 10));
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        __int128 ans = 0;
        top = 0;
        for(int i = 1; i <= n; i++){
            while(top && a[sk[top]] <= a[i]) top--;
            if(top) l[i] = sk[top] + 1;
            else l[i] = 1;
            sk[++top] = i;
        }
        top = 0;
        for(int i = n; i; i--){
            while(top && a[sk[top]] < a[i]) top--;
            if(top) r[i] = sk[top] - 1;
            else r[i] = n;
            sk[++top] = i;
        }
        for(int i = 1; i <= n; i++) ans += (__int128)a[i] * (i - l[i] + 1) * (r[i]
- i + 1);
        top = 0;
        for(int i = 1; i <= n; i++){
            while(top && a[sk[top]] >= a[i]) top--;
            if(top) l[i] = sk[top] + 1;
            else l[i] = 1;
            sk[++top] = i;
        }
        top = 0;
        for(int i = n; i; i--){
            while(top && a[sk[top]] > a[i]) top--;
            if(top) r[i] = sk[top] - 1;
            else r[i] = n;
            sk[++top] = i;
        }
        for(int i = 1; i <= n; i++) ans -= (__int128)a[i] * (i - l[i] + 1) * (r[i]
- i + 1);
        print(ans); puts("");
    }
}

```

题目描述

czz 在离散数学课上对一只猫娘一见钟情，于是他找到猫娘乐园的园长 XIAO7 寻求帮助。

XIAO7 告诉他：「如果你能答对以下问题，下周三我就帮你略施小计。」

具体来说，在离散数学课上 czz 已经学到了组合数的定义，对于非负整数 n 和 m 有：

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}$$

其中 $n! = 1 \times 2 \times \cdots \times n$ ，特别地， $0! = 1$ 。

由组合意义可得，当 $n \geq m$ 时组合数是一个正整数。

现在对于给定的三个整数 n, m, k ，XIAO7 想要知道 $\binom{n}{m}$ 在 k 进制下末尾 0 的数量。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，每行三个整数 n, m, k ($0 \leq m \leq n \leq 10^9, 2 \leq k \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示 $\binom{n}{m}$ 在 k 进制下末尾 0 的数量。

输入样例

```
3
10 6 10
8 1 2
27 3 3
```

输出样例

```
1
3
2
```

样例解释

十进制下， $\binom{10}{6} = 210$ ，末尾有一个 0。

十进制下， $\binom{8}{1} = 8$ ，8 在二进制下为 1000，末尾有三个 0。

十进制下， $\binom{27}{3} = 2925$ ，2925 在三进制下为 11000100，末尾有两个 0。

Hint

XIAO7：发出喵喵的声音~



AUTHOR：YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<map>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 3000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
```

```

while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
while(c>='0' && c<='9'){num=num*10+c-48;c=getchar();}
return flag ? -num : num;
}
int pri[N], num[N];
int getzero(int n, int x){
    int cnt = 0;
    while(n)
        cnt += n / x, n /= x;
    return cnt;
}
int getpri(int n){
    int h = sqrt(n + 0.5), cnt = 0;
    for(int i = 2; i <= h; i++){
        if(n % i) continue;
        pri[++ cnt] = i, num[cnt] = 0;
        while(n % i == 0)
            n /= i, num[cnt]++;
    }
    if(n > 1) pri[++ cnt] = n, num[cnt] = 1;
    return cnt;
}
int main(){
    int T = getint();
    while(T--){
        int n = getint(), m = getint(), k = getint();
        int cnt = getpri(k), ans = inf;
        for(int i = 1; i <= cnt; i++){
            int zero = getzero(n, pri[i]);
            zero -= getzero(m, pri[i]);
            zero -= getzero(n - m, pri[i]);
            ans = min(ans, zero / num[i]);
        }
        cout << ans << endl;
    }
}

```

H XIAO7 与 A+B problem

题目描述

猫娘「雪风」上周没有穿到裙子，她非常难过。

XIAO7 决定再给她一次机会：「如果你能答对以下若干个问题，我就给你买裙子。」

请你帮帮可爱的「雪风」！

具体来说，给定两个整数 A, B ，你需要找到一个整数 X 且 $|X| \leq 2^{64}$ 使得

$$(A \oplus X) + (B \oplus X) = X$$

其中 $\oplus$ 表示按位异或运算，所有数按照 128 位补码进行运算。

若不存在这样的 X 则请大声喊出 `I would like to see xf wear a skirt every day !`

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个整数 A, B ($-2^{60} \leq A, B \leq 2^{60}$)。

输出格式

对于每组数据，输出一行：

- 若存在合法的 X ，输出 X ，容易证明此时仅有唯一的 X 满足条件。
- 否则，输出 `I would like to see xf wear a skirt every day !`。

输入样例

```
5
-1 -1
1 2
3 5
6 6
15 16
```

输出样例

```
I would like to see xf wear a skirt every day !
3
-8
4
31
```

Hint

XIAO7: 发出喵喵的声音~



AUTHOR: YUKI

```
#include<cstdio>
#include<cmath>
#include<cstring>
#include<cstdlib>
#include<iostream>
#include<algorithm>
#include<queue>
#include<set>
#include<map>
#include<vector>
typedef long long int LL;
typedef unsigned long long ULL;
const int INF=0x3f3f3f3f;
const LL INFLL=0x3f3f3f3f3f3f3f3fll;
LL getint(){
    LL ret=0;bool f=0;char c;
    while((c=getchar())<'0' || c>'9')if(c=='-')f^=1;
    while(c>='0'&&c<='9'){ret=ret*10+c-'0';c=getchar();}
    return f?-ret:ret;
}
using namespace std;
const int MAXN=100;
bool xa[MAXN+10],xb[MAXN+10],ans[MAXN+10];
void update(LL a,bool x[MAXN+10]){
    if(a>=0){
        memset(x,0,(MAXN+10)*(sizeof(bool)));
        for(int i=0;a;++i,a>=1)
            x[i]=a&1;
    }
    else{
        memset(x,1,(MAXN+10)*(sizeof(bool)));
        for(int i=0;~a;++i,a>=1)
            x[i]=a&1;
    }
}
int main(){
```

```

int T=getint();
while(T--){
    LL a=getint(),b=getint();
    if(a<0&&b<0){
        printf("I would like to see xf wear a skirt every day !\n");
        continue;
    }
    memset(ans,0,sizeof ans);
    update(a,xa);update(b,xb);
    bool c=0;
    for(int i=0;i<70;++i){
        ans[i]=c^xa[i]^xb[i];
        c=((ans[i]^xa[i])+(ans[i]^xb[i])+c)>1;
    }
    LL ret;
    if(ans[65]){
        ret=-1;
        for(int i=0;i<70;++i)if(!ans[i])
            ret-=(1ll<<i);
    }
    else{
        ret=0;
        for(int i=0;i<70;++i)if(ans[i])
            ret+=(1ll<<i);
    }
    cout<<ret<<'\n';
}
return 0;
}

```

I 小水獭和随机树（困难版）

题目描述



小水獭正在观摩一棵 n 个节点的树，其中每个点的权值来自一个随机数生成器，树的结构也在代码中直接钦定。它想知道对于每个点来说，所有与其距离小于 k 的祖先中（包括其本身），权值最大点的权值是多少？它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
int fa[maxn];
unsigned a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 4; i <= n - 1000; i++) fa[i] = i / 2;
        for(int i = n - 998; i <= n; i++) fa[i] = i - 1;
        fa[n - 999] = 1;
        fa[2] = fa[3] = n;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        for(int i = 1; i <= n; i++){
            unsigned mx = 0;
            int x = i;
            for(int j = 1; j <= k; j++){
                mx = max(mx, a[x]);
                if(x == 1) break;
                x = fa[x];
            }
            ans += mx ^ i;
        }
        printf("%u\n", ans);
    }
}

```

其中 `fa[i]` 表示编号为 `i` 的节点的父节点编号。

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行三个整数 $n, k, seed$ ($10^4 \leq n \leq 10^7$, $1 \leq k \leq n$, $1 \leq seed < 2^{32}$)，表示树的节点数、距离参数和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

输入样例

```

2
10000 3 5
100000 5 7

```

输出样例

```

2472146199
2180026278

```

Hint

树的形态是一条链拼上一棵二叉树。

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
unsigned a[maxn];

```

```

unsigned ans;
int dep[maxn];
int q[maxn], l, r;

void dfs(int i){
    if(i > n) return;
    int L = l, R = r;
    while(l <= r && a[q[r]] <= a[i]) r--;
    int x = q[++r];
    q[r] = i;
    if(dep[i] - dep[q[l]] == k) l++;
    ans += a[q[l]] ^ i;
    if(i == 1){
        dep[n - 999] = dep[i] + 1;
        dfs(n - 999);
    }else if(i == n){
        dep[2] = dep[i] + 1, dfs(2);
        dep[3] = dep[i] + 1, dfs(3);
    }else if(i > n - 1000){
        dep[i + 1] = dep[i] + 1;
        dfs(i + 1);
    }else{
        if((i << 1) <= n - 1000){
            dep[i << 1] = dep[i] + 1;
            dfs(i << 1);
        }
        if((i << 1 | 1) <= n - 1000){
            dep[i << 1 | 1] = dep[i] + 1;
            dfs(i << 1 | 1);
        }
    }
    q[r] = x;
    l = L, r = R;
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand();
        ans = 0, l = 1, r = 0, dfs(1);
        printf("%u\n", ans);
    }
}

```

J XIAO7 和序列

题目描述

猫娘 Asahi 每周都要前往沙河，她觉得好麻烦~！(´・ω・´)

XIAO7 告诉她：「如果你能答对以下若干个问题，我就能把你传送到沙河！」

请你帮帮可爱的 Asahi！

具体来说，给定一个长度为 n 的序列 $a_1, a_2, \cdots, a_n$ 。对于一次询问，给定两个正整数 $1 \leq l \leq r \leq n$ ，你需要求解 $a_l, a_{l+1}, \dots, a_r$ 中每种数字最靠右出现位置减去最靠左出现位置的和。

形式化地说，你需要求解：

$$\sum_{c \in S(l,r)} [R(l,r,c) - L(l,r,c)]$$

其中 $S(l,r)$ ($1 \leq l \leq r \leq n$) 表示 $a_l, a_{l+1}, \dots, a_r$ 这些元素所组成的集合， $L(l,r,c) = \min\{x \mid l \leq x \leq r, a_x = c\}$, $R(l,r,c) = \max\{x \mid l \leq x \leq r, a_x = c\}$ 。

你需要处理 q 次询问。

输入格式

第一行两个整数 n, q ($1 \leq n, q \leq 3 \times 10^5$)，表示序列的长度和询问次数。

第二行 n 个整数 $a_1, a_2, \cdots, a_n$ ($1 \leq a_i \leq 10^9$)，表示序列的元素。

接下来 q 行，第 i ($1 \leq i \leq q$) 行两个正整数 l_i, r_i ($1 \leq l_i \leq r_i \leq n$)，表示第 i 次询问的两个参数。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个非负整数，表示第 i 次询问的答案。

输入样例

```
7 5
1 2 2 3 1 2 3
1 3
1 4
2 6
2 7
2 5
```

输出样例

```
1
1
4
7
1
```

Hint

XIAO7：发出喵喵的声音~

AUTHOR：YUKI

```
#include<bits/stdc++.h>
#define maxn 3000005
using ll = long long;
using namespace std;

inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

map<int, int> mp;

int a[maxn], n, q;
ll ans[maxn], bit[maxn];
void add(int x, int v){for(int i = x; i <= n; i += i & -i) bit[i] += v;}
ll getb(int x){ll ans = 0;for(int i = x; i; i -= i & -i) ans += bit[i];return ans;}
vector<pair<int, int>> v[maxn];

int main(){
```

```

n = read(), q = read();
for(int i = 1; i <= n; i++) a[i] = read();
for(int i = 1; i <= q; i++){
    int l = read(), r = read();
    assert(1 <= l && l <= r && r <= n);
    v[r].push_back({l, i});
}
for(int i = 1; i <= n; i++){
    if(mp.count(a[i])){
        int x = mp[a[i]];
        add(x, i - x);
    }
    mp[a[i]] = i;
    for(auto x: v[i]){
        ans[x.second] = getb(i) - getb(x.first - 1);
    }
}
for(int i = 1; i <= q; i++) printf("%11d\n", ans[i]);
}

```

A 喜迎北航 70 周年校庆！

时间限制：1000ms 内存限制：65536kb

通过率：184/192 (95.83%) 正确率：184/755 (24.37%)

本题希望同学们学会使用 C++ STL (标准模板库) 中的 `priority_queue` (优先队列)，更多请见下文的 Hint。

题目描述

Zhoues 虽然求出了钢条切割方案，使得总销售价格最大，但是真正进行钢条切割的时候，其实还有加工费用需要考虑。

最近钢条厂的生意很红火，尤其正值北航 70 周年校庆，各种英寸的钢条供不应求。现在钢条厂接到北航一份大的订单，需要 n 根钢条，订单中第 i ($1 \leq i \leq n$) 根钢条的长度是 l_i 英寸。

钢条厂有一种电锯，每开动一次可以把一根钢条锯成两段，锯出 n 根钢条就需要开动 $n - 1$ 次电锯，但是每次电锯开动都需要电费等加工费用，而且加工费用是需要锯开的钢条长度的两倍，比如要把一段长 3 英寸的钢条锯开，就要收 6 元。Zhoues 发现不同的切割次序会产生不同的费用，请帮 Zhoues 设计一个完成订单的钢条切割顺序方案，使得钢条厂付出的加工费用最少。

输入格式

第一行一个正整数 n ($1 \leq n \leq 2 \times 10^5$)，表示订单中需要钢条的个数。

接下来一行 n 个正整数 $l_1, l_2, \dots, l_n$ ($1 \leq l_i \leq 5 \times 10^4$)，表示订单中每一根钢条需要的长度。

输出格式

一行一个正整数，表示钢条厂付出的最少加工费用。

输入样例

```
3
8 5 8
```

输出样例

```
68
```

```
#include <iostream>
#include <algorithm>
#include <queue>
using namespace std;
priority_queue<long long, vector<long long>, greater<long long>> q;
long long n, i, tmp, a, b;
long long ans;
void init()
{
    ans = 0;
    while (!q.empty())
        q.pop();
}
int main()
{
    scanf("%lld", &n);
    init();
    for (i = 0; i < n; i++)
    {
        scanf("%lld", &tmp);
        q.push(tmp);
    }
    while (q.size() != 1)
    {
        a = q.top();
        q.pop();
        ans += 2*a;
        b = q.top();
```

```

        q.pop();
        ans += 2*b;
        q.push(a + b);
    }
    printf("%lld\n", ans);
    return 0;
}

```

2

B DNA 配对

时间限制：1000ms 内存限制：65536kb

通过率：190/193 (98.45%) 正确率：190/316 (60.13%)

题目描述

Zhoues 沉迷于医学实验，尤其是 DNA 配对。由于他最近一直在为北航校庆切钢条，因此 DNA 配对实验也停滞不前。他现在遇到了一个实验难题，希望大家可以帮他一下。具体来说，给定两个由 A, T, C, G 组成的字符串，请你求出它们的最长公共子序列。

输入格式

共两行，每行一个由 A, T, C, G 组成的字符串，长度均不超过 10^3 。两个字符串的长度不一定相同，且最长公共子序列的长度至少为 1。

输出格式

一行一个字符串，表示两个字符串的最长公共子序列。如果有多种可能的答案，你可以输出任意一种。

输入样例

```

AGCTTCGA
AGTTCAGC

```

输出样例

```

AGTTCA

```

```

#include<iostream>
#include<string>
#include<cstring>
#include<algorithm>
#include<set>
using namespace std;
char A[1100],B[1100];
int dp[1100][1100];
int n,m;
int targetlen;
set<string> ans;
bool flag = false;
void LCS() {
    for(int i = 1; i<=n; i++) {
        for(int j = 1; j<=m; j++) {

            if(A[i] == B[j]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j],dp[i][j-1]);
            }
        }
    }
    targetlen = dp[n][m];
}

```

```

void dfs(int i, int j, string s) {

    if(i<=0 || j<=0 || flag)    return ;
    if(A[i] == B[j]) {
        s.push_back(A[i]);
        if(s.length() == targetlen) {
            reverse(s.begin(),s.end());
            ans.insert(s);
            flag = true;
        } else    dfs(i-1,j-1,s);
    } else {
        if(dp[i-1][j] >= dp[i][j-1])    dfs(i-1,j,s);
        if(dp[i-1][j] <= dp[i][j-1])    dfs(i,j-1,s);
    }
}

int main() {
    int k;
    scanf("%s%s",A+1,B+1);
    n = strlen(A+1);
    m = strlen(B+1);
    LCS();
    dfs(n,m,"");
    for(string s : ans) {
        puts(s.c_str());
        break;
    }
    return 0;
}

```

3

C 奇怪的 OBST

时间限制：1000ms 内存限制：65536kb

通过率：181/182 (99.45%) 正确率：181/321 (56.39%)

题目描述

对于集合 $\{1, 2, \dots, n\}$ 的一棵二叉搜索树，定义一个节点的深度为该节点到根节点的距离，设权值为 i 的点深度为 d_i ，给定序列 $p_1, p_2, \dots, p_n$ ，定义该二叉搜索树的代价和为：

$$\sum_{i=1}^n (d_i + 1) p_i$$

请你求解集合 $\{1, 2, \dots, n\}$ 所有二叉搜索树中最小的代价和。

输入格式

第一行一个正整数 n ($1 \leq n \leq 500$)，含义同题目描述。

第二行 n 个正整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 10^4$)，含义同题目描述。

输出格式

一行一个正整数，表示最小的代价和。

输入样例

```

5
15 10 5 10 20

```

输出样例

```

130

```

```
#include<iostream>
```

```

#define maxn 1010
typedef long long LL;
using namespace std;
LL e[maxn][maxn], w[maxn][maxn], p[maxn], q[maxn], n;
void OBST()
{
    for(LL i=1;i<=n+1;i++)
        e[i][i-1] = w[i][i-1] = q[i-1];
    for(LL k=1;k<=n;k++)
    {
        for(LL i=1;i<=n-k+1;i++)
        {
            LL j = i+k-1;
            e[i][j] = 0x7fffffff;
            w[i][j] = w[i][j-1] + p[j] + q[j];
            for(LL r = i;r<=j;r++)
            {
                LL t = e[i][r-1] + e[r+1][j] + w[i][j];
                if(t < e[i][j]) e[i][j] = t;
            }
        }
    }
}
int main(){
    scanf("%lld",&n);
    for(LL i = 1;i<=n;i++) scanf("%lld",&p[i]);
    for(LL i = 0;i<=n;i++) scanf("%lld",&q[i]);
    OBST();
    printf("%lld\n",e[1][n]);
    return 0;
}

```

4

D 小水獭和切钢条

时间限制：5000ms 内存限制：65536kb

通过率：16/22 (72.73%) 正确率：16/136 (11.76%)

题目描述

互联网寒冬袭来，小水獭也打算提前进厂，体验一下流水线的感觉是什么样的。更为重要的是，北航下的钢条订单太大以至于切钢条的人手不够啦！

小水獭也给钢条厂发送了一份自己的简历，和 Zhoues 的经历类似，厂里负责招聘的工作人员正好最近也遇到了一个难题，需要一个会动态规划算法的同学帮帮忙。但与 Zhoues 不同的是，需要解决的问题发生了小小的变化：

- 给定一段长度为 n 英寸的钢条和一个价格表，该价格表表示仅有长度不超过 m 英寸的钢条，其中 i ($i = 1, 2, \dots, m$) 英寸的钢条的价格为 p_i 。求将钢条切割为 $\left\lceil \frac{n}{m} \right\rceil, \left\lceil \frac{n}{m} \right\rceil + 1, \dots, n$ 段的方案，使得总销售价格最大，注意钢条的长度必须为整英寸。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 5 \times 10^3, 1 \leq m \leq \min(n, 10)$)，含义同题目描述。

第二行 m 个正整数 $p_1, p_2, \dots, p_m$ ($1 \leq p_i \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 $n - \left\lceil \frac{n}{m} \right\rceil + 1$ 个正整数，表示将钢条切割为 $\left\lceil \frac{n}{m} \right\rceil, \left\lceil \frac{n}{m} \right\rceil + 1, \dots, n$ 段的最大总销售价格。

输入样例

输入样例

```
2
4 4
1 5 8 9
10 10
1 5 8 9 10 17 17 20 24 30
```

输出样例

```
9 10 7 4
30 26 27 26 25 22 19 16 13 10
```

样例解释

```
#include <bits/stdc++.h>
#define maxn 5086

using namespace std;

typedef long long ll;

int t, n, m;
int a[maxn];
ll f[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= m; i++) scanf("%d", &a[i]);
        for(int i = 1; i <= n; i++) f[i] = -1e18;
        f[0] = 0;
        for(int i = 1; i <= n; i++){
            for(int j = n; j >= 0; j--){
                f[j] = -1e18;
                for(int l = 1; l <= m && j - l >= 0; l++) f[j] = max(f[j], f[j - l]
+ a[l]);
            }
            if(i >= (n + m - 1) / m) printf("%lld ", f[n]);
        }
        puts("");
    }
}
```

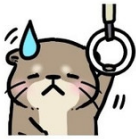
E 小水獭和求解器

时间限制：1000ms 内存限制：65536kb

通过率：11/13 (84.62%) 正确率：11/17 (64.71%)

本题提交显示 Other Error 代表 Wrong Answer。

题目描述



小水獭正在学习《高级算法分析与设计》，但遇到了一点点小问题，你能帮帮它吗？

具体来说，给定 4 个序列 $[a_1, a_2, \dots, a_m], [b_1, b_2, \dots, b_m], [c_1, c_2, \dots, c_m], [d_1, d_2, \dots, d_m]$ ，请你求出一个序列 $[x_1, x_2, \dots, x_n]$ ，满足 $x_i \in \{0, 1\}$ 对 $1, 2, \dots, n$ 成立，且在如下 m 个不等式中有不少于 $\left\lceil \frac{m}{2} \right\rceil$ 个不等式成立：

$$\begin{aligned} (x_{a_1} \oplus b_1) + (x_{c_1} \oplus d_1) &\geq 1 \\ (x_{a_2} \oplus b_2) + (x_{c_2} \oplus d_2) &\geq 1 \\ &\dots \\ (x_{a_m} \oplus b_m) + (x_{c_m} \oplus d_m) &\geq 1 \end{aligned}$$

其中 $\oplus$ 表示按位异或运算。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n, m \leq 50$)，含义同题目描述。

接下来 m 行，每行四个非负整数 a_i, b_i, c_i, d_i ($1 \leq a_i, c_i \leq n, 0 \leq b_i, d_i \leq 1$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个长度为 n 的 01 串，从左到右第 i ($1 \leq i \leq n$) 位表示 x_i 的取值。如果有多种可能的答案，你可以输出任何一种。

输入样例

```
2
4 2
1 0 2 0
3 1 4 1
2 2
1 0 1 0
1 1 1 1
```

输出样例

```
1100
11
```

```
#include <bits/stdc++.h>
#define maxn 55

using namespace std;

int t, n, m;
int a[maxn], b[maxn], c[maxn], d[maxn];
bool vis[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= m; i++) scanf("%d%d%d%d", &a[i], &b[i], &c[i], &d[i]),
vis[i] = false;
        for(int i = 1; i <= n; i++){
            int cnt[2] = {0};
            for(int j = 1; j <= m; j++) if(!vis[j]){
                if(a[j] == i) cnt[b[j]]++, vis[j] = true;
                if(c[j] == i) cnt[d[j]]++, vis[j] = true;
            }
        }
    }
}
```

```
        printf("%d", cnt[0] > cnt[1]);
    }
    puts("");
}
}
```

6

F Build Your Tree

时间限制: 1000ms 内存限制: 65536kb
通过率: 10/12 (83.33%) 正确率: 10/38 (26.32%)

题目描述

the_ignorant 有一棵 n 个结点的树。但是这棵树太过繁密，他想要去繁为简，但同时又想保持一种整齐的美感。因此，他希望你能够将其删除若干条边，使得剩余连通块的结点数均为 k 。

若存在这样的一种方案，请你输出 **YES**，否则输出 **NO**。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行两个正整数 n, k ($1 \leq k \leq n \leq 10^3$)，含义同题目描述。

第二行 $n - 1$ 个正整数 $a_2, a_3, \dots, a_n$ ($1 \leq a_i < i$)，表示树上存在边 (i, a_i) 。

输出格式

对于每组数据，输出一行：

- 若存在满足条件的方案，输出 **YES**。
- 否则，输出 **NO**。

输入样例

```
2
2 1
1
3 2
1 2
```

输出样例

```
YES
NO
```

```
#include<bits/stdc++.h>
#define N 2000

using namespace std;
vector<int>son[N];
int fa[N];
bool vis[N];
int ans[N];
void solve(int x)
{
    if(ans[x])
    {
        return;
    }
    ans[x]=1;
    for(int i=0;i<son[x].size();i++)
    {
        solve(son[x][i]);
        ans[x]+=ans[son[x][i]];
    }
}
int main()
{
}
```

```

int t,n,k,x;
int tmp;
scanf("%d",&t);
while(t--)
{
    tmp=0;
    scanf("%d%d",&n,&k);
    for(int i=2;i<=n;i++)
    {
        scanf("%d",&x);
        fa[i]=x;
        son[x].push_back(i);
    }
    for(int i=n;i>=1;i--)
    {
        solve(i);
    }
    for(int i=1;i<=n;i++)
    {
        if(ans[i]%k==0)
        {
            tmp++;
        }
    }
    if(n%k==0&&tmp==n/k)
    {
        printf("YE5\n");
    }
    else
    {
        printf("NO\n");
    }
    for(int i=1;i<=n;i++)
    {
        son[i].clear();
    }
    memset(ans,0,sizeof(ans));
}
}

```

G 杀戮尖塔

时间限制: 1000ms 内存限制: 65536kb

通过率: 32/33 (96.97%) 正确率: 32/105 (30.48%)

题目描述

Bellalabella 喜欢玩一个叫做杀戮尖塔的游戏。游戏一共有 n 层，每层有 k 个关卡，编号为 $1, 2, \dots, k$ 。玩家每通过一关，则可以进入下一层 k 个关卡中任意一个，当通过 n 层时则会抵达 Boss 房挑战最终 Boss。

Bellalabella 觉得这个游戏太简单了，因此她给自己设置了两个 debuff：

- 相邻两层不能挑战编号相同的关卡，即如果进入了第 i ($1 \leq i < n$) 层的第 j ($1 \leq j \leq k$) 关，则不可进入第 $i+1$ 层的第 j 关。
- 共有 m 层被设置了陷阱，层数分别为 $a_1, a_2, \dots, a_m$ ，被设置陷阱的层不可进入第 k 关，即仅可以进入第 $1, 2, \dots, k-1$ 关。

请在 debuff 的影响下，Bellalabella 一共有多少条不同的路线能够抵达 Boss 房。两条路线不同当且仅当存在一层两条路线所挑战的关卡不同。

由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行三个整数 n, m, k ($1 \leq n \leq 10^6, 0 \leq m \leq n, 2 \leq k \leq 10^9$)，含义同题目描述。

第二行 m 个互不相同的正整数 $a_1, a_2, \dots, a_m$ ($1 \leq a_i \leq n$)，含义同题目描述。

输出格式

一行一个非负整数，表示不同路线数对 998 244 353 取模后的结果。

输入样例 1

```
3 1 2
2
```

输出样例 1

```
1
```

```
#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 1000005
#define ls (tot << 1)
#define rs (tot << 1 | 1)
#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acosl(-1.0L);
const int mod = 998244353;
inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n, m, k;
bool vis[maxn];
int dp[maxn][2];
set<int> s;

void solve(){
    n = read(), m = read(), k = read();
    for(int i = 1; i <= m; i++) vis[read()] = 1;
    dp[1][0] = k - 1, dp[1][1] = !vis[1];
    for(int i = 2; i <= n; i++){
```

```

dp[i][0] = (111 * dp[i - 1][0] * (k - 2) + 111 * dp[i - 1][1] * (k - 1))
% mod;

dp[i][1] = dp[i - 1][0];
if(vis[i]) dp[i][1] = 0;
}

printf("%d", (dp[n][0] + dp[n][1]) % mod);
}

int main(){
    int t = 1, cnt = 1;
    while(t--){
        solve();
    }

    return 0;
}

```

H XIAO7 和数集 (困难版)

由于所有的喵币都分给「雪风」和「星瑶」了，猫娘「凉宫秋小七」对此很不高兴又哭又闹，XIAO7 只好告诉她，若你能答对以下问题，我就把上一题扔掉的喵币捡回来偷偷给你。

- $S(11523) = \{1, 2, 3, 5\}$
- $S(67654) = \{4, 5, 6, 7\}$

输入格式

对于每组数据，一行一个正整数 n ($1 \leq n < 10^{10^3}$)，含义同题目描述。

输出格式

输入样例

输出样例

```
#include <bits/stdc++.h>
#define maxn 1086

using namespace std;

const int p = 998244353;

int t, n;
char s[maxn];
int f[maxn][1 << 10][2][2];

void add(int &x, int y){
    x += y;
    if(x >= p) x -= p;
}
```

```

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        for(int i = 1; i <= n; i++) for(int j = 0; j < 1024; j++) for(int k = 0; k <=
1; k++) for(int l = 0; l <= 1; l++) f[i][j][k][l] = 0;
        f[0][0][1][0] = 1;
        for(int i = 1; i <= n; i++) for(int j = 0; j < 1024; j++) for(int k = 0; k <=
9; k++) for(int l = 0; l <= 1; l++){
            int jj = j | (1 << k), ll = l | (k > 0);
            if(!ll) jj = 0;
            add(f[i][jj][0][ll], f[i - 1][j][0][ll]);
            if(k <= s[i] - '0') add(f[i][jj][k == s[i] - '0'][ll], f[i - 1][j][1]
[ll]);
        }
        int ans = 0;
        for(int i = 1; i < 1024; i++) add(ans, (f[n][i][0][1] + f[n][i][1][1]) *
(1ll * f[n][i][0][1] + f[n][i][1][1] + p - 1) % p * (p + 1 >> 1) % p);
        printf("%d\n", ans);
    }
}

```

9

I XIAO7 和最长上升子序列

时间限制: 1000ms 内存限制: 204800kb

通过率: 2/4 (50.00%) 正确率: 2/11 (18.18%)

本题提交显示 Other Error 代表 Wrong Answer。

题目描述

XIAO7 在算法课上学会了如何计算最长上升子序列，但她觉得这对于猫猫来说实在是太简单了，于是她决定把问题升级一下来考核她的猫猫们。

具体来说，给定长度为 n 的序列 $f_1, f_2, \dots, f_n$ ，请你求解如下两个问题：

1. 构造一个 1 到 n 的排列 $a_1, a_2, \dots, a_n$ 满足以 a_i 结尾的最长上升子序列的长度为 f_i 。
2. 对于 $i = 1, 2, \dots, n$ ，求解所有满足第一个问题的排列中， a_i 的最大值。

对于一个序列 $a_1, a_2, \dots, a_n$ 来说，若序列 $x_1, x_2, \dots, x_m$ 满足 $1 \leq x_1 < x_2 < \dots < x_m \leq n$ 且 $a_{x_1} < a_{x_2} < \dots < a_{x_m}$ 则称 $a_{x_1}, a_{x_2}, \dots, a_{x_m}$ 为一个上升子序列。最长上升子序列是指所有上升子序列中长度最大的上升子序列。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个正整数 $f_1, f_2, \dots, f_n$ ($1 \leq f_i \leq i$)，含义同题目描述。

保证至少存在一个排列满足第一个问题的条件。

输出格式

第一行一个 1 到 n 的排列，表示第一个问题的答案。如果有多种可能的答案，你可以输出任意一种。

第二行 n 个正整数，第 i ($1 \leq i \leq n$) 个整数表示所有满足第一个问题的排列中， a_i 的最大值。

输入样例

```

3
1 1 2

```

输出样例

```

2 1 3
3 1 3

```

```

#include<iostream>
#include<cstdio>
#include<cmath>

```

```

#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
using namespace std;
typedef long long ll;
const int N = 1000005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int f[N], fa[N], layer[N], ans[N], sum[N], sz[N], a[N];
vector<int> son[N];
vector<int> layer_point[N];
void dfs(int x){
    sz[x] = 1;
    int m = son[x].size();
    for(int i = 0; i < m; i++){
        dfs(son[x][i]);
        sz[x] += sz[son[x][i]];
    }
    ans[x] = sum[f[x]] + sz[x] - 1;
    sum[f[x]] += sz[x];
}
int main(){
    int n = getint();

    for(int i = 1; i <= n; i++){
        f[i] = getint();

        for(int i = 1; i <= n; i++){
            fa[i] = layer[f[i] - 1];
            layer[f[i]] = i;
            layer_point[f[i]].push_back(i);
            son[fa[i]].push_back(i);
        }
        dfs(0);

        for(int i = 1, j = 0; i <= n; i++){
            int k = (j += layer_point[i].size());
            for(auto x : layer_point[i])
                a[x] = k--;
        }
        for(int i = 1; i <= n; i++)
            printf("%d%c", a[i], i == n ? 10 : 32);

        for(int i = 1; i <= n; i++)
            printf("%d%c", n - ans[i], i == n ? 10 : 32);

        exit(0);
    }
}

```

J 小水獭和体育馆（困难版）

时间限制：15000ms 内存限制：204800kb

通过率：1/3 (33.33%) 正确率：1/15 (6.67%)

题目描述



《与国同行》建校 70 周年庆祝晚会将于 10 月 24 日（预演、备播录制）、10 月 25 日（正式演出）进行两场演出，演出地点沙河体育馆，每晚 18:30 开始检票。

小水獭正在学习动态规划，但想去看看《与国同行》放松一下。

体育馆可以被抽象为一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，如果体育馆第 i 行第 j 列的座位有人，则 $a_{i,j} = 1$ ，否则 $a_{i,j} = 0$ 。

众所周知，为防控疫情，相邻座位不能同时有人。形式化地说， $(a_{i,j} = 1 \wedge a_{i',j'} = 1) \Rightarrow (|i - i'| + |j - j'| \neq 1)$ 。

此外，有 k 个额外限制某个座位必须有人或没人。形式化地说，第 i 个限制开闭为 $a_{x_i,y_i} = z_i$ 。

小水獭想知道，有多少种不同的矩阵所对应的体育馆满足疫情防控要求和 k 个额外限制，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 3$)，表示数据组数。

对于每组数据，第一行三个整数 n, m, k ($1 \leq nm \leq 625, 1 \leq k \leq 10$)，含义同题目描述。

接下来 k 行，第 i 行三个整数 x_i, y_i, z_i ($1 \leq x_i \leq n, 1 \leq y_i \leq m, 0 \leq z_i \leq 1$)，表示第 i 个限制的三个参数。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

输入样例

```
2
2 2 1
1 1 0
6 6 3
1 1 1
2 2 0
3 3 1
```

输出样例

```
5
231661
```

```
#include <bits/stdc++.h>
#define maxn 200086

using namespace std;

int t, n, m, k;
map<int, int> mp;
int cnt;

void dfs(int i, int j){
    if(i & (i << 1)) return;
    if(j == m){
        mp[i] = ++cnt;
        return;
    }
    dfs(i, j + 1), dfs(i | (1 << j), j + 1);
```

```

}

int a[maxn][30][2];
unsigned f[maxn], g[maxn];
int b[1005][1005];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%d", &n, &m, &k);
        bool tag = false, flag = false;
        if(n < m) swap(n, m), tag = true;
        for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) b[i][j] = -1;
        while(k--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            if(tag) swap(x, y);
            x--, y--;
            if(b[x][y] == -1) b[x][y] = z;
            else if(b[x][y] != z) flag = true;
        }
        if(flag){
            puts("0");
            continue;
        }
        mp.clear();
        cnt = 0, dfs(0, 0);
        for(auto it : mp){
            int s = it.first, x = it.second;
            for(int i = 0; i < m; i++){
                a[x][i][0] = mp[s & ~(1 << i)];
                if((i == 0 || !(s & (1 << (i - 1)))) && !(s & (1 << i))){
                    a[x][i][1] = mp[(s | (1 << i)) & ~(1 << (i + 1))];
                }else{
                    a[x][i][1] = 0;
                }
            }
        }
        for(int i = 1; i <= cnt; i++) f[i] = 0;
        f[mp[0]] = 1;
        for(int i = 0; i < n; i++) for(int j = 0; j < m; j++){
            for(int k = 1; k <= cnt; k++){
                if(b[i][j] != 1) g[a[k][j][0]] += f[k];
                if(b[i][j] != 0 && a[k][j][1]) g[a[k][j][1]] += f[k];
            }
            for(int k = 1; k <= cnt; k++) f[k] = g[k], g[k] = 0;
        }
        unsigned ans = 0;
        for(int i = 1; i <= cnt; i++) ans += f[i];
        printf("%u\n", ans);
    }
}

```

A 购物

时间限制: 1000ms 内存限制: 65536kb

通过率: 190/195 (97.44%) 正确率: 190/1064 (17.86%)

题目描述

虽然 Zhoues 习惯在学校和钢厂之间来回奔波，但是每到周末还是需要放松一下的，于是 Zhoues 和小水獭打算周末一起去购物放松心情。

Zhoues 手中有一份地图，地图上每一个购物中心都有一个唯一的编号，而且地图上也会标注出每两个购物中心之间的距离。但是因为最近车流量大，地图上的双行道都变成单行道以缓解交通压力。小水獭想知道给定任意两个购物中心，二者之间的最短路径长度是多少。

形式化来说，给定 n 个点 m 条的有向图，你需要处理 q 次询问，第 i 次询问的内容为点 x_i 到点 y_i 的最短路径长度（若不可达则输出 `No Solution`）。

输入格式

第一行三个正整数 n, m, q ($3 \leq n \leq 500, 1 \leq m, q \leq 2.5 \times 10^5$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^7$)，表示第 i 条边 $u_i \rightarrow v_i$ 的权值为 w_i 。

接下来 q 行，每行两个正整数 x_i, y_i ($1 \leq x_i, y_i \leq n$)，含义同题目描述。

图中有可能存在重边和自环。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行表示第 i 次询问的答案：

- 如果点 x_i 可以到点 y_i ，则输出最短路径长度。
- 否则，输出 `No Solution`。

输入样例

```
3 3 2
1 2 3
2 3 1
3 1 2
1 2
1 3
```

输出样例

```
3
4
```

```
#include<bits/stdc++.h>
const int MN=500+5;
typedef long long ll;
using namespace std;
ll mat[MN][MN];
ll inf = 0x3f3f3f3f3f3f3f3f;
int m,n,p;
int main()
{
    memset(mat,0x3f,sizeof mat);
    scanf("%d%d%d",&n,&m,&p);
    ll w;
    for(int i=1,u,v;i<=m;i++){
        scanf("%d%d%lld",&u,&v,&w);
        mat[u][v]=min(mat[u][v],w); //00±ß
    }
    for(int i=1;i<=n;i++){
        mat[i][i]=0; // ×0»·
    }
    for(int k=1;k<=n;k++)
        for(int i=1;i<=n;i++)
            for(int j=1;j<=n;j++)
                mat[i][j]=min(mat[i][j],mat[i][k]+mat[k][j]);

    for(int i=0,u,v;i<p;i++){
        scanf("%d%d",&u,&v);
```

```

        if(mat[u][v]<inf) printf("%lld\n",mat[u][v]);
        else puts("No Solution");
    }
    return 0;
}

```

2

B 铺光缆

时间限制：1000ms 内存限制：65536kb

通过率：179/195 (91.79%) 正确率：179/1077 (16.62%)

题目描述

罗川老师在担任 21 级的「算法分析与设计」老师的同时，也担任 20 级的「计算机网络」老师。罗老师有一次在讲解计算机网络中物理层的时候，提到了北航光缆铺设问题，问题大致如下：

- 北航主要办公科研楼有新主楼、逸夫楼、如心楼、办公楼、图书馆、主楼、一号楼等等。北航网络中心计划要给相关建筑物间铺设光缆进行网络连通。给定建筑物之间的距离，计算铺设光缆用料最少是多少。

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的。于是他希望你帮忙计算一下铺设光缆用料最少是多少。

形式化地说，给定一个 n 个点 m 条边的带权无向连通图，请你求解最小生成树的边权之和。

输入格式

第一行三个正整数 n, m ($1 \leq n \leq 2 \times 10^5$, $n - 1 \leq m \leq 5 \times 10^5$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $1 \leq w_i \leq 10^9$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环，保证图是连通的。

输出格式

输出一行一个正整数，表示最小生成树的边权之和。

输入样例

```

3 3
1 2 1
1 3 2
2 3 1

```

输出样例

```

2

```

```

#include <bits/stdc++.h>
#define mp make_pair
#define ms(a,b) memset(a,b,sizeof(a))
#define maxn 1000007
typedef long long LL;
using namespace std;

struct edge
{
    LL u,v;
    LL w;
};
struct edge edges[60000100];
LL i;
LL Father[1010000];
LL cnt;
long long res;
void initFather(LL vertexNum)
{
    LL i;
    for(i=1;i<=vertexNum;++i)
    {
        Father[i]=i;
    }
}

```

```

    }
}
LL getFather(LL x)
{
    return Father[x]==x?(Father[x]=getFather(Father[x])):
}
void kruskal(LL vertexNum,LL edgeNum)
{
    LL p,q;
    cnt=0,res=0;

    for(i=0;i<edgeNum;++i)
    {
        p=getFather(edges[i].u);
        q=getFather(edges[i].v);
        if(p!=q)
        {
            Father[p]=q;
            res+=edges[i].w;
            cnt++;
        }
        if(cnt==vertexNum-1)
        {
            break;
        }
    }
}

}
int cmp(const void*p1,const void*p2)
{
    struct edge *a=(struct edge*)p1;
    struct edge *b=(struct edge*)p2;
    return a->w-b->w;
}
int main()
{
    LL n,m;
    scanf("%lld%lld",&n,&m);
    initFather(n);
    LL i;
    for(i=0;i<m;++i)
        scanf("%lld %lld %lld",&edges[i].u,&edges[i].v,&edges[i].w);
    qsort(edges,m,sizeof(struct edge),cmp);
    kruskal(n,m);
    printf("%lld\n",res);
}

```

C 上班 II

时间限制: 2000ms 内存限制: 204800kb

通过率: 173/190 (91.05%) 正确率: 173/1226 (14.11%)

题目描述

观赛完运动会之后, Zhoues 需要立马回去钢厂上班, 由于从家到钢厂的路径有很多条, 他希望选择一条最短的!

但是现在 Zhoues 很急, 所以希望可以计算的更快一点!

形式化来说, 给一个 n 个点 m 条边的无向图, 求点 s 到点 t 的最短路长度, 题目保证至少存在一条点 s 到点 t 的路径。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 10^6, 1 \leq m \leq 1.6 \times 10^6, 1 \leq s, t \leq n$), 含义同题目描述。

接下来 m 行, 每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^7$), 表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环, 保证至少存在一条点 s 到点 t 的路径。

输出格式

输出一行一个非负整数, 表示点 s 到点 t 的最短路长度。

输入样例

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

输出样例

```
7
```

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0', c=getchar();
    return x*w;
}

const ll N=10000000+10, M=15000000+10;

ll n,m,s,t;
struct edge { ll v,w,nxt; } e[M];
ll head[N];
void addEdge(ll u,ll v,ll w) {
    static ll cnt=0;
    e[++cnt]=(edge){v,w,head[u]}, head[u]=cnt;
}

priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
ll dis[N];
void dijkstra(ll s) {
    memset(dis,0x3f,sizeof(dis));
    dis[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
```

```

        if (d!=dis[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
        }
    }
}

int main() {
    n=read(),m=read(),s=read(),t=read();
    for (ll i=1;i<=m;++i) {
        ll u=read(),v=read(),w=read();
        addEdge(u,v,w);
        addEdge(v,u,w);
    }
    dijkstra(s);
    printf("%lld",dis[t]);
    return 0;
}

```

4

D 小水獭和负环

时间限制: 1000ms 内存限制: 65536kb

通过率: 172/179 (96.09%) 正确率: 172/781 (22.02%)

题目描述



观赛完运动会之后，Zhoues 需要立马回去钢条厂上班，由于北航到钢条厂的路径有很多条，他希望选择一条最短的！

但是，图中可能存在负环，Zhoues 不想无穷循环下去，因此他找到小水獭帮忙，希望小水獭可以帮它判断是否有负环。

形式化来说，给一个 n 个点 m 条边的带权有向图，如果图中存在负环，你需要告知 Zhoues，否则你需要求解从点 1 到所有点的最短路（如果不可达，认为最短路长度是 10^9 ）。

负环是指一条边权之和为负数的回路。

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 2 \times 10^3, 1 \leq m \leq 6 \times 10^3$)，含义同题目描述。

接下来 m 行，每行三个整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, -10^4 \leq w_i \leq 10^4$)，表示第 i 条边 $u_i \rightarrow v_i$ 的权值为 w_i 。

图中有可能存在重边和自环。

输出格式

对于每组数据，输出一行：

- 如果图中存在负环，输出 `boo how`。
- 否则输出 n 个整数，表示从点 1 到点 $1, 2, \dots, n$ 的最短路长度（如果不可达，认为最短路长度是 10^9 ）。

输入样例

输入样例

```
2
4 5
1 2 -1
2 3 0
3 4 0
4 1 1
4 1 0
4 1
1 2 1
```

输出样例

```
boo how
0 1 1000000000 1000000000
```

```
#include <bits/stdc++.h>
#define maxn 2086

using namespace std;

int t, n, m;
vector<pair<int, int> > v[maxn];
int d[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) v[i].clear(), d[i] = 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            v[x].push_back({y, z});
        }
        d[1] = 0;
        bool tag = false;
        for(int j = 1; j <= n; j++){
            for(int i = 1; i <= n; i++){
                for(auto it : v[i]){
                    int to = it.first, co = it.second;
                    if(d[to] > d[i] + co && j == n) tag = true;
                    d[to] = min(d[to], d[i] + co);
                }
            }
        }
        if(tag) puts("boo how");
        else{
            for(int i = 1; i <= n; i++) printf("%d ", d[i]); puts("");
        }
    }
}
```

E 小水獭和哈密顿路径

时间限制: 1000ms 内存限制: 65536kb

通过率: 156/167 (93.41%) 正确率: 156/629 (24.80%)

题目描述



众所周知，判断一个图是否有哈密顿路径是 **NP** 的。
但小水獭发现，如果一个图是 DAG，那么这个问题是 **P** 的，你能帮它实现一下嘛？
形式化来说，给一个 n 个点 m 条边的有向无环图，请你判断图中是否存在一条哈密顿路径。
哈密顿路径是指一条经过每个点恰好一次的路径。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。
对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。
接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 $u_i \rightarrow v_i$ 。
保证图是有向无环图，且没有重边。

输出格式

对于每组数据，输出一行：

- 如果图中存在哈密顿路径，输出 **yy**。
- 否则输出 **nn**。

输入样例

```
2
4 4
1 2
1 3
2 3
3 4
4 3
1 2
1 3
1 4
```

输出样例

```
yy
nn
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<int> v[maxn];
int d[maxn];

int dfs(int i){
    if(d[i] != -1) return d[i];
    d[i] = 0;
    for(auto to : v[i]){
        d[i] = max(d[i], dfs(to));
    }
    return ++d[i];
}

int main(){
    scanf("%d", &t);
    while(t--){
```

```

scanf("%d%d", &n, &m);
for(int i = 1; i <= n; i++) v[i].clear(), d[i] = -1;
while(m--){
    int x, y;
    scanf("%d%d", &x, &y);
    v[x].push_back(y);
}
int ans = 0;
for(int i = 1; i <= n; i++) ans = max(ans, dfs(i));
puts(ans == n ? "yy" : "nn");
}
}

```

6

F 这下不得不进厂了

时间限制: 1000ms 内存限制: 65536kb

通过率: 157/167 (94.01%) 正确率: 157/752 (20.88%)

题目描述

这几周 Zhoues 一直在钢厂上班，the_ignorant 却啥也没干，他感到了强烈的危机感，他急了，这下不得不进厂了！

the_ignorant 进了自来水厂，可他刚入职就碰到了一个难题。在水管分布图中，有 n 个阀门和 m 个单向管道，每个单向管道连接两个阀门，并且有各自的容量限制（即经过的水流不能超过该管道的最大容量）。the_ignorant 想知道，在阀门 s 放水时，阀门 t 最大能接收到多大的水流。

形式化地说，给定 n 个点 m 条边的有向图，每条边有最大容量，求从点 s 到点 t 的最大流。

输入格式

第一行一个正整数 T ($1 \leq T \leq 10$)，表示数据组数。

对于每组数据，第一行四个正整数 n, m, s, t ($1 \leq n \leq 100, 1 \leq m \leq 5 \times 10^3, 1 \leq s, t \leq n$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 0 \leq w_i < 2^{31}$)，表示第 i 条有向边 $u_i \rightarrow v_i$ 的最大容量为 w_i 。

图中有可能存在重边和自环。

输出格式

对于每组数据，输出一行一个非负整数，表示点 s 到点 t 的最大流。

输出格式

对于每组数据，输出一行一个非负整数，表示点 s 到点 t 的最大流。

输入样例

```

1
7 14 1 7
1 2 5
1 3 6
1 4 5
2 3 2
2 5 3
3 2 2
3 4 3
3 5 3
3 6 7
4 6 5
5 6 1
6 5 1
5 7 8
6 7 7

```

输出样例

```

14

```

```

#include <bits/stdc++.h>
#define mp make_pair
#define ms(a,b) memset(a,b,sizeof(a))
#define maxn 1000007
typedef long long LL;
using namespace std;
inline LL read() {
    LL x=0,w=1;
    char ch=0;

```

```

while(ch<'0' || ch>'9') {
    if(ch=='-') w=-1;
    ch=getchar();
}
while(ch>='0' && ch<='9') x=(x<<3)+(x<<1)+ch-'0',ch=getchar();
return x*w;
}

inline void write(LL x) {
    if(x < 0)putchar('-'),x=-x;
    if (x > 9)write(x / 10);
    putchar(x % 10 + 48);
}

const LL inf=0x3f3f3f3f;
const LL N=2000005;
const LL M=2000005;
LL cnt,tot;
LL head[N],d[N],ans[N];
bool vis[N];
struct Edge {
    LL v,next;
    LL cap,flow;
} E[M<<1]; //双边

void init() { //初始化
    memset(head,-1,sizeof(head));
    cnt=0;
}

void add(LL u,LL v,LL c) {
    E[cnt].v=v;
    E[cnt].cap=c;
    E[cnt].flow=0;
    E[cnt].next=head[u];
    head[u]=cnt++;
}

void adde(LL u,LL v,LL c) {
    add(u,v,c);
    add(v,u,0);
}

bool bfs(LL s,LL t) { //分层
    memset(d,0,sizeof(d));
    queue<LL>q;
    d[s]=1;
    q.push(s);
    while(!q.empty()) {
        LL u=q.front();
        q.pop();
        for(LL i=head[u]; ~i; i=E[i].next) {
            LL v=E[i].v;
            if(!d[v]&&E[i].cap>E[i].flow) {
                d[v]=d[u]+1;
                q.push(v);
                if(v==t) return 1;
            }
        }
    }
}

```

```

    }
}
return 0;
}

LL dfs(LL u, LL flow, LL t) { //在分层的基础上dfs
    if(u==t) return flow;
    LL rest=flow;
    for(LL i=head[u]; ~i&&rest; i=E[i].next) {
        LL v=E[i].v;
        if(d[v]==d[u]+1&&E[i].cap>E[i].flow) {
            LL k=dfs(v, min(rest, E[i].cap-E[i].flow), t);
            if(!k) d[v]=0;
            E[i].flow+=k;
            E[i^1].flow-=k;
            rest-=k;
        }
    }
    return flow-rest;
}

LL Dinic(LL s, LL t) {
    LL maxflow=0;
    while(bfs(s, t)) {
        maxflow+=dfs(s, inf, t);
    }
    return maxflow;
}

LL n, m, s, t, T;
int main() {
    T = read();
    while(T--) {
        n = read(), m = read(), s = read(), t = read();
        init();
        for(LL i = 0; i < m; i++) {
            LL x = read();
            LL y = read();
            LL w = read();
            add(x, y, w);
            add(y, x, 0);
        }
        write(Dinic(s, t));
        putchar('\n');
    }

    return 0;
}

```

G 旅行计划

时间限制: 1500ms 内存限制: 65536kb

通过率: 24/36 (66.67%) 正确率: 24/195 (12.31%)

题目描述

教练们很累，写不动题目背景力！

形式化地说，给定一张 n 个点 m 条边的带权无向连通图。你需要处理 q 次询问，第 i 次询问的内容为从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度是多少。

输入格式

第一行三个整数 n, m, q ($1 \leq n \leq 100, n - 1 \leq m \leq \frac{n(n-1)}{2}, 1 \leq q \leq 10^6$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i < v_i \leq n, 1 \leq w_i \leq 10^9$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

接下来 q 行，每行三个正整数 s_i, t_i, k_i ($1 \leq s_i, t_i \leq n, 1 \leq k_i \leq m$)，含义同题目描述。

保证图连通且没有重边和自环。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 如果不存在一条从 s_i 到 t_i 且经过边数不超过 k_i 的路径，输出 -1 。
- 否则输出从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 如果不存在一条从 s_i 到 t_i 且经过边数不超过 k_i 的路径，输出 -1 。
- 否则输出从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度。

输入样例

```
4 4 4
1 2 10
2 3 20
2 4 1000
1 4 30
1 3 1
1 3 2
2 4 1
2 4 4
```

输出样例

```
-1
30
1000
40
```

```
#include <stdio>
#include <string.h>
const int N = 105;
struct edge {
    int u, v;
    long long w;
};
edge e[N * N];
int cnt = 0;
long long dis[N][N][N];
int n, m, q, u, v, w, s, t, k;
void BF(int s) {
    for (int i = 0; i <= n - 1; i++) {
        dis[i][s][s] = 0ll;
    }
    for (int i = 1; i <= n - 1; i++) {
        for (int j = 1; j <= n; j++) {
            dis[i][s][j] = dis[i - 1][s][j];
        }
        for (int j = 1; j <= cnt; j++) {
            if (dis[i][s][e[j].v] > dis[i - 1][s][e[j].u] + e[j].w) {
```

```

        dis[i][s][e[j].v] = dis[i - 1][s][e[j].u] + e[j].w;
    }
    if (dis[i][s][e[j].u] > dis[i - 1][s][e[j].v] + e[j].w) {
        dis[i][s][e[j].u] = dis[i - 1][s][e[j].v] + e[j].w;
    }
}
}

int main() {
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= m; i++) {
        scanf("%d%d%d", &u, &v, &w);
        e[++cnt].u = u;
        e[cnt].v = v;
        e[cnt].w = (long long)w;
    }
    for (int i = 0; i <= n - 1; i++) {
        for (int j = 1; j <= n; j++) {
            for (int k = 1; k <= n; k++) {
                dis[i][j][k] = 111 << 60;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        BF(i);
    }
    for (int i = 1; i <= q; i++) {
        scanf("%d%d%d", &s, &t, &k);
        if (k >= n) {
            if (dis[n - 1][s][t] == 111 << 60) {
                printf("-1\n");
            } else {
                printf("%11d\n", dis[n - 1][s][t]);
            }
        } else {
            if (dis[k][s][t] == 111 << 60) {
                printf("-1\n");
            } else {
                printf("%11d\n", dis[k][s][t]);
            }
        }
    }
    return 0;
}

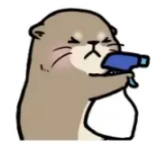
```

H 小水獭和补图

时间限制：1000ms 内存限制：65536kb

通过率：20/27 (74.07%) 正确率：20/133 (15.04%)

题目描述



小水獭正在研究补图的连通性问题。

具体来说，对于无重边自环的无向图 G ，设其节点为 $1, 2, \dots, n$ ，边集为 E ；定义它的补图为 G' ，其中 G' 的点集和 G 相同， G' 的边集为 $\{(u, v) \mid 1 \leq u, v \leq n, u \neq v, (u, v) \notin E\}$ 。

给定 n 个点 m 条边且无重边自环的无向图，小水獭想知道其补图中每个点所在连通块的大小。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 (u_i, v_i) 。

保证图中没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示补图中点 $1, 2, \dots, n$ 所在连通块的大小。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 (u_i, v_i) 。

保证图中没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示补图中点 $1, 2, \dots, n$ 所在连通块的大小。

输入样例

```
2
3 2
1 2
1 3
4 6
1 2
1 3
1 4
2 3
2 4
3 4
```

输出样例

```
1 2 2
1 1 1 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<int> v[maxn];
int fa[maxn], siz[maxn];

int find(int x){
    return x == fa[x] ? x : fa[x] = find(fa[x]);
}

void merge(int x, int y){
    x = find(x), y = find(y);
    if(x == y) return;
```

```

        if(siz[x] > siz[y]) swap(x, y);
        fa[x] = y, siz[y] += siz[x];
    }

    bool vis[maxn];

    void cal(int i, int j){
        for(auto to : v[i]) vis[to] = j;
    }

    int main(){
        scanf("%d", &t);
        while(t--){
            scanf("%d%d", &n, &m);
            for(int i = 1; i <= n; i++) v[i].clear(), fa[i] = i, siz[i] = 1;
            while(m--){
                int x, y;
                scanf("%d%d", &x, &y);
                v[x].push_back(y), v[y].push_back(x);
            }
            int x = 1;
            for(int i = 2; i <= n; i++) if(v[i].size() < v[x].size()) x = i;
            cal(x, 1);
            for(int i = 1; i <= n; i++) if(!vis[i]) merge(x, i);
            cal(x, 0);
            for(auto to : v[x]){
                cal(to, 1);
                for(int i = 1; i <= n; i++) if(!vis[i]) merge(to, i);
                cal(to, 0);
            }
            for(int i = 1; i <= n; i++) printf("%d ", siz[find(i)]); puts("");
        }
    }
}

```

9

I XIAO7 不想努力了

时间限制: 1000ms 内存限制: 131072kb

通过率: 8/12 (66.67%) 正确率: 8/54 (14.81%)

题目描述

XIAO7: 「我不想努力了, 我今天创造的价值已经够多了。」

天上传来一个声音: 「如果你能答对以下问题, 你就可以不努力了。」

具体来说, 给定一个 n 个点 m 条边的有向图, 请你给所有边分组, 使得每一组内边的编号是连续的, 并且不存在一条从点 1 到点 n 的路径满足路径上所有边属于同一组。你需要求解最少需要分多少组。

请你帮帮可爱的 XIAO7!

输入格式

第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 10^6$), 含义同题目描述。

接下来 m 行, 每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$), 表示第 i 条边为 $u_i \rightarrow v_i$ 。

图中可能有重边, 但没有自环, 且不存在边 $1 \rightarrow n$ 。

输出格式

一行一个正整数, 表示最少的分组数。

输出格式

一行一个正整数, 表示最少的分组数。

输入样例

```
3 5
2 3
3 2
2 1
1 2
3 2
```

输出样例

```
2
```

样例解释

一种分组数最少的方案为 $[1, 2, 3], [4, 5]$ 。

仅考虑第一组的边得到的图如下:

仅考虑第二组的边得到的图如下:

均不存在从点 1 到点 n 的路径。

1 2 ... n

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 100005;
const int M = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int cnt, fir[N], tar[M], nxt[M];
int vis[N], U[M], V[M];
void link(int a, int b) {
    tar[++ cnt] = b, nxt[cnt] = fir[a], fir[a] = cnt;
}
void dfs(int x) {
    for(int i = fir[x]; i; i = nxt[i]) {
        if(vis[tar[i]] == vis[x])
            continue ;
        vis[tar[i]] = vis[x], dfs(tar[i]);
    }
}
int main(){
    int n = getint(), m = getint();
```

```

int group = 1, last = 0;
vis[1] = group;
for(int i = 1; i <= m; i++) {
    int x = getint(), y = getint();
    link(x, y), U[i] = x, V[i] = y;
    if(vis[x] == group)
        dfs(x);
    if(vis[n] == group) {
        vis[1] = ++ group;
        for(int j = i; j >= last; j--)
            fir[U[j]] = 0;
        last = i, link(x, y);
        if(x == 1) vis[y] = group;
    }
}
cout << group << endl;
}

```

10

A B C D E F G H I J

小水獭和最小环

时间限制: 8000ms 内存限制: 65536kb

通过率: 2/9 (22.22%) 正确率: 2/76 (2.63%)

题目描述



小水獭正在研究无向图的最小环问题。

具体来说，给定 n 个点 m 条边的带权无向图，请你求出对于点 $i = 1, 2, \dots, n$ 来说，包含点 i 的最小环大小（如果不存在包含点 i 的环，认为最小环大小为 10^9 ）。

环是指点数至少为 3 且不重复经过相同点的回路，最小环是指边权之和最小的环。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 500, 1 \leq m \leq \frac{n(n-1)}{2}$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^6$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证给出的图没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示对于点 $i = 1, 2, \dots, n$ 来说，包含点 i 的最小环大小（如果不存在包含点 i 的环，认为最小环大小为 10^9 ）。

输入样例

```

2
4 4
1 2 5
1 3 1
1 4 1
2 4 1
3 3
1 2 1
2 3 2
3 1 3

```

输出样例

```

7 7 1000000000 7
6 6 6

```

```

#include <bits/stdc++.h>
#define maxn 505

using namespace std;

int t;
int n, m;
int a[maxn][maxn];
int d[maxn][maxn];
vector<int> v[maxn];
int fa[maxn];

void dfs(int i, int x){
    fa[i] = x;
    for(auto to : v[i]) dfs(to, x);
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) a[i][j] = d[i][j] = i
        == j ? 0 : 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            a[x][y] = a[y][x] = d[x][y] = d[y][x] = z;
        }
        for(int k = 1; k <= n; k++) for(int i = 1; i <= n; i++) for(int j = 1; j <=
n; j++) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++) v[j].clear();
            for(int j = 1; j <= n; j++) if(i ^ j){
                for(int k = 1; k <= n; k++) if(k ^ j) if(d[i][k] + a[k][j] == d[i]
[j]){
                    v[k].push_back(j);
                    break;
                }
            }
            for(auto to : v[i]) dfs(to, to);
            int ans = 1e9;
            for(int j = 1; j <= n; j++) if(i ^ j){
                if(fa[j] ^ j) ans = min(ans, a[i][j] + d[i][j]);
            }
            for(int j = 1; j <= n; j++) if(i ^ j) for(int k = j + 1; k <= n; k++)
if(i ^ k){
                if(fa[j] ^ fa[k]) ans = min(1ll * ans, 1ll * a[j][k] + d[i][j] +
d[i][k]);
            }
            printf("%d ", ans);
        }
        puts("");
    }
}

```

E5

1

A 小水獭和高精度乘法

时间限制: 3000ms 内存限制: 65536kb

通过率: 171/194 (88.14%) 正确率: 171/1180 (14.49%)

题目描述



小水獭想请你编写一个高精度乘法计算器，给定正整数 A 和 B ，请你输出 $C = AB$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 A, B ($1 \leq A, B < 10^{10^5}$)，含义如题目所示。

输出格式

对于每组数据，输出一行一个正整数 $C = AB$ 。

输入样例

```
3
1 1
4 5
1 4
```

输出样例

```
1
20
4
```

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

const double pi = acos(-1);

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
```

```

        return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
    }

    int pos[maxn];

    void FFT(Complex *a, int lim, int type){
        for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
        for(int mid = 1; mid < lim; mid <= 1){
            Complex wn(cos(pi / mid), type * sin(pi / mid));
            for(int i = mid < 1, j = 0; j < lim; j += i){
                Complex w(1, 0);
                for(int k = 0; k < mid; k++, w = w * wn){
                    Complex x = a[j + k], y = w * a[j + k + mid];
                    a[j + k] = x + y;
                    a[j + k + mid] = x - y;
                }
            }
        }
        if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
    }

    int t;
    Complex a[maxn], b[maxn];
    char s[maxn];
    int n, m;
    int ans[maxn];

    int main(){
        scanf("%d", &t);
        while(t--){
            scanf("%s", s);
            n = strlen(s);
            for(int i = 0; i < n; i++) a[n - i - 1].x = s[i] - '0';
            scanf("%s", s);
            m = strlen(s);
            for(int i = 0; i < m; i++) b[m - i - 1].x = s[i] - '0';
            int lim = 1;
            while(lim <= n + m - 2) lim <= 1;
            for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
            FFT(a, lim, 1), FFT(b, lim, 1);
            for(int i = 0; i < lim; i++) a[i] = a[i] * b[i];
            FFT(a, lim, -1);
            for(int i = 0; i < maxn; i++) ans[i] = 0;
            for(int i = 0; i < lim; i++){
                ans[i] = (int)(a[i].x + 0.5);
                a[i] = b[i] = {0, 0};
            }
            for(int i = 0; i + 1 < maxn; i++){
                ans[i + 1] += ans[i] / 10;
                ans[i] %= 10;
            }
            bool tag = false;
            for(int i = maxn - 1; ~i; i--){
                if(ans[i]) tag = true;
                if(!ans[i] && !tag) continue;

```

```

        printf("%d", ans[i]);
    }
    puts("");
}
}

```

2

B 旋转卡壳怎么读

时间限制: 1000ms 内存限制: 65536kb

通过率: 147/170 (86.47%) 正确率: 147/1051 (13.99%)

题目描述

教给司发现二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 。定义两点 (x_i, y_i) 和 (x_j, y_j) 的欧几里得距离为 $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。教给司要求同学们求出任意两点间最大欧几里得距离，即：

$$\max_{1 \leq i < j \leq n} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

输入格式

第一行一个正整数 n ($2 \leq n \leq 10^5$)，表示点的数量。

接下来 n 行，每行两个至多六位小数的浮点数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 5 \times 10^5$)，表示第 i 个点的坐标。

输出格式

输出一行一个浮点数，保留六位小数，表示任意两点间最大欧几里得距离。

输入样例

```

4
0 1
1 0
1 1
-1 -1

```

输出样例

```

2.828427

```

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#include<stdbool.h>
#include<vector>
#include<algorithm>
#include<iostream>
using namespace std;
typedef long long ll;
const int N=200005;
const double eps=1e-7;
int getint(){
    char c;int flag=0,num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag?-num:num;
}
double getdb(){double x;scanf("%lf",&x);return x;}
inline int sgn(double x){return (x>eps)-(x<-eps);}
inline double sqr(double x){return x*x;}
struct Point{
    double x,y;
    Point operator + (const Point&b)const{
        return (Point){x+b.x,y+b.y};
    }
}

```

```

    }
    Point operator - (const Point&b)const{
        return (Point){x-b.x,y-b.y};
    }
    double operator * (const Point&b)const{
        return x*b.x+y*b.y;
    }
    double operator ^ (const Point&b)const{
        return x*b.y-y*b.x;
    }
    double dis(Point b){
        return sqrt(sqr(x-b.x)+sqr(y-b.y));
    }
};

vector<Point> p,ans;
bool cmpx(Point a,Point b){
    return sgn(a.x-b.x)==0 ? a.y<b.y : a.x<b.x;
}

vector<Point> ConvexHull(vector<Point> A,int flag){
    int n=A.size(),now=-1;
    vector<Point> ans(n<<1);
    sort(A.begin(),A.end(),cmpx);
    for(int i=0;i<n;i++){
        while(now>0 && sgn((ans[now]-ans[now-1])^(A[i]-ans[now-1]))<flag) now--;
        ans[++now]=A[i];
    }
    int pre=now;
    for(int i=n-2;i>=0;i--){
        while(now>pre && sgn((ans[now]-ans[now-1])^(A[i]-ans[now-1]))<flag) now--;
        ans[++now]=A[i];
    }
    ans.resize(now);return ans;
}

double convexDiameter(vector<Point> A){
    int n=A.size(),now=0;
    double ans=0;
    for(int i=0;i<n;i++){
        now=max(now,i);
        while(1){
            double k1=A[i].dis(A[now%n]),k2=A[i].dis(A[(now+1)%n]);
            ans=max(ans,max(k1,k2));
            if(k2>k1) now++;else break ;
        }
    }
    return ans;
}

int main(){
    int n=getint();
    for(int i=0;i<n;i++){
        double x=getdb(),y=getdb();
        p.push_back((Point){x,y});
    }
    ans=ConvexHull(p,1);
    printf("%.6f\n",convexDiameter(ans));
    return 0;
}

```

}

3



去做题

C 约会大作战-青春版

时间限制: 1000ms 内存限制: 65536kb

通过率: 88/98 (89.80%) 正确率: 88/251 (35.06%)

题目描述

学校正在举办联谊活动! 目前报名的有 n 个男生和 n 个女生。其中第 i 个男生魅力值为 a_i ，只喜欢魅力值不小于 p_i 的女生; 第 i 个女生魅力值为 b_i ，只喜欢魅力值不小于 q_i 的男生。

男生与女生能约会当且仅当两人相互喜欢。作为一个没有钞能力的主办方，the_ignorant 只能希望让尽可能多的人能约会成功，那么最多有多少对情侣能约会成功呢?

输入格式

第一行一个正整数 n ($1 \leq n \leq 400$)，表示男生和女生的数量。

第二行 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)，表示男生的魅力值。

第三行 n 个非负整数 $p_1, p_2, \dots, p_n$ ($0 \leq p_i \leq 10^9$)，表示男生喜欢的女生魅力值最小值。

第四行 n 个非负整数 $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq 10^9$)，表示女生的魅力值。

第五行 n 个非负整数 $q_1, q_2, \dots, q_n$ ($0 \leq q_i \leq 10^9$)，表示女生喜欢的男生魅力值最小值。

输出格式

一行一个非负整数，表示最多能有几对情侣约会成功。

输入样例

```
5
3 10 2 1 5
9 7 2 10 1
1 6 10 8 0
9 4 4 1 8
```

输出样例

```
3
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 500;
char s[N];
int a[N],p[N],b[N],q[N];
bool mat[N][N];
bool vis[N];
int girl[N];
int n;
bool find(int x)
{
    for(int i=1;i<=n;i++)
    {
        if(mat[x][i]&&!vis[i])
        {
            vis[i]=true;
            if(!girl[i]||find(girl[i]))
            {
                girl[i]=x;
                return true;
            }
        }
    }
    return false;
}
int main()
```

```

{

    int t=1;
    while(t-->0)
    {
        scanf("%d",&n);
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&a[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&p[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&b[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&q[i]);
        }
        for(int i=1;i<=n;i++)
        {
            for(int j=1;j<=n;j++)
            {
                if(b[j]>=p[i]&& a[i]>=q[j])
                {
                    mat[i][j]=true;
                }
            }
        }
        int ans=0;
        for(int i=1;i<=n;i++)
        {
            memset(vis,0,sizeof(vis));
            if(find(i))
            {
                ans++;
            }
        }
        printf("%d",ans);
    }
}

```

D 直直的多边形面积

时间限制: 1000ms 内存限制: 65536kb

通过率: 130/141 (92.28%) 正确率: 130/495 (26.26%)

题目描述

Zhoues 最近在研究直直的多边形, 直直的多边形是所有边均平行于两条坐标轴之一的简单多边形。

现给出一个直直的多边形, Zhoues 想知道这个直直的多边形的面积。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$), 表示数据组数。

对于每组数据, 第一行一个正整数 n ($4 \leq n \leq 4 \times 10^4$), 表示直直的多边形的顶点数。

接下来 n 行, 每行两个整数 x_i, y_i ($-5 \times 10^4 \leq x_i, y_i \leq 5 \times 10^4$), 表示直直的多边形一个顶点的坐标。保证顶点坐标按顺时针方向给出, 且所有边均平行于两条坐标轴之一。

输出格式

对于每组数据, 输出一行一个非负整数, 表示直直的多边形的面积。

输入样例

```
1
4
-200 200
200 200
200 -200
-200 -200
```

输出样例

```
160000
```

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define inf 1e100
#define eps 1e-8
//用于浮点数正负判断, 根据题目精度修改
const double pi = acos(-1.0); //圆周率
long long sgn(long long x) {
    if(fabs(x)<eps)return 0;
    if(x<0)return -1;
    return 1;
} //判断浮点数正负
long long sqr(long long x) {
    return x*x;    //距离等运算涉及大量平方, 简便
}
//使用Point时注意部分函数是返回新Point而非修改本身值
struct Point {
    long long x,y;
    /*构造函数*/
    Point() {}
    Point(long long xx,long long yy) {
        x=xx;
        y=yy;
    }
    /*重载一些点的基础运算符*/
    bool operator == (Point b)const {
        return sgn(x-b.x) == 0 && sgn(y-b.y) == 0;
    }
    bool operator < (Point b)const {
        return sgn(x-b.x)== 0?sgn(y-b.y)<0:x<b.x;
    }
    Point operator -(const Point &b)const {
        return Point(x-b.x,y-b.y);
    }
}
```

```

Point operator +(const Point &b) const {
    return Point(x+b.x,y+b.y);
}
Point operator *(const long long &k) const {
    return Point(x*k,y*k);
}
Point operator /(const long long &k) const {
    return Point(x/k,y/k);
}
//叉积
long long operator ^(const Point &b) const {
    return x*b.y - y*b.x;
}
//点积
long long operator *(const Point &b) const {
    return x*b.x + y*b.y;
}
/*当前点为p, 求角apb大小*/
long long rad(Point a,Point b) {
    Point p = *this;
    return fabs(atan2( fabs((a-p)^(b-p)), (a-p)*(b-p) ));
}
/*逆时针旋转90度*/
Point rotleft() {
    return Point(-y,x);
}
/*顺时针旋转90度*/
Point rotright() {
    return Point(y,-x);
}
//两点距离
long long dis(Point p) {
    return sqrt(sqr(x-p.x)+sqr(y-p.y));
}
long long dis2(Point p) {
    return sqr(x-p.x)+sqr(y-p.y);
}
//原点距离
long long abs() {
    return sqrt(abs2());
}
long long abs2() {
    return sqr(x)+sqr(y);
}
//改变向量长度
Point trunc(long long r) {
    long long l = abs();
    if(!sgn(l))return *this;
    r /= l;
    return Point(x*r,y*r);
}
//单位化
Point unit() {
    return *this/abs();
}
//IO

```

```

void input() {
    scanf("%lld%lld",&x,&y);
}
void output() {
    printf("%lld %lld\n",x,y);
}
//绕着p点逆时针旋转angle
Point rotate(Point p,long long angle) {
    Point v = (*this) - p;
    long long c = cos(angle), s = sin(angle);
    return Point(p.x + v.x*c - v.y*s,p.y + v.x*s + v.y*c);
}
};

//多边形面积，需保证A逆时针
long long area(vector<Point> A) {
    long long ans = 0;
    for (int i = 0; i < A.size(); i++) ans += (A[i]^A[(i + 1) % A.size()]);
    return ans / 2;
}

long long n,cnt;
vector<Point> p;
int main() {
    //freopen("2.in","r",stdin);
    int t;
    scanf("%d",&t);
    while(t--) {
        scanf("%lld",&n);
        p.clear();
        p.resize(n);
        for(int i = 0; i<n; i++) {
            p[i].input();
        }
        printf("%lld\n",-area(p));
    }
}

```

题目描述

观赛完运动会之后，Zhoues 需要立马回去钢厂上班，由于北航到钢厂的路径有很多条，他希望选择一条最短的！
但是貌似最短的路径有很多条，Zhoues 想知道有多少条最短的路径呢？
形式化来说，给一个 n 个点 m 条边的无向图，每条边的权值均为 1，求从点 s 到点 t 的最短路数量，两条最短路不同当且仅当包含了不同的边。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 10^6, 1 \leq m \leq 1.6 \times 10^6, 1 \leq s, t \leq n$)，含义同题目描述。
接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边 (u_i, v_i) ，所有边的权值均为 1。
图中有可能存在重边和自环，保证至少存在一条从点 s 到点 t 的路径。

输出格式

输出一行一个正整数，表示从点 s 到点 t 的最短路数量对 998 244 353 取模后的结果。

输入样例

```
5 7 1 5
1 2
1 3
2 4
3 4
2 3
4 5
4 5
```

输出样例

```
4
```

```
#include<cstdio>
#include<cstring>
#include<queue>
#define maxn 10000006
using namespace std;
int n,m,s,t;
int dis[maxn];
int ans[maxn];
struct Edge{
    int t,nxt,w;
}edge[maxn*4];
int book[maxn];
int head[maxn],tot=0;
priority_queue< pair<int,int> > hep;
void add(int st,int to,int we){
    edge[tot].t=to;
    edge[tot].nxt=head[st];
    edge[tot].w=we;
    head[st]=tot;
    tot++;
}
void init(){
    scanf("%d %d %d %d",&n,&m,&s,&t);
    memset(book,0,sizeof(book));
    memset(head,-1,sizeof(head));
    memset(dis,0x3f,sizeof(dis));
    memset(ans,0,sizeof(ans));
    for(int i=1;i<=m;i++){
        int a,b;
        scanf("%d %d",&a,&b);
        add(a,b,1);
        add(b,a,1);
    }
}
```

```

    }
    dis[s]=0;
    ans[s]=1;
    return;
}

void dij(){
    hep.push(make_pair(0-dis[s],s));
    while(!hep.empty()){
        int np=hep.top().second;hep.pop();
        if(book[np]) continue;
        book[np]=1;
        for(int i=head[np];i!=-1;i=edge[i].nxt){
            int w=edge[i].w;
            int t=edge[i].t;
            if(dis[t]>dis[np]+w){
                dis[t]=dis[np]+w;
                ans[t]=ans[np];
                hep.push(make_pair(0-dis[t],t));
            }
            else if(dis[t]==dis[np]+w){
                ans[t]+=ans[np];
                ans[t]%=998244353;
            }
        }
    }
    printf("%d\n",ans[t]);
    return;
}

int main(){
    init();
    dij();
    return 0;
}

```

F 剪刀石头布！

时间限制：1000ms 内存限制：65536kb

通过率：10/18 (55.56%) 正确率：10/51 (19.61%)

题目描述

BellalaBella 发现了一个剪刀石头布的新玩法，初始给定一个长度为 n 的剪刀石头布序列 S ，其中只有 S, R, P 三种字符，分别表示剪刀、石头与布。游戏会进行 $n - 1$ 轮，每一轮会根据上一轮的序列生成一个新的剪刀石头布序列，每一轮操作按如下方式进行：

- 设上一轮的序列（若为第一轮则为初始给定的序列）为 S ，这一轮生成的新序列为 T ，有 $|T| = |S| - 1$ 。
- 对于 $i = 1, 2, \dots, n - 1$ ，如果 $S_i = S_{i+1}$ 则有 $T_i = S_i$ ，否则 T_i 为 S_i 和 S_{i+1} 进行普通剪刀石头布游戏获胜的一方所对应的字符。

Bellalabella 发现最后一轮所生成的序列仅包含一个字符，她想知道具体是哪个字符。

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，一行一个由 S, R, P 组成的字符串 S ($1 \leq |S| \leq 10^6$)，含义同题目描述。

对于所有数据，保证 $|S|$ 之和不超过 3×10^6 。

输出格式

对于每组数据，输出一行一个字符，表示最后一轮生成序列所包含的字符。

输入样例

```
1
SRP
```

输出样例

```
P
```

```
#include <bits/stdc++.h>
#define maxn 1000086

using namespace std;

typedef long long ll;

int n, m;
vector<pair<int, int>> now;

char s[maxn];

int gettp(char x){
    if(x == 'S') return 0;
    if(x == 'R') return 1;
    return 2;
}

char getans(int x){
    if(x == 0) return 'S';
    if(x == 1) return 'R';
    return 'P';
}

bool check(int a, int b){
    if(a == 0 && b == 2) return 1;
    if(a == 1 && b == 0) return 1;
    if(a == 2 && b == 1) return 1;
    return 0;
}

void insert(pair<int, int> x){
    if(now.empty()) now.push_back(x);
```

```

else{
    pair<int, int> y = now[now.size() - 1];
    now.pop_back();
    if(y.second == x.second){
        y.first += x.first;
        now.push_back(y);
    }else{
        if(check(x.second, y.second)){
            x.first += y.first;
            insert(x);
        }
        else{
            now.push_back(y);
            now.push_back(x);
        }
    }
}
}

int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        now.clear();
        for(int i = 1; i <= n; i++){
            int tp = gett(s[i]);
            insert({1, tp});
        }
        printf("%c\n", getans(now[0].second));
    }
}

```

7

G 重建猫娘乐园

时间限制: 1000ms 内存限制: 131072kb

通过率: 7/7 (100.00%) 正确率: 7/12 (58.33%)

题目描述

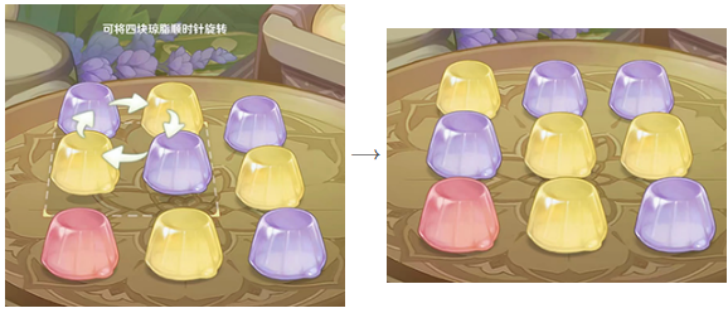
XIAO7 准备利用新抓的草兽来重建猫娘乐园，为了提高重建效率，她需要将「花花琼脂」调配成指定的组合以供草兽吸收，来激发草兽的潜能。请你帮她！

「花花琼脂」有三种颜色，分别用 0, 1, 2 表示，排成 3×3 的矩阵，你可以通过以下三种操作，调整「花花琼脂」的排列顺序，直至其变成指定的组合。

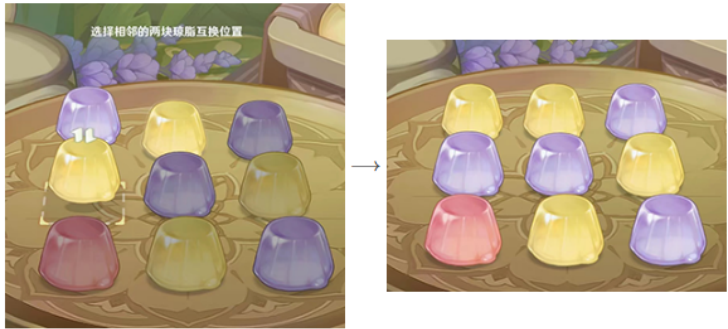
1. 「旋转」：花费 a 枚摩拉，选择任意一个 2×2 的子矩阵，将其顺时针旋转 90° 。例如：

「花花琼脂」有三种颜色，分别用 $0, 1, 2$ 表示，排成 3×3 的矩阵，你可以通过以下三种操作，调整「花花琼脂」的排列顺序，直至其变成指定的组合。

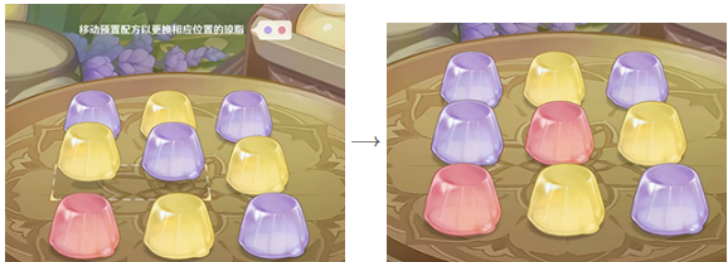
1. 「旋转」：花费 a 枚摩拉，选择任意一个 2×2 的子矩阵，将其顺时针旋转 90° 。例如：



2. 「对换」：花费 b 枚摩拉，选择相邻的两个「花花琼脂」，并交换它们的位置。例如：



3. 「预制」：花费 c 枚摩拉，选择一个 $n \times m$ 的子矩阵，将它预制成指定的样式。例如，若预制指定样式为：



由于重建猫娘乐园的花岗非常大 XIAO7 需要尽可能的节俭，所以请你告诉她，她需要花费的最少摩拉数。

输入格式

第一行三个正整数 a, b, c ($1 \leq a, b, c \leq 10^6$)，分别表示「旋转」、「对换」、「预制」三种操作分别需要花费的摩拉数。

第二行两个正整数 n, m ($1 \leq n, m \leq 3$)，表示「预制」操作可选择的子矩阵大小。

接下来一个 $n \times m$ 的矩阵，表示「预制」操作的指定样式。

接下来一个 3×3 的矩阵，表示「花花琼脂」的初始排列。

接下来一个 3×3 的矩阵，表示「花花琼脂」的最终目标排列。

上述三个矩阵的元素均在 $\{0, 1, 2\}$ 中。

输出格式

一行一个非负整数，表示 XIAO7 最少需要花费的摩拉数。

输入样例

```
1 1 1
3 1
0
0
0
0 0 0
1 1 1
2 2 2
1 0 0
1 0 2
2 2 1
```

输出样例

2

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
```

```

#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
#include<assert.h>
#include<queue>
using namespace std;
typedef long long ll;
typedef pair<ll, int> pi;
const int N = 20005;
const int M = 1000005;
const ll inf = 1ll << 60;
int getint(){
    char c; bool flag=0; int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9') {num=num*10+c-48; c=getchar();}
    return flag ? -num : num;
}
const int d[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
int cnt, fir[N], tar[M], nxt[M], w[M];
ll dis[N]; bool vis[N];
int Hash(int a[][3]) {
    int h = 0;
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            h = h * 3 + a[i][j];
    return h;
}
void reHash(int h, int a[][3]) {
    for(int i = 2; i >= 0; i--)
        for(int j = 2; j >= 0; j--)
            a[i][j] = h % 3, h /= 3;
}
void rotate(int x[][3], int y[][3], int si, int sj) {
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            y[i][j] = x[i][j];
    // si, sj          si, sj + 1
    // si + 1, sj      si + 1, sj + 1

    y[si][sj + 1] = x[si][sj];
    y[si + 1][sj + 1] = x[si][sj + 1];
    y[si + 1][sj] = x[si + 1][sj + 1];
    y[si][sj] = x[si + 1][sj];
}
void link(int a, int b, int c) {
    tar[++ cnt] = b, w[cnt] = c;
    nxt[cnt] = fir[a], fir[a] = cnt;
}
bool inarea(int x, int y) {
    return x >= 0 && x < 3 && y >= 0 && y < 3;
}

priority_queue<pi, vector<pi>, greater<pi> > q;

```

```

void dij(int x) {
    for(int i = 0; i < 19683; i++)
        dis[i] = inf;
    q.push(make_pair(dis[x] = 0, x));
    while(! q.empty()) {
        x = q.top().second, q.pop();
        if(vis[x]) continue;
        vis[x] = 1;

        for(int i = fir[x]; i; i = nxt[i])
            if(dis[tar[i]] > dis[x] + w[i]) {
                dis[tar[i]] = dis[x] + w[i];
                q.push(make_pair(dis[tar[i]], tar[i]));
            }
    }
}

int main() {
    int x[3][3], y[3][3], color[3][3];
    int a = getint(), b = getint(), c = getint();

    int n = getint(), m = getint();
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            color[i][j] = getint();

    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            x[i][j] = getint();

    int st = Hash(x), Ed = 19683;
    for(int s = 0; s < Ed; s++) {
        reHash(s, x);

        // Ðýxª
        rotate(x, y, 0, 0), link(s, Hash(y), a);
        rotate(x, y, 0, 1), link(s, Hash(y), a);
        rotate(x, y, 1, 0), link(s, Hash(y), a);
        rotate(x, y, 1, 1), link(s, Hash(y), a);

        // ¶Œ»»
        for(int i = 0; i < 3; i++)
            for(int j = 0; j < 3; j++)
                for(int k = 0; k < 3; k++)
                    if(inarea(i + d[k][0], j + d[k][1])) {
                        swap(x[i][j], x[i + d[k][0]][j + d[k][1]]);
                        link(s, Hash(x), b);
                        swap(x[i][j], x[i + d[k][0]][j + d[k][1]]);
                    }

        // ŒªŒ
        for(int i = 0; i + n - 1 < 3; i++)
            for(int j = 0; j + m - 1 < 3; j++) {
                for(int di = 0; di < n; di++)
                    for(int dj = 0; dj < m; dj++) {
                        y[di][dj] = x[i + di][j + dj];
                        x[i + di][j + dj] = color[di][dj];
                    }
            }
    }
}

```

```

    }
    link(s, Hash(x), c);

    for(int di = 0; di < n; di ++)
        for(int dj = 0; dj < m; dj ++)
            x[i + di][j + dj] = y[di][dj];
    }
}

dij(st);

for(int i = 0; i < 3; i ++)
    for(int j = 0; j < 3; j ++)
        x[i][j] = getint();
//cout << dis[Hash(x)] << '\n';
printf("%11d\n", dis[Hash(x)]);

return 0;
}

```

8

H 小水獭和科学抗疫 (Δ 版)

时间限制: 3000ms 内存限制: 65536kb

通过率: 21/22 (95.45%) 正确率: 21/43 (48.84%)

题目描述



小水獭幼儿园出现了疫情!

小水獭幼儿园可以被抽象为一个数轴, 其中共有 n 名学生, 第 i 名学生位于坐标 x_i , 任意两名学生的坐标均不相同。对于每名学生均有 $\frac{1}{2}$ 的概率被感染, 且每名学生是否被感染相互独立。

基于最新的科学研究, 如果第 i 名学生和第 j 名学生都被感染 ($i < j$), 那么需要额外投入 $2^{x_i - x_j}$ 的核酸费用。设核酸费用之和的数学期望为 C , 容易证明 $4C$ 是一个整数, 请你求解 $4C$ 对 998 244 353 取模后的结果。

同学们众志成城, 根据科学精准防控, 一定能战胜疫情!

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正整数 n ($2 \leq n \leq 10^5$), 含义如题目所示。

第二行 n 个正整数 $x_1, x_2, \dots, x_n$ ($1 \leq x_i \leq 10^5$), 含义如题目所示。保证 $x_i \neq x_j$ 对 $1 \leq i < j \leq n$ 成立。

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998 244 353 取模后的结果。

样例输入: 2
4
1 2 4 8
3
1 50000 100000

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998 244 353 取模后的结果。

输入样例

```

2
4
1 2 4 8
3
1 50000 100000

```

输出样例

```

222
227493521

```

```
#include<bits/stdc++.h>
```

```

using namespace std;
typedef long long LL;
const int maxn = 1e5 + 5;
const LL mod = 998244353;

LL x[maxn], POW[maxn], ans[maxn];

int main() {
    POW[0] = 1;
    for(int i=1; i<maxn; i++) POW[i] = (POW[i-1] * 2) % mod;

    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        for(int i=1; i<=n; i++) scanf("%lld", &x[i]);
        sort(x+1, x+1+n);
        ans[n-1] = POW[x[n]-x[n-1]];
        for(int i=n-2; i>=1; i--) ans[i] = (POW[x[i+1]-x[i]] * (1 + ans[i+1])) %
mod;

        int res = 0;
        for(int i=1; i<=n-1; i++) res = (res + ans[i]) % mod;
        printf("%lld\n", res);
    }

    return 0;
}

```

9

I 小水獭和紧急演习

时间限制: 1000ms 内存限制: 65536kb

通过率: 125/128 (97.66%) 正确率: 125/330 (37.88%)

题目描述



加帕里幼儿园正在预防出现宇宙爆炸!

加帕里幼儿园可以被抽象为一个 m 维欧氏空间, 其中有 n 个实验室, 第 i 个实验室的坐标为 $(x_{i,1}, x_{i,2}, \dots, x_{i,m})$.

加帕里幼儿园即将紧急演习, 为计算财政支出, 他们请小水獭帮忙计算一下加帕里幼儿园的直径, 即任意两所实验室之间曼哈顿距离的最大值。

在 m 维欧氏空间中, 两点 $(x_1, x_2, \dots, x_m)$ 和 $(y_1, y_2, \dots, y_m)$ 的曼哈顿距离为:

$$\sum_{i=1}^m |x_i - y_i|$$

小水獭的浏览器弹窗了, 无法前往实地计算, 你能帮帮它嘛?

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$), 表示数据组数。

对于每组数据, 第一行两个正整数 n, m ($2 \leq n \leq 10^4, 1 \leq m \leq 5$), 含义同题目描述。

接下来 n 行, 第 i ($1 \leq i \leq n$) 行 m 个整数 $x_{i,1}, x_{i,2}, \dots, x_{i,m}$ ($-10^9 \leq x_{i,j} \leq 10^9$), 表示第 i 个实验室的坐标。保证没有两个不同的实验室位于相同的位置。

输出格式

对于每组数据, 输出一行一个正整数, 表示任意两所实验室之间曼哈顿距离的最大值。

输出格式

对于每组数据，输出一行一个正整数，表示任意两所实验室之间曼哈顿距离的最大值。

输入样例

```
2
2 1
1
2
3 2
1 1
2 2
3 3
```

输出样例

```
1
4
```

```
#include <stdio.h>
#define max(a, b) ((a) > (b) ? (a) : (b))

int main()
{
    int t, n, m, a[5], i, j, k;
    long long val[32], x, ans;

    scanf("%d", &t);
    while (t--)
    {
        scanf("%d%d", &n, &m);
        for (i = 0; i < (1 << m); i++)
            val[i] = -1e18;
        for (i = 1; i <= n; i++)
        {
            for (j = 0; j < m; j++)
                scanf("%d", &a[j]);
            for (j = 0; j < (1 << m); j++)
            {
                x = 0;
                for (k = 0; k < m; k++)
                {
                    if (j & (1 << k))
                        x += a[k];
                    else
                        x -= a[k];
                }
                val[j] = max(val[j], x);
            }
        }
        ans = -1e18;
        for (i = 0; i < (1 << m); i++)
            ans = max(ans, val[i] + val[i ^ ((1 << m) - 1)]);
        printf("%lld\n", ans);
    }

    return 0;
}
```

J 小水獭和机动冈达战士

时间限制: 1000ms 内存限制: 65536kb

通过率: 7/8 (87.50%) 正确率: 7/24 (29.17%)

题目描述



小水獭正在重温「机动冈达战士：铁血の达因斯尼夫」，它决定代替西洛驾驶「流星号」前往目标点狙击莱斯特尔·艾里安。

小水獭所在的战场可以被抽象成一个 $n \times m$ 的矩阵，它现在处于 $(1, 1)$ ，目标点为 (n, m) ，除了这两点外，每一点都有一个敌方机动战士，位于 i 行 j 列的敌方机动战士的战斗力为 $a_{i,j}$ 。

小水獭每次可以向下移动或向右移动，并击败沿途的所有敌方机动战士，它希望一路上击败的敌方机动战士的战斗力之和最小，这样才能保证最高的战力来进行狙击。

然而，残忍的冈达使用冈达瞄准了一个子矩阵区域，使得小水獭不能经过这个子矩阵，否则它会被立刻粉碎。

现在，奥尔如提前获知了 q 种冈达可能选择的子矩阵，三日月希望你可以帮助小水獭算一算：对于每种情况，它能否到达目标点；如果可以到达目标点，那么如果选择最优路线，一路上击败的敌方机动战士的战斗力之和的最小值是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行三个正整数 n, m, q ($2 \leq n, m \leq 200, 1 \leq q \leq 5 \times 10^4$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个非负整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($0 \leq a_{i,j} \leq 10^9$)，含义同题目描述。保证 $a_{1,1} = a_{n,m} = 0$ 。

接下来 q 行，第 i ($1 \leq i \leq q$) 行四个正整数 x_1, y_1, x_2, y_2 ($1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m$)，表示第 i 种情况冈达所选择的子矩阵的左上角 (x_1, y_1) 和右下角 (x_2, y_2) 。

输出格式

对于每组数据，输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数，表示对于第 i 种情况，如果小水獭可以到达目标点，输出它一路上击败的敌方机动战士的战斗力之和最小值；否则，输出 -1 。

输出格式

对于每组数据，输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数，表示对于第 i 种情况，如果小水獭可以到达目标点，输出它一路上击败的敌方机动战士的战斗力之和最小值；否则，输出 -1 。

输入样例

```
2
3 3 3
0 1 2
3 4 5
6 7 0
2 1 2 2
1 1 2 2
1 2 3 2
4 4 3
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 0
2 2 3 3
2 1 2 3
3 2 3 4
```

输出样例

```
8
-1
-1
24
24
51
```

```
#include <stdio.h>

#define maxn 205
#define min(a, b) (a < b ? a : b)

typedef long long ll;

int main()
{
    int a[maxn][maxn], t, n, m, q, i, j, x1, y1, x2, y2;
    ll f[maxn][maxn], g[maxn][maxn], pre[maxn][maxn], suf[maxn][maxn], ans;

    scanf("%d", &t);
    while (t--)
    {
        scanf("%d%d%d", &n, &m, &q);
        for (i = 1; i <= n; i++)
```

```

{
    for (j = 1; j <= m; j++)
    {
        scanf("%d", &a[i][j]);
        f[i][j] = g[i][j] = 1e18;
    }
}
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= m; j++)
    {
        if (i == 1 && j == 1) f[i][j] = 0;
        if (i - 1 > 0) f[i][j] = min(f[i][j], f[i - 1][j] + a[i][j]);
        if (j - 1 > 0) f[i][j] = min(f[i][j], f[i][j - 1] + a[i][j]);
    }
}
for (i = n; i; i--)
{
    for (j = m; j; j--)
    {
        if (i == n && j == m) g[i][j] = 0;
        if (i + 1 <= n) g[i][j] = min(g[i][j], g[i + 1][j] + a[i][j]);
        if (j + 1 <= m) g[i][j] = min(g[i][j], g[i][j + 1] + a[i][j]);
    }
}
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= m; j++)
    {
        if (j == 1)
            pre[i][j] = f[i][j] + g[i][j] - a[i][j];
        else
            pre[i][j] = min(pre[i][j - 1], f[i][j] + g[i][j] - a[i][j]);
    }
    for (j = m; j; j--)
    {
        if (j == m)
            suf[i][j] = f[i][j] + g[i][j] - a[i][j];
        else
            suf[i][j] = min(suf[i][j + 1], f[i][j] + g[i][j] - a[i][j]);
    }
}
while (q--)
{
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
    if (x2 - x1 + 1 == n || y2 - y1 + 1 == m || (x1 == 1 && y1 == 1) ||
(x2 == n && y2 == m))
    {
        puts("-1");
        continue;
    }
    ans = 1e18;
    if (x2 < n && y1 > 1) ans = min(ans, pre[x2 + 1][y1 - 1]);
    if (x1 > 1 && y2 < m) ans = min(ans, suf[x1 - 1][y2 + 1]);
    printf("%lld\n", ans);
}

```

```
}  
  
    return 0;  
}
```